Faculty of Engineering

Department of Software Engineering

Smart HR Recruitment System

(intelljob)

A senior 2 project report - submitted to complete the requirements

for obtaining a bachelor's degree in informatics engineering

Prepared by

Kasem Al Kilani Abdullah Khadem Aljamee Yasser Dukmak

Supervised by

Dr. Riad Sonbol

Eng. Anas Abdulaziz

2024-2025

SUPERVISION CERTIFICATION

Supervisor Certification

I Certify that the preparation of this project entitled **Smart HR Recruitment System (intelljob)**

prepared by **Kasem al Kilani, Abdullah Khadem Al Jamee, Yasser Dukmak** was made under my supervision at Faculty of Informatics Engineering in partial Fulfillment of the Requirements for the Degree of Bachelor of Software and Information System Engineering and AI & Data Science.

Name: .....

Signature: .....

Name: .....

Signature: .....

Date: 1/03/2025

شكر و تقدير ACKNOWLEDGEMENTS

إلى أستاذي الدكتور رياض سنبل، والمهندس أنس عبد العزيز.

To Dr. Riad Sonbol and Eng. Anas Abdulaziz

ABSTRACT

In the fast-evolving and competitive world of human resource management, optimizing the hiring process by aligning job opportunities with the right candidates is essential for increasing recruitment efficiency and ensuring successful job placements. Businesses and organizations require a smart and efficient recruitment system that automates job matching, facilitates seamless communication between employers and job seekers, and enhances decision-making through data-driven insights.

This report presents **IntellJob**, an intelligent HR recruitment system designed to streamline the hiring process by leveraging advanced AI-driven algorithms. The system allows businesses to post job vacancies, filter applications, and track the hiring process efficiently. Additionally, it provides automated job recommendations to candidates, helping them find suitable positions that align with their skills and experience.

IntellJob is particularly beneficial for industries that require dynamic hiring processes, such as the restaurant sector, where hiring demands vary frequently. The system enables businesses to handle different order types, automate shortlisting, and analyze recruitment trends using AI-based analytics. Through real-time notifications, performance tracking, and role-based access control, **IntellJob** enhances the recruitment experience for both employers and job seekers.

This report details the system's core functionalities, technological framework, challenges, and future improvements to make **IntellJob** a leading solution in the HR recruitment domain.

ملخص

في عالم إدارة الموارد البشرية سريع التغير والتنافس، يعد تحسين عملية التوظيف من خلال مواءمة فرص العمل مع المرشحين المناسبين أمرًا بالغ الأهمية لزيادة كفاءة التوظيف وضمان عمليات توظيف ناجحة. تحتاج الشركات والمؤسسات إلى نظام توظيف ذكي وفعال يقوم بأتمتة عملية مطابقة الوظائف، وتسهيل التواصل بين أصحاب العمل والباحثين عن العمل، وتعزيز اتخاذ القرار من خلال التحليلات القائمة على البيانات.

يقدم هذا التقرير نظام IntellJob، وهو نظام توظيف ذكي مصمم لتبسيط عملية التوظيف من خلال الاستفادة من الخوارزميات الذكية وتقنيات الذكاء الاصطناعي. يتيح النظام للشركات نشر الوظائف الشاغرة، وتصنيف الطلبات، ومتابعة عملية التوظيف بفعالية. بالإضافة إلى ذلك، يوفر توصيات وظائف آلية للمرشحين، مما يساعدهم على العثور على فرص عمل مناسبة تتماشى مع مهاراتهم وخبراتهم.

يعد IntellJob مفيدًا بشكل خاص للقطاعات التي تتطلب عمليات توظيف ديناميكية، مثل قطاع المطاعم، حيث تتغير متطلبات التوظيف بشكل متكرر. يوفر النظام للشركات القدرة على التعامل مع أنواع مختلفة من طلبات التوظيف، وأتمتة تصفية المرشحين، وتحليل اتجاهات التوظيف باستخدام التحليلات المستندة إلى الذكاء الاصطناعي. من خلال التنبيهات الفورية، وتتبع الأداء، والتحكم في الوصول المستند إلى الأدوار، يعزز IntellJobتجربة التوظيف لأصحاب العمل والباحثين عن وظائف.

يسلط هذا التقرير الضوء على الوظائف الأساسية للنظام، والإطار التكنولوجي المستخدم، والتحديات التي يواجهها، بالإضافة إلى التحسينات المستقبلية لجعل IntellJob الحل الرائد في مجال التوظيف الذكي.

TABLE OF CONTENT

Table of Contents

List of abbreviations

Table 1list of abbreviations

Abbreviation	Definition
API	Application Program Interface
NLP	Natural Language Processing
AI	Artificial Intelligence
DL	Deep Learning
ML	Machine Learning
DM	Data Mining
UML	Unified Modeling Language
NoSQL	Not only Structured Query Language.
SQL	Structured Query Language.
SOW	Statement of Work

NER	Named Entity Recognition
ATS	applicant tracking systems
CBRS	Content-Based Recommender System
CFRS	Collaborative Filtering-Based Recommendation System
RS	Recommendation System
LM	Language Model
SRS	Software Requirements Specification
RTM	Requirements Traceability Matrix
DB	Database
JPA	Java Persistence API
DI	Dependency Injection
DTO	Data Transfer Object
JWT	JSON Web Token

Chapter1 Introduction

1.1. Introduction:

In this chapter, we will introduce our project, discussing the main issues and reasons for building this system. We will also explain the objectives and goals we aim to accomplish with this system. Finally, we will provide an overview the report organization.

1.2. Problem Definition:

In the modern job market, recruitment processes face significant challenges, including inefficient hiring workflows, difficulty in matching candidates with the right job opportunities, and delays in candidate selection. Employers struggle to manage large volumes of applications effectively, leading to lost opportunities and prolonged hiring periods. Similarly, job seekers often face difficulties in finding suitable positions that align with their skills and experience.

To stay competitive, companies require a smart system that can automate and enhance recruitment processes. Traditional hiring methods are time-consuming and often fail to provide optimal candidate-job matches, resulting in lower job satisfaction and higher turnover rates. Moreover, the lack of **AI-driven insights** in recruitment limits the ability of businesses to make data-informed hiring decisions.

Without an efficient and intelligent system to manage and analyze job applications, hiring managers may struggle to find the best talent quickly

and effectively. The absence of an advanced **AI-powered recruitment system** leads to inefficiencies, missed opportunities, and increased recruitment costs. Therefore, there is a strong need for a robust and intelligent HR solution that streamlines hiring, optimizes job matching, and enhances decision-making for recruiters and job seekers alike.

1.3. Project objectives:

The primary objective of **IntellJob** is to develop a **smart HR recruitment system** that enhances hiring efficiency, ensures precise job-candidate matching, and provides intelligent insights for better decision-making.

To accomplish this, IntellJob will:

- Utilize **AI-driven algorithms** to optimize job-candidate matching based on skills, experience, and employer requirements.
- Provide **automated job recommendations** for candidates, enhancing job discovery.
- Implement a **real-time application tracking system (ATS)** to streamline hiring workflows for employers.
- Offer **data-driven analytics and insights**, enabling companies to refine their recruitment strategies.
- Enable businesses to handle various order types, including restaurant-specific hiring needs.

- Enhance recruitment processes through automated shortlisting and resume screening.
- Deliver an intuitive and user-friendly React-based interface for seamless interaction.

Ultimately, **IntellJob** aims to revolutionize the hiring process by making it smarter, faster, and more efficient, ensuring that businesses find the right talent while job seekers access better career opportunities.

1.4. Report organization:

The report will be organized in the following chapters:

- Chapter 1: introduction.
- Chapter 2: Basic concepts and literature review.
- Chapter 3: project management.
- Chapter 4: System analysis.
- Chapter 5: design study for the system
- Chapter 6: practical implementation.
- Chapter 7: Evaluations, Results and Discussions.
- Chapter 8: Conclusion.

Chapter2 Basic Concepts and Literature Review

2.1. Introduction:

In this chapter, we provide an overview of the key concepts and technologies that form the foundation of the Smart HR Recruitment System (IntellJob). The system leverages Artificial Intelligence (AI), microservices architecture, and modern web development technologies to enhance job recruitment, resume parsing, and candidate evaluation.

To ensure the system is built on a strong technical and theoretical foundation, we explore basic concepts related to AI-driven job matching, recruitment platforms, and microservices. Additionally, we conduct a literature review of existing systems, analyzing their strengths and limitations to improve the design and functionality of our system.

This chapter provides insight into the technologies and methodologies that drive IntellJob, helping to understand how modern recruitment solutions operate and how our system advances traditional hiring methods.

2.2. Basic concepts:

The Smart HR Recruitment System integrates several key concepts to provide a seamless and AI-enhanced hiring experience. Below are the fundamental principles that shape the system:

2.2.1. Recruitment Platforms

Recruitment platforms are digital solutions that connect job seekers with employers, facilitating the hiring process through job postings, applications,

and candidate evaluations. Traditional recruitment systems often rely on manual job matching and resume reviews, leading to inefficiencies.

Our system enhances this process by integrating AI-powered job matching and automated resume parsing, ensuring employers find the most relevant candidates quickly.

2.2.2. AI in Recruitment

Artificial Intelligence plays a crucial role in modern hiring solutions, allowing systems to:

- **Analyze job descriptions and resumes** for relevant skills and experience.
- **Match candidates to job postings** based on AI-driven recommendation algorithms.
- **Generate skill-based quizzes** to assess applicant qualifications.

By integrating AI-powered resume parsing and automated job matching, our system enhances recruitment efficiency and accuracy.

2.2.3. Microservices Architecture

The microservices architecture is a software development approach where applications are divided into independent, loosely coupled services that communicate via APIs.

Advantages of microservices in our system:

- **Scalability:** Each service (e.g., User Management, Job Posting, AI Processing) operates independently, allowing easy updates.

- **Flexibility:** Services can be developed in different programming languages (Django, FastAPI, Spring Boot).
- **Improved performance:** Workloads are distributed across microservices, improving system responsiveness.

Our recruitment system leverages microservices to ensure modularity, better maintainability, and enhanced scalability.

2.2.4. Resume Parsing

The rapid growth of job applications has necessitated automated approaches for candidate screening

So, we worked on an AI-based resume parsing module designed to extract structured information — such as names, emails, skills, education, and work experience — from unstructured resume data.

That's why we've developed an AI-based resume parsing module designed to extract structured information – such as names, emails, skills, education, and work experience – from unstructured resume data, using the latest Natural Language Processing (NLP) technologies and models. This addition not only reduces the effort of manual screening, but also enhances the accuracy and consistency of candidate evaluation.

2.2.5. AI-Generated Quiz System

One of the unique features of IntellJob is its automated quiz generation system, modern recruitment platforms increasingly rely on automated assessments to gauge the technical and practical abilities of job candidates. This service develops a Test/Question Generation module that automatically

produces multiple-choice and true/false questions based on specific skills, difficulty levels, and question types. Using a transformer-based model accessed via a local Ollama client (e.g., Deepseek r1:1.5b) and integrated into a FastAPI application, the system generates questions in a strict JSON format.

- AI analyzes job descriptions and applicant skills to generate relevant quiz questions.
- This helps recruiters evaluate candidate expertise before scheduling interviews.
- It provides objective assessments to ensure fair candidate evaluation.

By combining AI job matching, resume parsing, and automated assessments, our system modernizes recruitment and reduces manual effort in candidate evaluation.

2.2.6. Job Recommendation

Modern recruitment platforms aim to streamline candidate selection and improve job matching. A crucial component in these systems is a job recommendation engine that can accurately assess how well a candidate's qualifications align with job requirements. The goal of this project is to build a job recommendation module that uses pre-trained Sentence Transformer models to generate embeddings for job descriptions and applicant profiles. By computing the cosine similarity between these embeddings, the system produces a similarity percentage that indicates the degree of match. This report explains the technical and academic foundations of the approach, describes the system architecture and

implementation details, and discusses the evaluation and future improvements.

2.2.6.1. RS approaches:

I. Content-Based Recommender System (CBRS):

Generally, user preferences are compared against the contents of the items. Characteristics of the user interest can be extracted explicitly from questionnaires or implicitly by learning user's transactional behavior over time

Some of the problems associated with CBRS are limited content analysis, difficult to apply non-textual data, overspecialization by recommending only items that bear strong resemblance to those the user already knows and sparsity of data

II. Collaborative Filtering-Based Recommendation System (CFRS)

CFRS are based on the assumptions that people have similar preferences and interests that we can predict their choice according to their past preferences

two categories of CFRS:

i. Memory-Based Collaborative Filtering Algorithm

Performances of these algorithms are mainly based on time takes to search for neighbors among a large population of potential neighbors

There are two types of memory-based CF algorithms discussed in the literature, namely:

a. user-based Collaborative Filtering

User-based CF recommends items to the active user based on the user's past preferences or on the opinions of their closest neighbors or similar users.

user-based CF systems calculate similarities among users using some measures to find the similar users.

The next step is to select user neighbors based on these similarities and then combine the neighbor users' rating score.

b. Item-Based Collaborative Filtering

item-based algorithms provide dramatically better performance than user-based algorithm.

In these systems, two items are similar when they received similar ratings from different users. The target item is compared with the set of similar items rated by the target user and compute how similar they are to the target item. Hence, the recommendation is a two-step process. First, find similar items by calculating similarity Sij between the co-rated item i and item j, by considering all users who rated both i and j, next step is to compute the predictions by taking weighted average of rating given to most similar items by the active user

ii. Model-Based Collaborative Filtering Algorithm

Model-based approach use algorithmic approach to performs recommendations.

These algorithms have been developed to overcome some limitations in data-intensive commercial RS such as prediction speed and over-fitting in the memory-based algorithms.

The model-based CF algorithms compress huge databases/dataset of ratings into a model which will positively contributes to the efficiency of prediction. These algorithms can be Data Mining (DM) algorithms including Machine Learning (ML), Deep Learning (DL) and Association Rule Mining (ARM)

precision of CFRS is very sensitive to the number of ratings in rating matrix and CFRS do not consider personal characteristics of the user or the domain knowledge.

III. Knowledge-Based Recommender System (KBRS)

The knowledge-based approaches use domain knowledge (knowledge about users and products) to generate recommendations which meet user preferences.

The main drawbacks in KBRS are difficulty of acquisition and representing the domain knowledge in a machine-readable structure and it needs extensive efforts to extract the knowledge. However, KBRS are more appropriate for designing course RS which need complex knowledge domain to make recommendations.

IV. Hybrid Techniques

Hybrid RS combine different recommendation techniques to overcome the limitations in single technique

Researchers have focused on incorporating different recommendation techniques in a single course RS to improve quality of final recommendation

i. Combining CFRS and CBRS:

This approach combines CFRS and CBRS recommenders and perform final recommendation at the end

The combination can be performed by various methods including weighted

ii. Developing a RS combining DM and statistical methods with CFRS

This type of hybrid RS techniques use DM and statistical methods as the main technique and combines with CFRS. These systems apply features of DM to analyze and discover hidden patterns in the large databases to uncover student behavior or learning patterns to provide personalize course recommendation to users

2.2.6.2. Text Data Representation Techniques

TF-IDF (Term Frequency-Inverse Document Frequency) and word embeddings are both techniques used to represent text data in numerical form for various tasks in natural language processing (NLP), but they work in fundamentally different ways and serve different purposes:

TF-IDF:

- Sparse Representation: TF-IDF converts text into high-dimensional vectors where each dimension corresponds to a unique word in the corpus. The vectors are sparse because any given document only uses a small subset of the words in the corpus.

- Term Frequency: TF-IDF reflects how frequently a term occurs in a document (TF) relative to its frequency across all documents in the corpus (IDF). It gives higher weight to terms that are more specific to a document.

- Context Ignorance: TF-IDF does not capture the context in which words appear; it treats each term independently. It doesn't account for the order of words or their semantic relationships.

- Suitability: It is suitable for tasks that require keyword importance and document similarity based on term frequency, such as document classification and information retrieval.

Word Embeddings:

- Dense Representation: Word embeddings map words into a continuous vector space where semantically similar words are mapped to nearby

points. Vectors are dense, with each dimension representing a latent feature that may capture some aspect of linguistic meaning.

- Semantic Meaning: Embeddings are trained to capture the context of words, so words that are used in similar contexts will have similar embeddings. This captures synonyms and different forms of a word.

- Order and Relationships: While individual word embeddings don't capture word order, the way they're used in models can.

- Suitability: Embeddings are particularly useful for tasks that require an understanding of semantic meaning, such as machine translation, sentiment analysis, and more sophisticated recommendation systems.

TF-IDF is a simple but effective way to represent the importance of words in documents, while embeddings provide a more accurate representation that captures the semantic relationships between words. Depending on the system architecture and after experimenting with both techniques, we found that in recommender systems, embeddings made more insightful suggestions because they could capture similarity between items that do not share specific keywords but are related by context and meaning.

2.3. Literature review:

To develop a modern and efficient hiring system, we studied existing recruitment platforms and analyzed their features, limitations, and technologies. This helped us design IntellJob to address gaps in existing solutions.

2.3.1. Existing Recruitment Platforms and Their Limitations

Below is an overview of existing hiring platforms, their functionalities, and the challenges they present:

System Name	Features	Limitations
LinkedIn Jobs	- Job posting and application tracking. - Candidate messaging. - AI-powered job recommendations.	- AI matching is not highly customized for specific employers. - No automated quiz generation. - Manual resume screening required.
Indeed	- Resume searching and job posting. - Applicant tracking system (ATS). - Employer dashboard.	- No AI-driven resume parsing. - Lacks real-time employer-candidate communication.
ZipRecruiter	- AI-driven job suggestions. - Employer candidate search tools.	- AI matching is limited in accuracy. - No real-time chat features.

Taleo (by Oracle)	- Enterprise-level recruitment system. - End-to-end hiring management.	- High cost and complexity. - Requires manual review of applicants.

Our system **improves upon these limitations** by:

✔ **Using AI-powered resume parsing** to extract applicant information efficiently.

✔ **Automating job matching** based on **skills, experience, and employer preferences**.

✔ **Providing real-time chat features** for direct employer-applicant interactions.

✔ **Generating AI-powered quizzes** to assess candidate knowledge.

2.3.2. Similar Systems

System/Feature	Managing the recruitment process	Connecting companies and job seekers	Resume Parsing	match job using AI	Generate tests using AI
jobscan.co	yes	no	yes	yes	no
forsa.com	yes	yes	no	no	no
LinkedIn.com	yes	yes	no	no	no
akoono.com	yes	yes	no	no	no
Our System	yes	yes	yes	yes	yes

2.3.3. Technologies Used in Recruitment Systems

To build a modern, scalable recruitment platform, we analyzed technologies used in industry-standard hiring platforms.

Technology	Usage in Recruitment Platforms
Machine Learning (ML)	Used for job matching and applicant ranking based on AI predictions.
Natural Language Processing (NLP)	Used for resume parsing and keyword extraction in job descriptions.
Microservices Architecture	Allows scalable and modular development, improving performance.
WebSockets & Real-Time Communication	Enables live chat and instant notifications between recruiters and applicants.
Relational and NoSQL Databases	MySQL for structured job listings and applications, MongoDB for unstructured AI-generated data.

Our system integrates these modern technologies to ensure efficient, AI-powered recruitment workflows.

2.3.4. How Our System Improves Traditional Recruitment

IntellJob improves traditional hiring systems by:

- Enhancing AI-driven job matching for more accurate candidate recommendations.
- Automating candidate evaluation using AI-generated quizzes and resume analysis.
- Enabling real-time communication to reduce hiring delays.
- Offering a scalable, microservices-based architecture for flexible system growth.

By learning from existing recruitment platforms and leveraging modern AI and microservices technologies, our system aims to redefine the hiring process for both job seekers and employers.

2.3.5. Reference Studies

2.3.5.1. Resume Parsing

There are many points that have been benefited from after researching many previous research papers, including:

- **Text Analytics for Resume Parsing:**
  Das et al. (Das, 2019) discuss the application of big data text analytics techniques to convert unstructured resumes into structured data. Their work demonstrates that automated text processing can significantly enhance data extraction accuracy and reduce processing time.
- **Entity Extraction with NLP:**
  Celik et al. (Çelik, 2019) propose an ontology-based approach for resume parsing that leverages named entity recognition (NER) to match resumes

with job descriptions. Their study highlights the importance of precise entity extraction in the recruitment process.

- **Deep Learning for Information Extraction:**
  Jadhav et al (Jadhav, 2016) illustrate the use of deep learning models to extract pertinent information from resumes, demonstrating improvements in recall and precision compared to traditional rule-based methods.
- **Hybrid Approaches in Resume Parsing:**
  Mittal et al. (Mittal, 2018) present a hybrid model combining rule-based methods with machine learning to extract candidate details from CVs. Their research emphasizes the benefits of integrating multiple techniques to handle the variability in resume formats.
- **Real-Time Processing and Scalability:**
  Banerjee et al. (Banerjee, 2020)examine scalable solutions for real-time CV parsing using distributed computing frameworks. Their findings are crucial for implementing systems that can handle high volumes of applications in enterprise settings.

2.3.5.2. AI-Generated Quiz System

The field of automatic question generation has evolved rapidly with advances in deep learning and natural language processing. Several research works have contributed to this domain:

1. **Transformer-based Question Generation:**
   Devlin et al. (2019) introduced BERT, which paved the way for using transformer architectures to capture contextual information. Later, models such as T5 and GPT-Neo have been applied to generate questions by fine-tuning on domain-specific datasets (Devlin, 2019)

2. **Structured Output Generation:**
   Research by Zhou et al. (2020) explored using controlled natural language generation techniques to produce output in a fixed format (e.g., JSON). This work emphasizes the importance of prompt engineering to constrain model outputs (Zhou, 2020)
3. **Hybrid Approaches:**
   Mittal et al. (2018) proposed a hybrid approach that combines rule-based systems with machine learning to generate questions, ensuring both consistency and creativity in the generated questions (Mittal V. , 2018)
4. **Evaluation Metrics in Question Generation:**
   Studies by Du et al. (2017) have introduced metrics like BLEU, ROUGE, and human evaluation to assess the quality of generated questions. These metrics help quantify the relevance, grammaticality, and diversity of questions (Du, 2017)
5. **Application in Recruitment Contexts:**
   Recent work in recruitment technology leverages automated question generation to provide dynamic assessments tailored to specific job roles, as seen in projects that integrate AI with applicant tracking systems

2.3.5.3. Job Recommendation

Several research efforts and studies have explored **embedding-based recommendation systems** in recruitment and related domains. Key contributions include:

1. **Semantic Matching for Job Recommendation:**
   Li et al. (2018) proposed a job recommendation system using word and sentence embeddings to capture semantic similarities between job descriptions and candidate profiles. Their work demonstrates that high-

quality embeddings improve matching accuracy by representing contextual information effectively. (Li, 2018)

2. **Embedding-Based Candidate Matching:**
   Research by Huang et al. (2019) uses deep learning techniques to generate embeddings from candidate resumes and job postings, subsequently computing cosine similarity to rank candidates for job openings. This approach shows that vector-based matching can capture nuanced relationships between skills and job requirements. (Huang, 2019)
3. **Transformer Models for Text Embedding:**
   Devlin et al. (2019) introduced BERT, and subsequent models such as Sentence Transformers (Reimers & Gurevych, 2019) have proven effective for generating contextual embeddings. These models have been widely used for semantic similarity tasks and recommendation systems. (Devlin J. , 2019) (Reimers, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.", 2019)
4. **Scalability and Efficiency in Recommendation Systems:**
   Studies by Koren et al. (2009) and subsequent work in collaborative filtering highlight the importance of scalable algorithms in recommendation engines. Although many of these studies target product recommendations, similar principles apply to job recommendations. (Koren, 2009)
5. **Comparison of Embedding Models:**
   Recent comparative studies have shown that models like all-MiniLM-L6-v2 offer a good balance between speed, resource efficiency, and output quality, making them suitable for real-time recommendation systems. (Reimers & Gurevych, 2019)

Chapter3 Project Management

3.1. Introduction:

Project management plays a crucial role in ensuring the efficient execution and delivery of the Smart HR Recruitment System. The project follows an Agile and Incremental approach and is developed using sprint-based iterations. These sprints allow for continuous progress, iterative refinement, and adaptability to changes, ensuring that the system is built efficiently and systematically.

The project is structured into multiple sprints, with each sprint delivering a functional increment of the system. Unlike Scrum, where there are rigid sprint ceremonies (stand-ups, retrospectives, and product backlog grooming), this project follows customized sprints focused on achieving key milestones with continuous improvement and flexibility.

This chapter covers the Project Charter, Scope of Work (SOW), Sprint-Based Development Plan, and Risk Management, providing a structured approach to managing the project.

3.2. Project charter:

Project Title	Smart HR Recruitment System
Project Start Date	Sep 16, 2024
Project Finish Date	Feb 23, 2025
Project Manager	Dr. Riad Sonbol
Project Methodology	**Agile and Incremental Development**

3.2.1. Stakeholders

Stakeholder	Role	Responsibility
Dr. Riad sonbol	Project Manager – Supervisor	Guides the project and provides feedback.
Kasem Al Kilani	Software Engineer	Developed User Management, Job Application, AI Processing, Notification Services, and Frontend (React.js).
Abdullah Khadem Aljamee	AI Engineer	Developed CV Parsing, AI-powered Resume Processing, and Quiz Generation. Researched AI methodologies and integrated them into the system.
Yasser Dukmak	Software Engineer	Developed backend of Quiz Management, Communication Service, and Job Posting. Responsible for database structuring and optimizing backend operations.

3.3. The SOW document:

3.3.1. Project Overview

The **Smart HR Recruitment System** is a student-developed project aimed at enhancing the hiring process through automation and AI-driven solutions. The system enables job seekers to find suitable opportunities and employers to efficiently recruit candidates by leveraging AI-based resume parsing, automated quiz generation, and real-time communication.

The system is designed using a microservices architecture, ensuring modularity, scalability, and independent service execution. It consists of key functionalities such as user management, job posting, job applications, AI-powered job matching, quiz generation, and real-time chat features.

As students, our goal is to apply industry-standard practices while keeping the system practical and achievable within the project scope.

3.3.2. Project Objectives

- Develop a **functional recruitment platform** using **Django, Spring Boot, FastAPI, and React.js**.
- Implement **secure authentication and role-based access** for job seekers and employers.
- Integrate **AI-powered resume parsing and job-matching** algorithms.
- Develop an **automated quiz generation system** to help companies evaluate candidates.

- Enable **real-time chat** for communication between job seekers and employers.
- Use **incremental sprints** to progressively build and test system functionalities.

3.3.3. Development Approach

The project follows an **Agile and Incremental** methodology using **sprint-based execution**. Each sprint focuses on **design, development, testing, and integration** of different components.

- **Sprint-Based Execution:** The system is built over **iterative sprints**, allowing continuous testing and improvements.
- **Microservices Architecture:** The system consists of **independent services**, making it **scalable and modular**.
- **Incremental Feature Delivery:** Each sprint delivers a **functional system component** that is integrated and tested.

3.3.4. Project Scope

✅ **User Management** – Secure registration, authentication, and profile management.

✅ **Job Posting & Applications** – Employers can post jobs, and job seekers can apply.

✅ **AI-Powered Resume Parsing** – Extracting key details from resumes.

✅ **AI-Based Job Matching** – Suggesting jobs based on AI analysis.

✅ **Quiz Generation System** – AI-powered quizzes to evaluate candidates.

- **Real-Time Chat & Notifications** – Employer-applicant communication.
- **System Security** – Implementing authentication and data protection.

3.4. Project plan – Gantt chart:

A document outlining tasks, deadlines, and resources needed to achieve project objectives, serving as a roadmap for successful project execution.

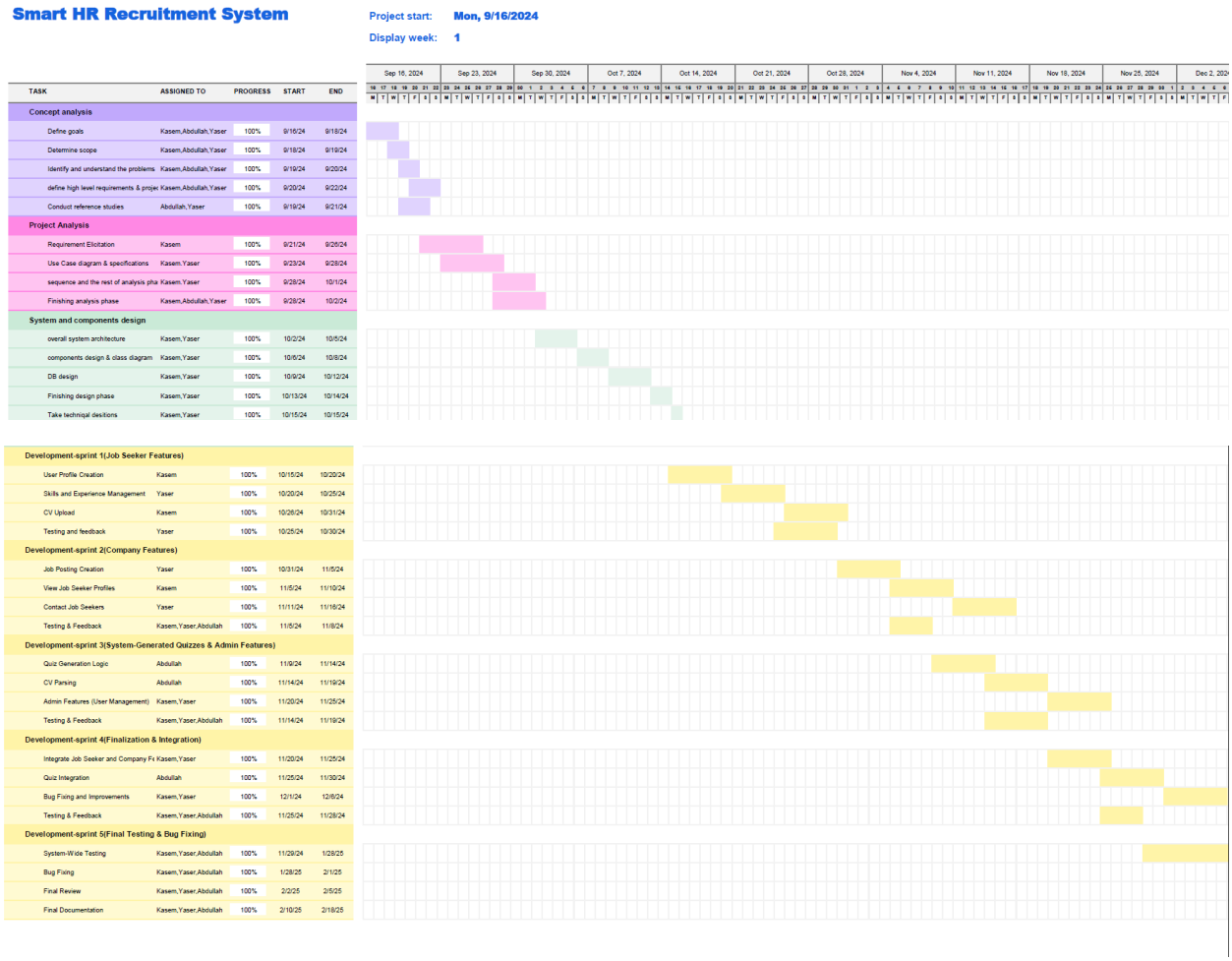


Table 2 project plan – Gantt chart

3.5. Risk Management:

The project follows a structured **risk management approach** to anticipate and mitigate potential challenges.

Risk Category	Risk Description	Impact	Mitigation Strategy
Technical	AI job matching may return inaccurate results	High	Regular AI model refinement and feedback-based improvements
Security	Risk of unauthorized data access or breaches	High	Implement strong authentication and encryption measures
Scalability	High system load affecting performance	Medium	Optimize database queries and use caching mechanisms
Integration	Issues with microservices communication	Medium	Conduct early integration testing and ensure proper API versioning
Development Delays	Complexity in implementing AI-based features	High	Maintain clear sprint goals and allocate time for research and debugging

3.6. Summary:

The Smart HR Recruitment System is a student-driven project designed to enhance the hiring process through automation, AI-driven job matching, and real-time communication tools. The project follows an incremental sprint-based approach, ensuring continuous improvement and adaptability.

This Statement of Work (SOW) outlines the project objectives, scope, deliverables, resources, and timelines, ensuring a structured and well-managed execution plan.

Chapter4 System Analysis

4.1. Introduction:

In this chapter we will introduce the analytical study of the system using the needed UML diagrams for system requirements modeling.

4.2. Requirements Elecitiation:

Requirements elicitation is a crucial phase in software development, where the needs of stakeholders are gathered and analyzed to define the system's functionalities. For the **Smart HR Recruitment System**, the requirements were collected through discussions with stakeholders, market research, and analysis of existing recruitment systems.

The resulted Requirement database:

Table 3 Requirements Database

req-id	Requirement title	description	category
RE-FR-JS-1	Job Seeker Profile Creation	Allow job seekers to create profiles with personal and professional details.	Job Seeker Features
RE-FR-JS-2	Job Seeker Profile Editing	Enable job seekers to update personal and professional information.	Job Seeker Features
RE-FR-JS-3	Profile Picture Upload	Allow job seekers to upload and update their profile pictures.	Job Seeker Features
RE-FR-JS-4	Contact Information Management	Enable job seekers to manage and update their email, phone number, and location.	Job Seeker Features
RE-FR-JS-5	Skills & Experience Management	Allow job seekers to add, edit, and remove skills, experiences, and certifications.	Job Seeker Features
RE-FR-JS-6	Education Management	Enable job seekers to add and update education details.	Job Seeker Features

RE-FR-JS-7	CV Upload & Management	Allow job seekers to upload and manage multiple CVs for different job applications.	Job Seeker Features
RE-FR-JS-8	AI Resume Optimization	AI-powered analysis of CVs to extract and optimize skills for better job matching.	AI-Powered Features
RE-FR-JS-9	Personalized Job Suggestions	Provide job recommendations based on AI job matching and applicant skills.	AI-Powered Features
RE-FR-JS-10	Job Search with Filters	Enable job seekers to search and filter jobs by category, salary range, location, and experience.	Job Seeker Features
RE-FR-JS-11	Advanced Job Search	Allow job seekers to apply multiple filters simultaneously for precise job searches.	Job Seeker Features
RE-FR-JS-12	Save & Bookmark Jobs	Allow job seekers to save or bookmark job postings for later reference.	Job Seeker Features
RE-FR-JS-13	Job Application Submission	Enable job seekers to apply to multiple job postings easily.	Job Seeker Features
RE-FR-JS-14	Application Tracking	Allow job seekers to track the status of their applications (e.g., viewed, shortlisted, rejected).	Job Seeker Features
RE-FR-JS-15	Social Media Integration	Integrate LinkedIn, GitHub, and other professional platforms.	Job Seeker Features
RE-FR-JS-16	Real-Time Job Alerts	Send automated notifications for new job postings matching user preferences.	System Features
RE-FR-CO-17	Company Registration & Verification	Enable companies to register, verify identity, and create employer accounts.	Company Features
RE-FR-CO-18	Job Posting Management	Allow companies to create, edit, delete, and manage job postings.	Company Features
RE-FR-CO-19	Candidate Search & Filtering	Enable employers to search and filter candidates based on skills, experience, and location.	Company Features
RE-FR-CO-20	Candidate Application Review	Allow companies to view, shortlist, and reject job applications.	Company Features
RE-FR-CO-21	Sorting Applicants by AI Match	Employers can sort applicants based on AI job-matching scores.	AI-Powered Features
RE-FR-CO-22	Sorting Applicants by Quiz Score	Employers can rank applicants based on quiz scores.	AI-Powered Features
RE-FR-CO-23	Candidate Information Access	Companies can view applicant profiles, experience, and uploaded CVs.	Company Features
RE-FR-CO-24	Direct Candidate Messaging	Provide companies with a real-time messaging system to communicate with applicants.	Communication Features
RE-FR-CO-25	Company Profile Management	Enable companies to manage their company details, logo, and industry information.	Company Features

RE-FR-CO-26	Employer Dashboard	Provide companies with a dashboard to manage job postings, applications, and messages.	Company Features
RE-FR-AI-27	AI-Based Job Matching	AI-driven job matching to recommend relevant jobs to candidates based on skills and experience.	AI-Powered Features
RE-FR-AI-28	AI Resume Parsing	Extract and categorize skills, experience, and education from uploaded CVs.	AI-Powered Features
RE-FR-AI-29	Smart Job Description Analysis	AI will analyze job descriptions and suggest improvements for better candidate attraction.	AI-Powered Features
RE-FR-SYS-30	Real-Time Notifications	Send alerts for job applications, new job postings, and employer actions.	System Features
RE-FR-SYS-31	Role-Based Access Control	Enforce access control for job seekers, employers, and admins.	System Features
RE-FR-SYS-32	Data Security & Privacy Compliance	Ensure compliance with GDPR and data protection policies.	System Features
RE-FR-SYS-33	Multi-Language Support	Support multiple languages for diverse user accessibility.	System Features
RE-FR-CH-34	Real-Time Messaging	Allow job seekers and employers to communicate instantly.	Communication Features
RE-FR-CH-35	Chat Management & History	Enable users to manage and view chat conversation history.	Communication Features
RE-FR-QZ-36	AI-Based Quiz Generation	AI will generate quizzes based on job roles and required skills.	AI-Powered Features
RE-FR-QZ-37	Quiz Creation by Companies	Allow employers to create skill-based quizzes for applicants.	Company Features
RE-FR-QZ-38	Quiz Management & Editing	Employers can modify, delete, and manage created quizzes.	Company Features
RE-FR-QZ-39	Automatic Quiz Scoring & Feedback	AI will evaluate applicant answers and provide automated scoring and feedback.	AI-Powered Features
RE-FR-QZ-40	Applicant Quiz Performance History	Track past quiz performance to improve job recommendations.	AI-Powered Features
RE-FR-QZ-41	Display User Quiz Answers	Allow employers to review job seeker quiz responses.	Company Features
RE-FR-QZ-42	Quiz-Based Shortlisting	Automatically shortlist candidates based on quiz performance and job criteria.	AI-Powered Features
RE-FR-QZ-43	Timed Quiz Feature	Each question has a set time limit; once time expires, the question cannot be answered.	AI-Powered Features

4.3. SRS Document:

4.3.1. Introduction

4.3.1.1. Purpose

The **Smart HR Recruitment System** aims to **simplify and enhance the hiring process** by leveraging AI-driven job matching, candidate screening, and application tracking. It provides a **platform for job seekers and employers** to interact efficiently, reducing recruitment time and effort.

4.3.1.2. Document Conventions

This document follows the **IEEE SRS Standard**, with clear functional and non-functional requirements.

4.3.1.3. Intended Audience and Reading Suggestions

This document is intended for:

- **Developers**: To understand system functionalities and requirements.
- **Project Managers**: To track system scope and development.
- **HR Professionals**: To provide insights into the hiring process.
- **QA Engineers**: To ensure functional compliance through testing.

4.3.1.4. Product Scope

The **Smart HR Recruitment System** provides a **web-based job recruitment platform** that offers:

- Job Posting & Management for companies.
- Profile Management & Job Search for job seekers.
- AI-Driven Job Matching & Resume Screening.
- System-Generated Skill-Based Assessments.
- Real-time Notifications & Analytics.

This system **bridges the gap between employers and job seekers** by utilizing **AI-based automation**, ensuring better hiring decisions and reducing bias in recruitment.

4.3.1.5. References

4.3.2. Overall Description

4.3.2.1. Product Perspective

The system follows a **microservices architecture**, integrating multiple services for **job posting, AI-driven recommendations, and application management.**

4.3.2.2. Product Functions

The key functionalities include:

1. **Job Seekers can**:
    - Create and manage profiles.
    - Search and filter job listings.
    - Apply for jobs and track their status.
    - Receive AI-powered job recommendations.
2. **Companies can**:

    - Post and manage job listings.
    - Search for candidates and shortlist them.
    - Communicate directly with applicants.
3. **Administrators can**:
    - Manage users and system security.
    - Oversee reports and analytics.

4.3.2.3. User Characteristics

- **Job Seekers**: Tech-savvy individuals looking for employment.
- **Employers**: HR managers posting job vacancies and reviewing applicants.
- **Admins**: System managers handling security and performance.

4.3.2.4. Constraints

- **Web-based access only (no mobile app initially).**
- **Basic authentication and security measures will be implemented.**
- **Limited AI implementation due to computational constraints.**

4.3.3. System Features

4.3.3.1. Functional Requirements

(Based on the expanded 40 functional requirements, structured by category)

Job Seeker Features

- Job seekers can create and edit profiles.
- Upload and manage multiple resumes.
- Search and filter job listings.

- Apply for jobs and track their status.
- Receive job alerts and notifications.

Company Features

- Employers can register and manage accounts.
- Post and edit job listings.
- Search for candidates based on filters.
- Track applicant submissions.

AI-Powered Features

- AI-based job matching and recommendations.
- Automated resume parsing for screening.
- Skill-based self-assessment tests for applicants.

System Features

- Multi-language support.
- Secure user authentication and role-based access.
- Basic user data protection and encryption.
- Real-time notifications.

4.3.3.2. Non-Functional Requirements

- Performance: The system should be able to handle 1000+ users at a time.
- Security: Implement basic authentication and encrypted password storage.
- Usability: The UI should be simple and easy to navigate for all users.

- Scalability: Should support basic expansion for more job postings and user registrations.
- Maintainability: The code should be modular and well-documented to allow future updates.
- Reliability: The system should be functional at least 95% of the time.

4.3.4. External Interface Requirements

4.3.4.1. User Interfaces

- Web-based UI (React.js with Material UI for design).
- Dashboard for job seekers and employers.

4.3.4.2. Hardware Interfaces

- The system will run on a standard web server or cloud hosting service.

4.3.4.3. Software Interfaces

- **Frontend**: React.js (TypeScript)
- **Backend**: Django (Python), FastAPI, Spring Boot (Java)
- **Database**: MYSQL,MongoDB,Redis.
- **Basic AI Services**: NLP-based job matching

4.3.4.4. Communication Interfaces

- RESTful API-based communication between frontend and backend.

4.3.5. Other Non-Functional Requirements

4.3.5.1. Security Requirements

- Authentication system (username and password).
- Hashed password storage using bcrypt.

4.3.5.2. Performance Requirements

- The system should load job listings within 2-3 seconds.
- Should process job applications without significant delays.

4.3.5.3. Reliability and Availability

- The system should be available 95% of the time with minimal downtime.
- Implement basic backup procedures for database recovery.

4.3.5.4. Maintainability and Support

- Code should be well-documented for future improvements.
- Open-source libraries should be used for easier debugging.

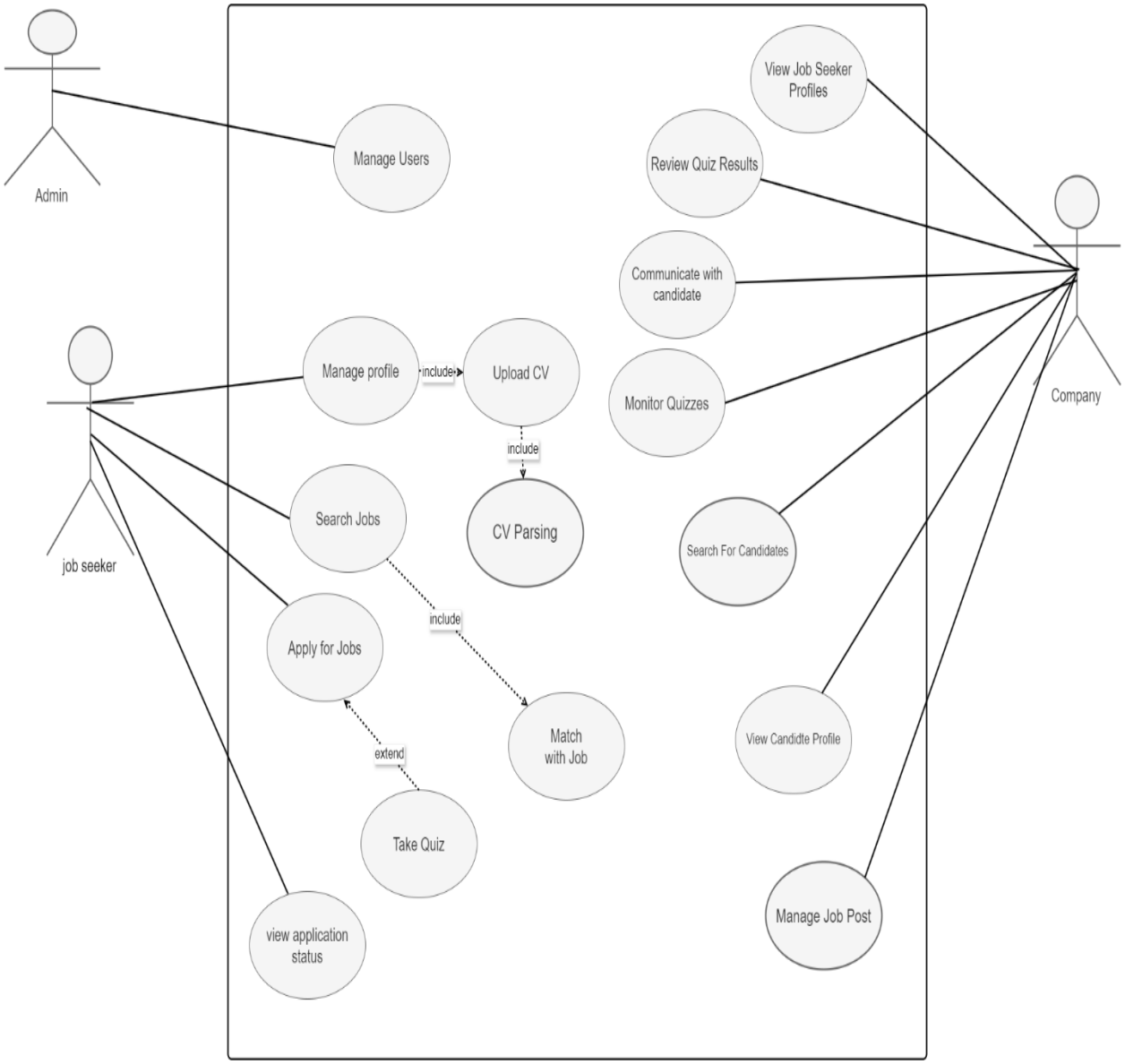
4.3.6. Conclusion:

This SRS document provides a practical and student-friendly outline of the Smart HR Recruitment System, ensuring clear development goals that can be achieved within a student project.

4.4. Requirements modeling:

4.4.1. Use case diagram:

use-case diagrams model the behavior of a system and help to capture the requirements of the system.



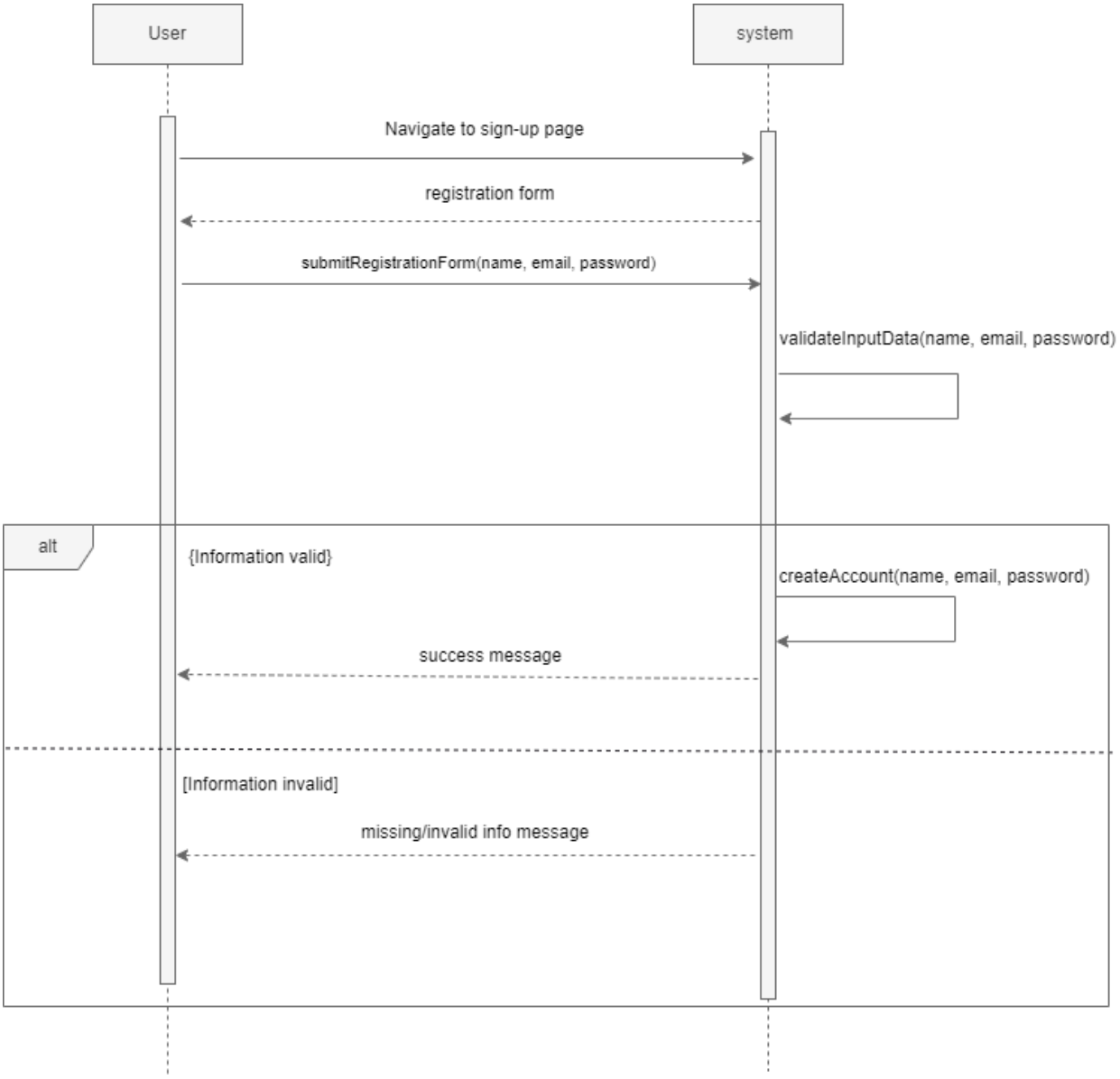
4.4.2. System features – use cases:

- Create Account (UC-01):

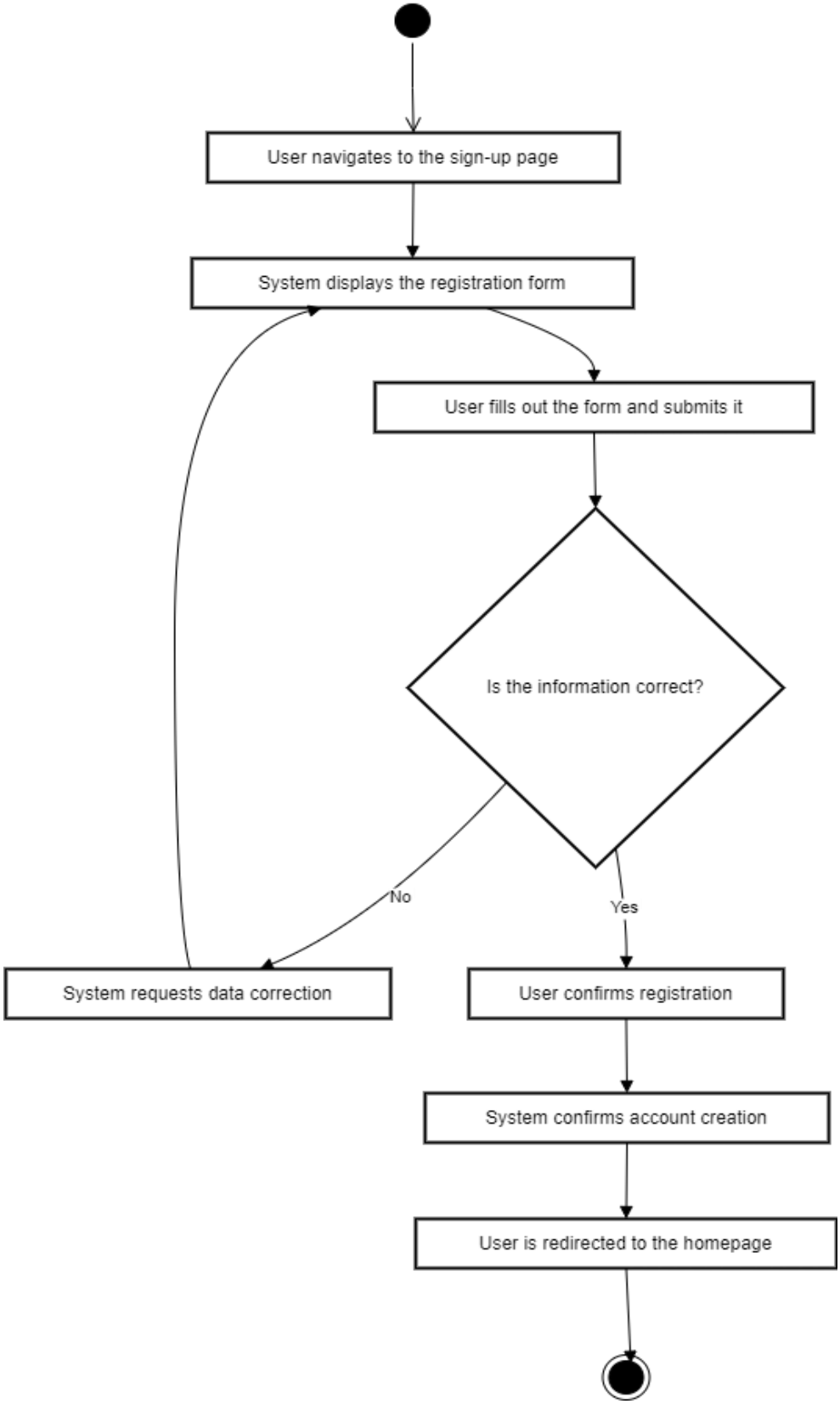
ID	UC-01
Participating Actors	Job Seeker, Company Representative

Flow of Events	1. Actor navigates to the sign-up page. 2. The system shows a registration form requesting personal information. 3. Actor fills out the form and submits it. 4. The system validates the entered information. 5. If valid, a verification code is sent to the email. 6. The user enters the code, and the system verifies it. 7. Account creation is confirmed, and the user is redirected to the homepage.
Alternative Flows	A1: Missing field – System prompts the user to complete all fields. A2: Duplicate email – System requests a different email. A3: Weak password – User is asked to create a stronger password
Entry Conditions	The user accesses the system website.
Exit Conditions	A new user account is successfully created.

Sequence diagram:



Activity diagram:



Use case specification:

- Upload CV (UC-02):

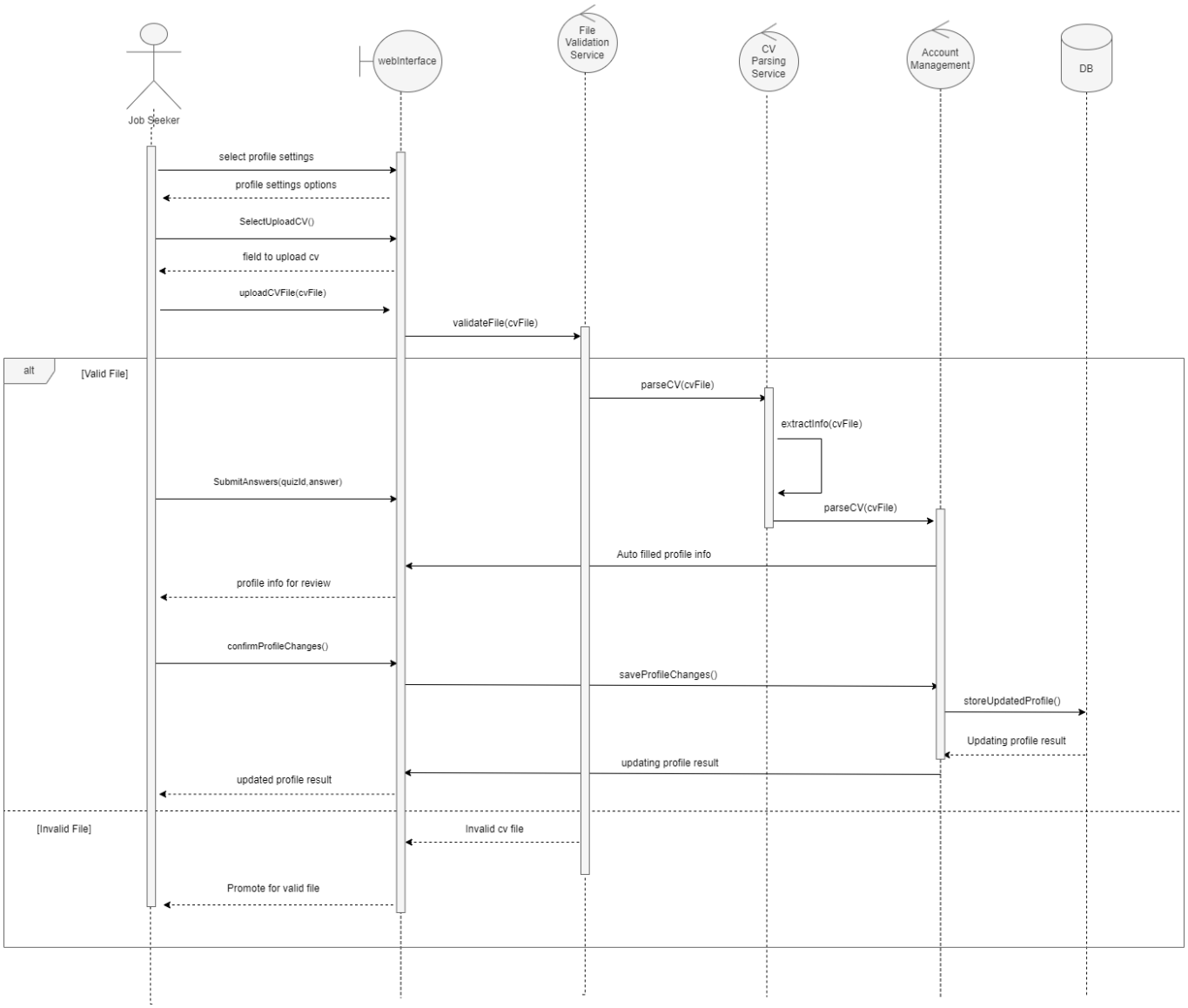
ID	UC-02
Participating Actors	Job Seeker

Flow of Events	1. Job Seeker logs in and navigates to their profile.
	2. The system displays the "Upload CV" option.
	3. Job Seeker selects and uploads a CV file.
	4. The system validates the file format and size.
	5. Once validated, the system parses the CV:
	  1. Extracts key information (e.g., name, education, skills, experience) from the CV.
	  2. Automatically fills the profile fields (e.g., name, skills, education) with the extracted data.
	6. The system allows the job seeker to review and edit the auto-filled profile information.
	7. The job seeker confirms the changes.
	8. The system saves the updated profile with the parsed CV information.

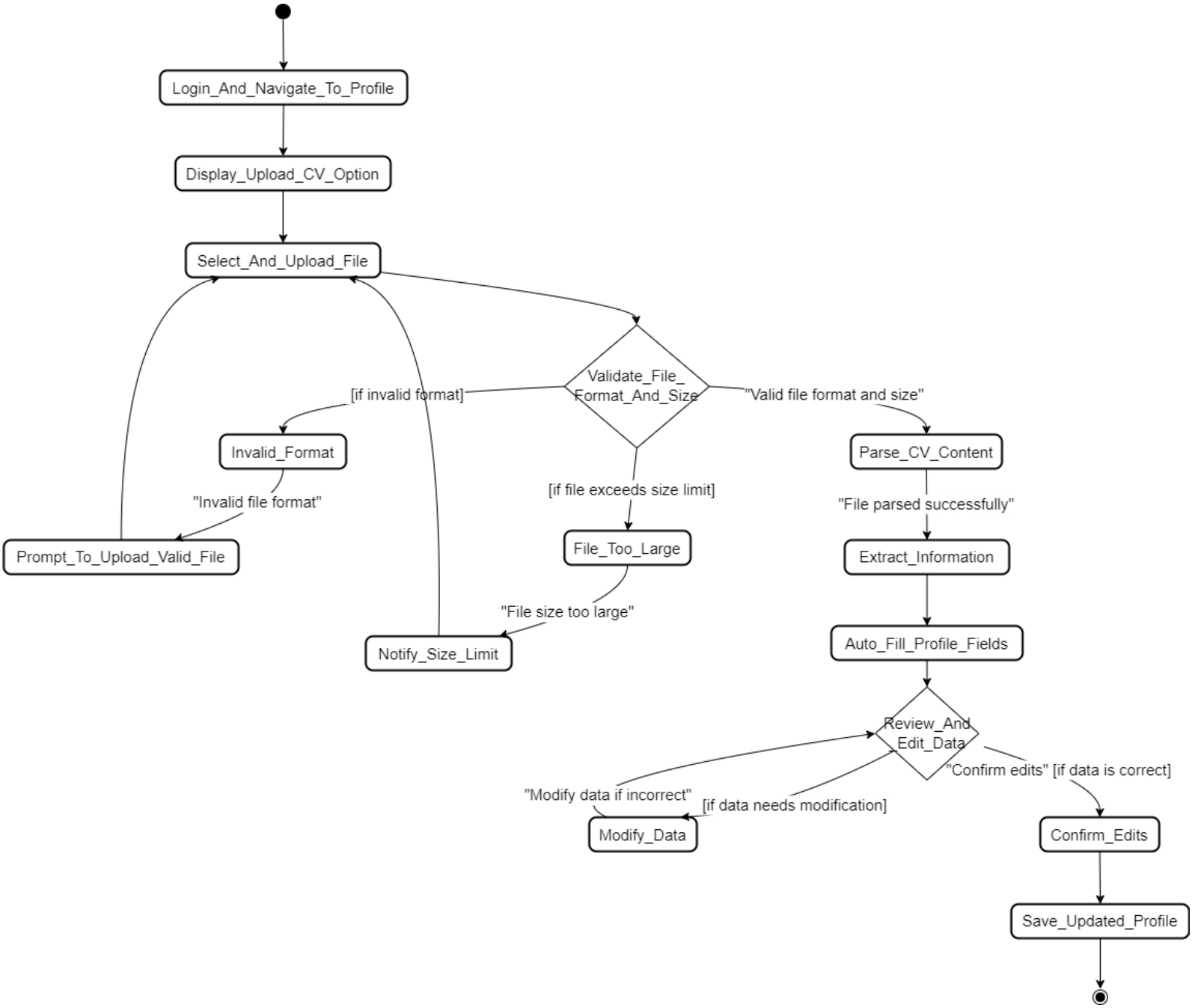
Alternative Flows	A1: Invalid file format – The system prompts the user to upload a valid file. A2: File size too large – The system notifies the user about the size limit.

Entry Conditions	The user must be logged into the system.
Exit Conditions	The CV is uploaded and available in the user's profile.

Sequence diagram:

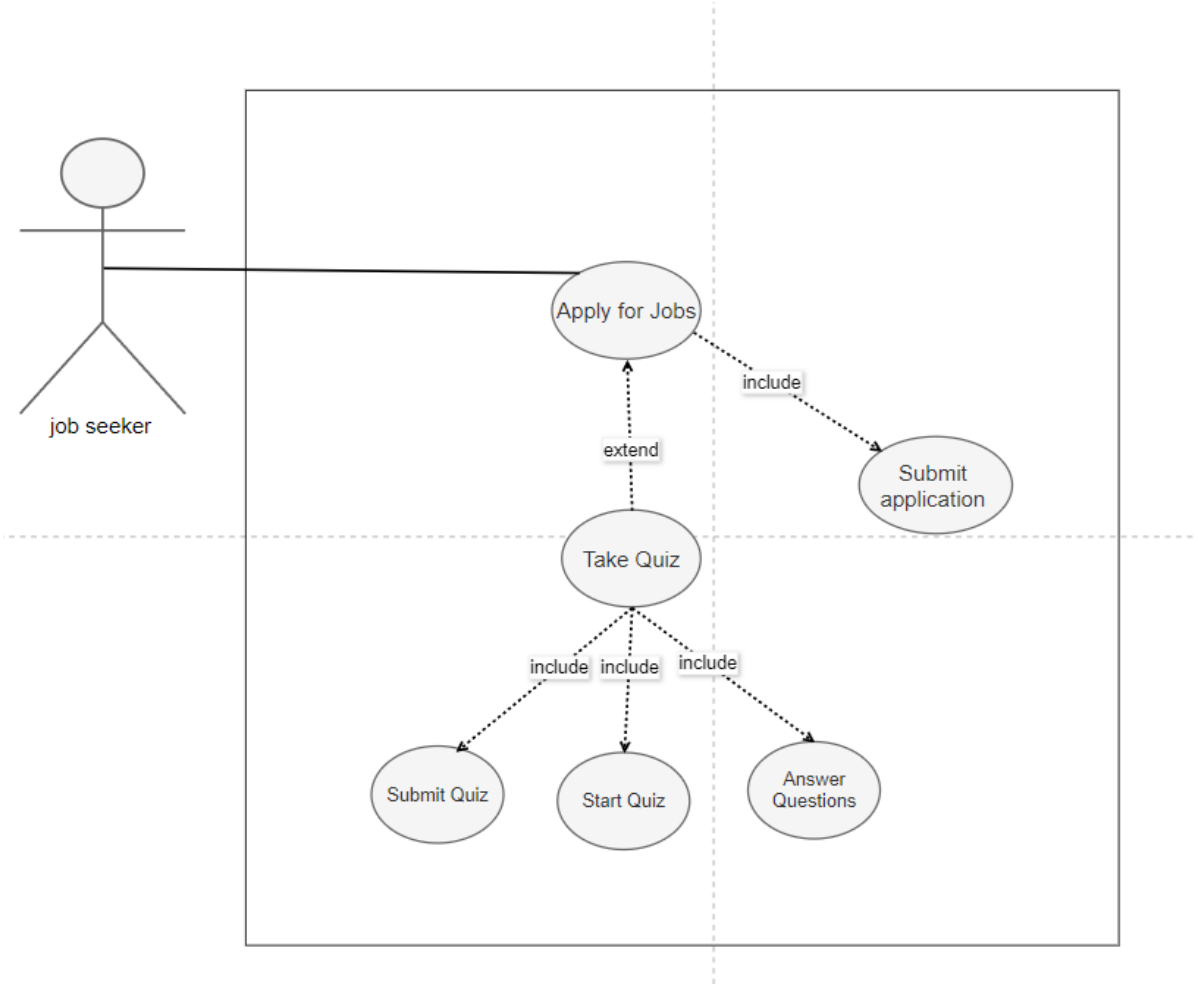


Activity diagram:



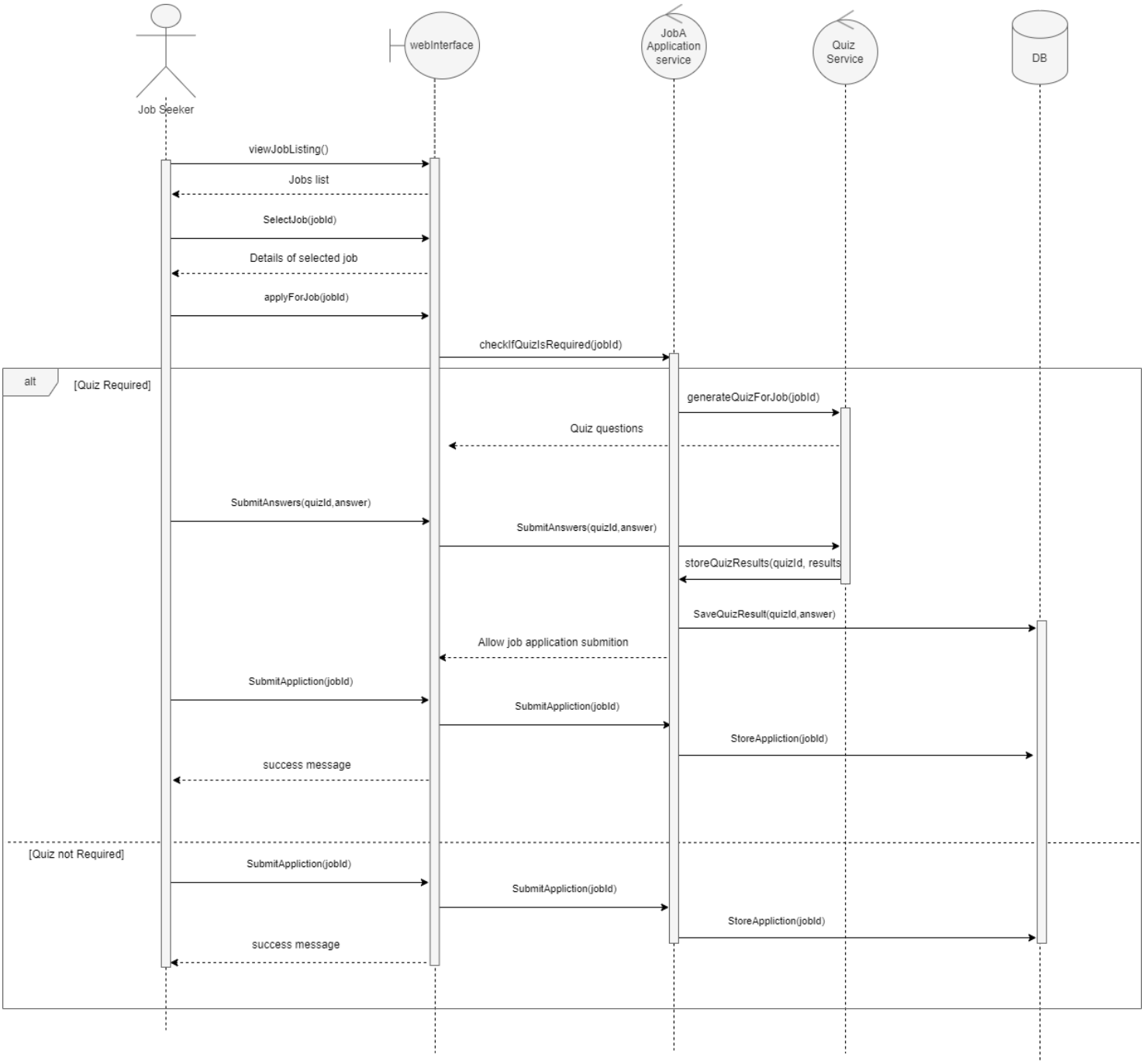
Use case specification:

- Apply for Jobs (UC-03):

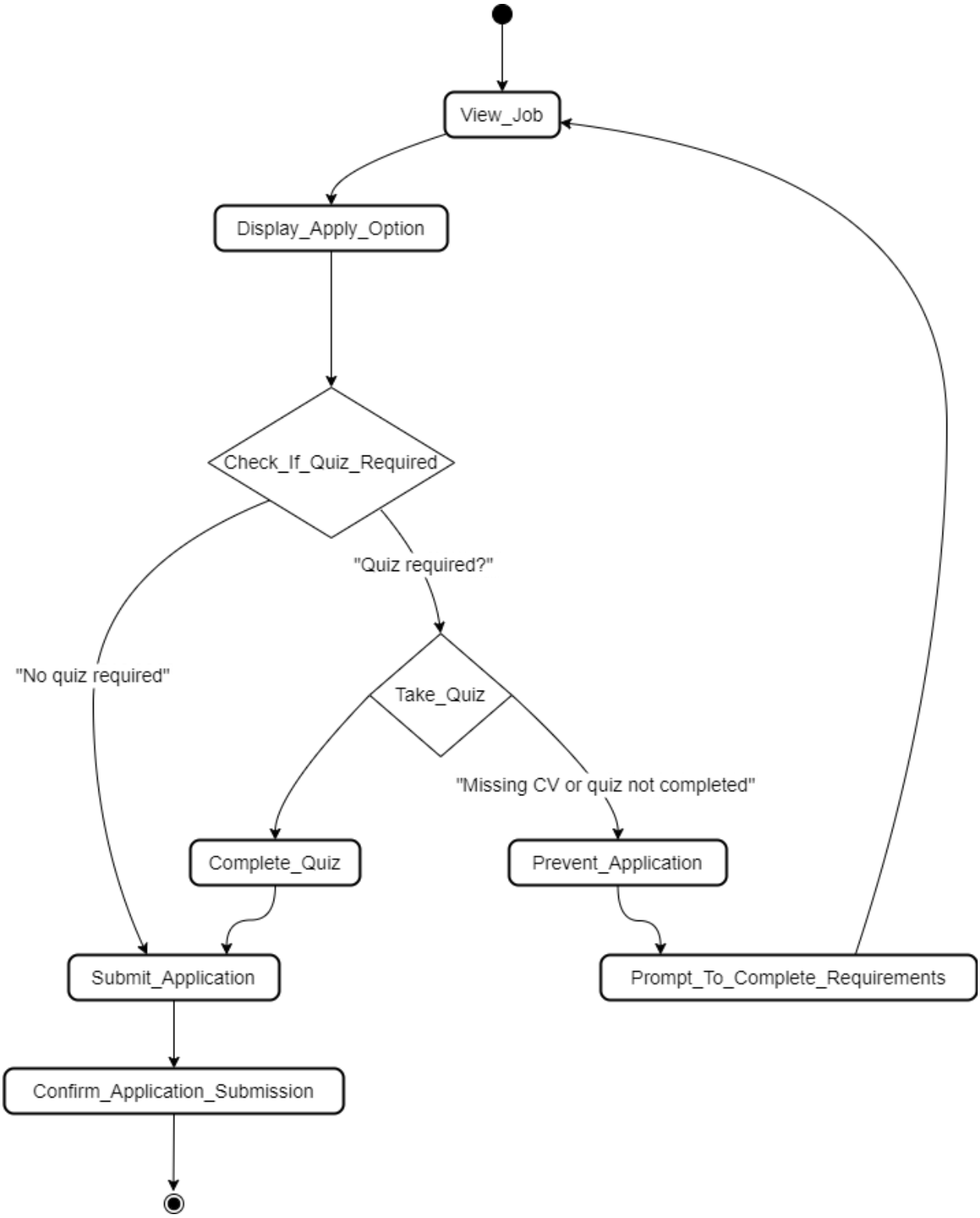


ID	UC-03
Participating Actors	Job Seeker
Flow of Events	1. Job Seeker views a job they want to apply for. 2. The system displays the option to apply and checks if the job requires a quiz. 3. If a quiz is required, the system notifies the Job Seeker to take the quiz before applying. 4. Job Seeker completes the quiz. 5. After completing the quiz, the system allows the Job Seeker to submit the application. 6. The system confirms the application has been submitted to the company.
Alternative Flows	A1: Missing CV or quiz not completed – The system prevents the user from applying and prompts them to upload a CV or complete the quiz.
Entry Conditions	The user has selected a job to apply for and must complete the quiz if required.
Exit Conditions	The application is successfully submitted after completing the quiz.

Sequence diagram:



Activity diagram:

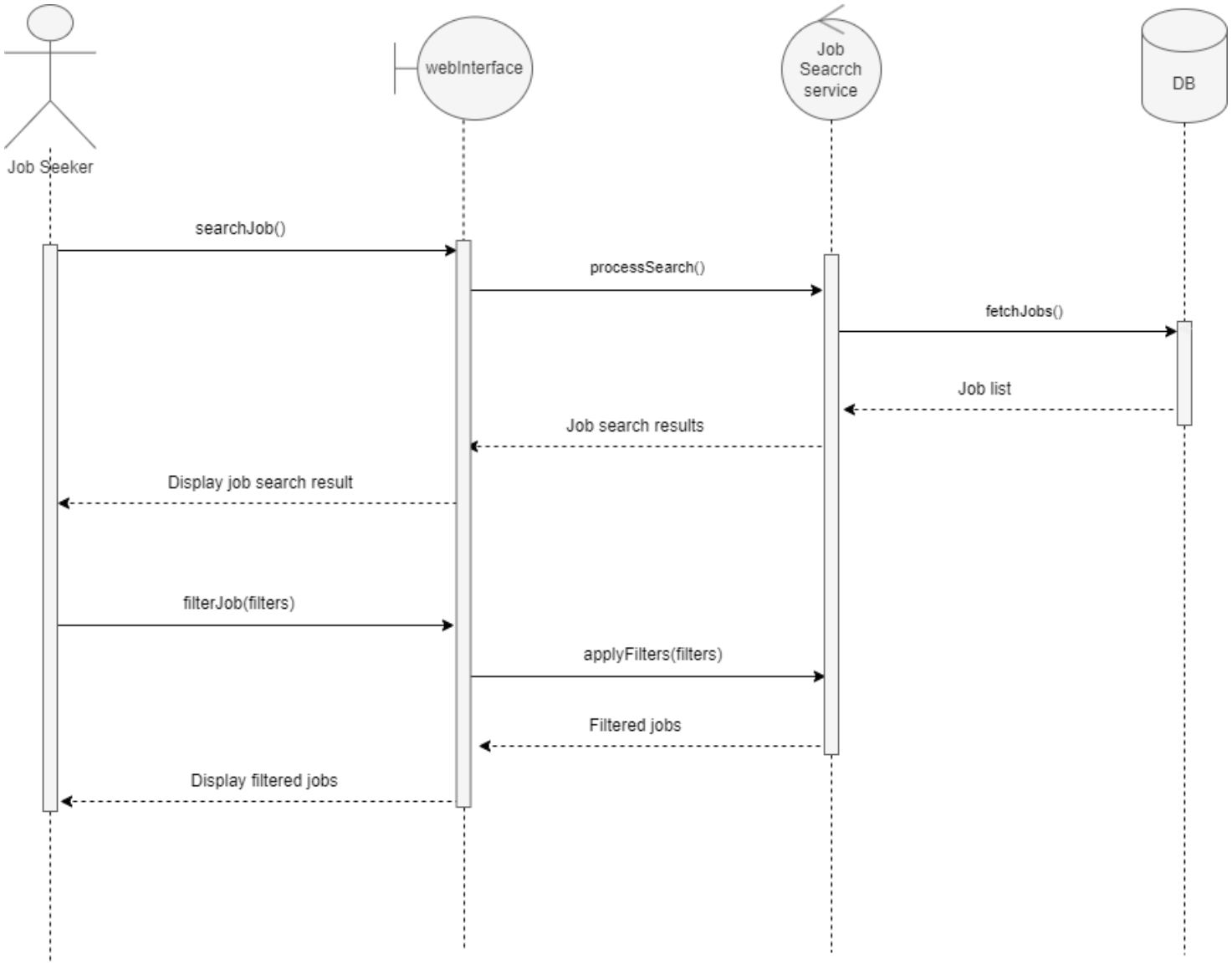


Use case specification:

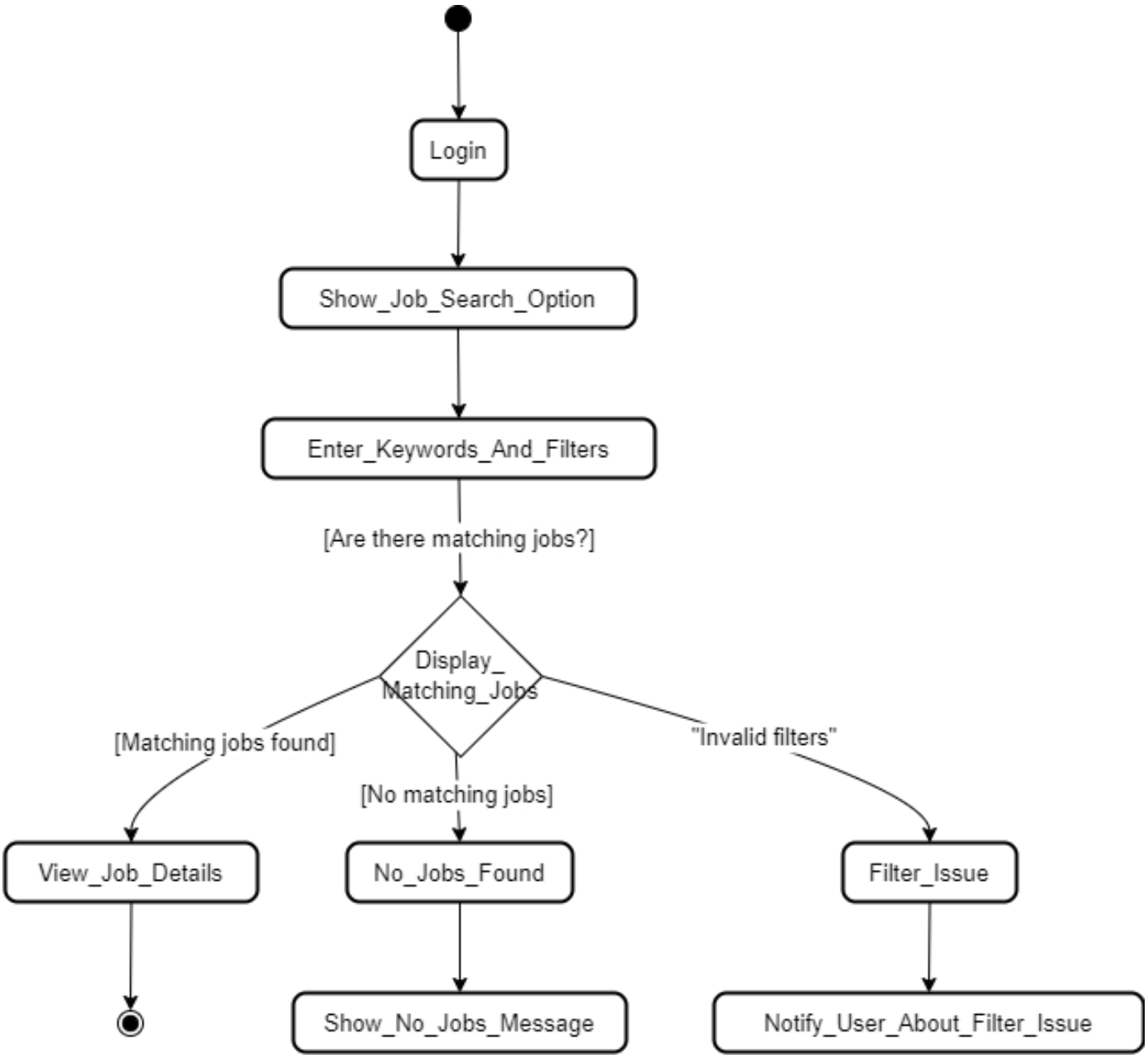
Search Jobs (UC-04):

ID	UC-04
Participating Actors	Job Seeker
Flow of Events	1. Job Seeker logs into the system. 2. The system provides a search option for available jobs. 3. Job Seeker enters keywords and filters (location, job type, etc.). 4. The system displays a list of matching jobs. 5. Job Seeker selects a job to view its details.
Alternative Flows	A1: No jobs found – The system displays a message saying no jobs match the criteria. A2: Filter issue – System notifies the user of any invalid filters used.
Entry Conditions	The user has logged in.
Exit Conditions	A list of jobs is displayed.

Sequence diagram:



Activity diagram:



Use case specification:

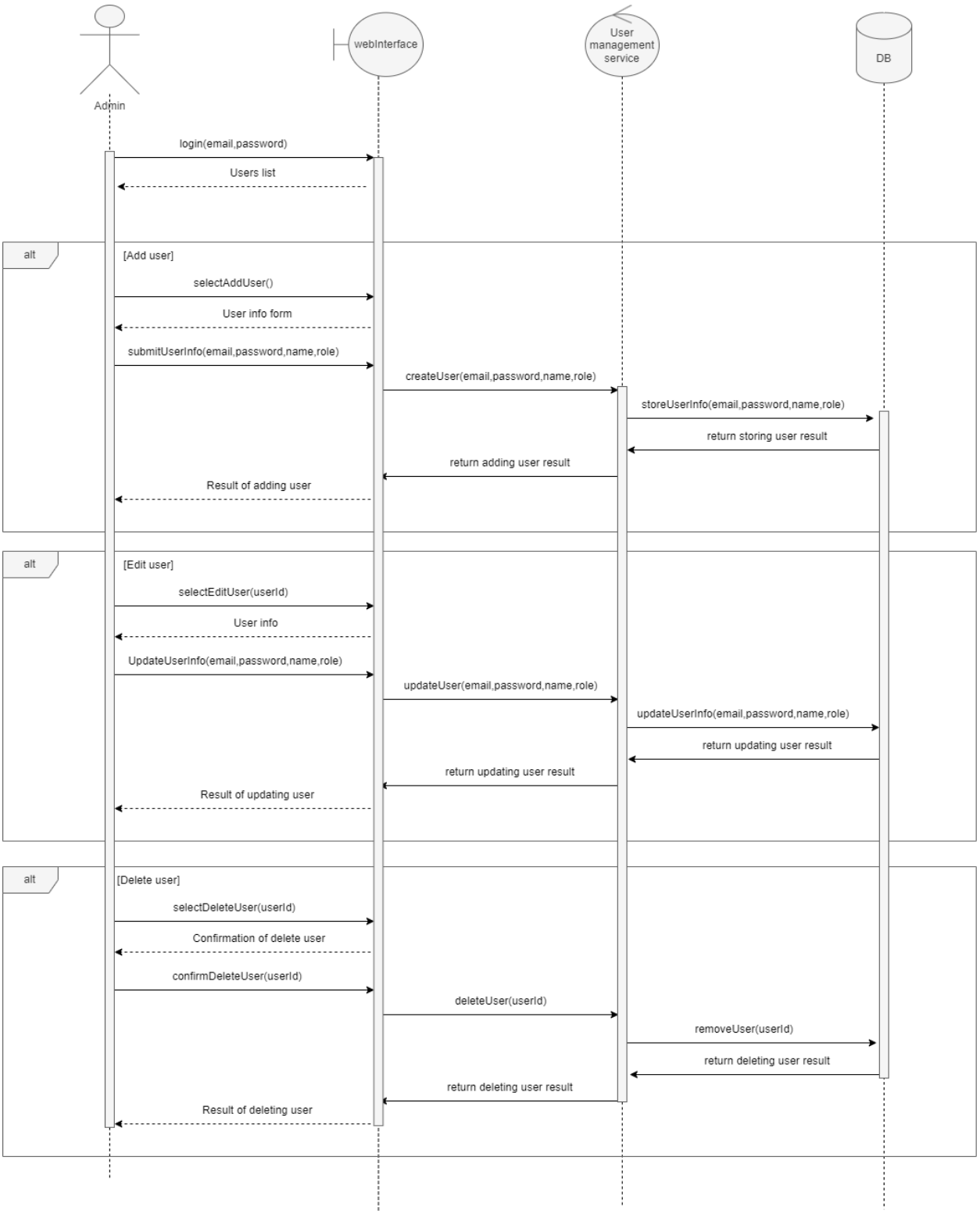
- Manage Users (UC-05):

ID	UC-05
Participating Actors	Admin

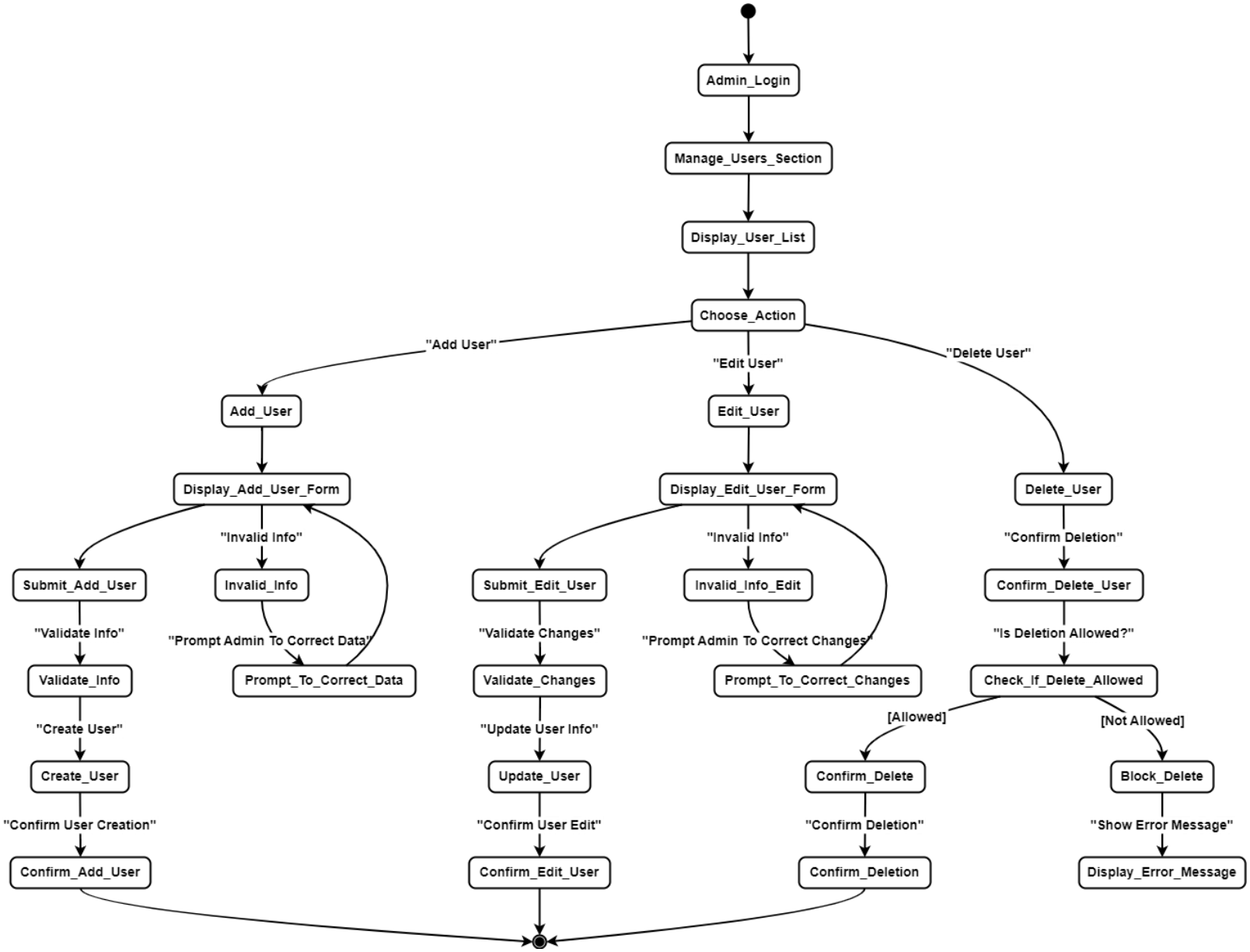
Flow of Events	1. Admin logs in and navigates to the "Manage Users" section.
	2. The system displays a list of users (job seekers and companies) and three optional actions:
	    1. **Add User**:
	        1. Admin selects the option to add a new user (optional).
	        2. The system presents a form to input the new user's information.
	        3. Admin submits the form, the system validates, creates the new user, and confirms.
	    2. **Edit User**:
	        1. Admin selects a user from the list to edit (optional).
	        2. The system presents the user's details, admin edits, and submits changes.
	        3. The system updates the user details and confirms.

	3. **Delete User**: 1. Admin selects a user from the list to delete (optional). 2. The system asks for confirmation, deletes the user, and confirms.
Alternative Flows	A1: Missing or incorrect information prompts the admin to correct data during the add or edit process. A2: If a user cannot be deleted (due to active connections), the system blocks the deletion and shows an error.
Entry Conditions	Admin is logged into the system and navigates to the user management section.
Exit Conditions	A user is optionally added, edited, or deleted, and the system is updated accordingly.

Sequence diagram:



Activity diagram:



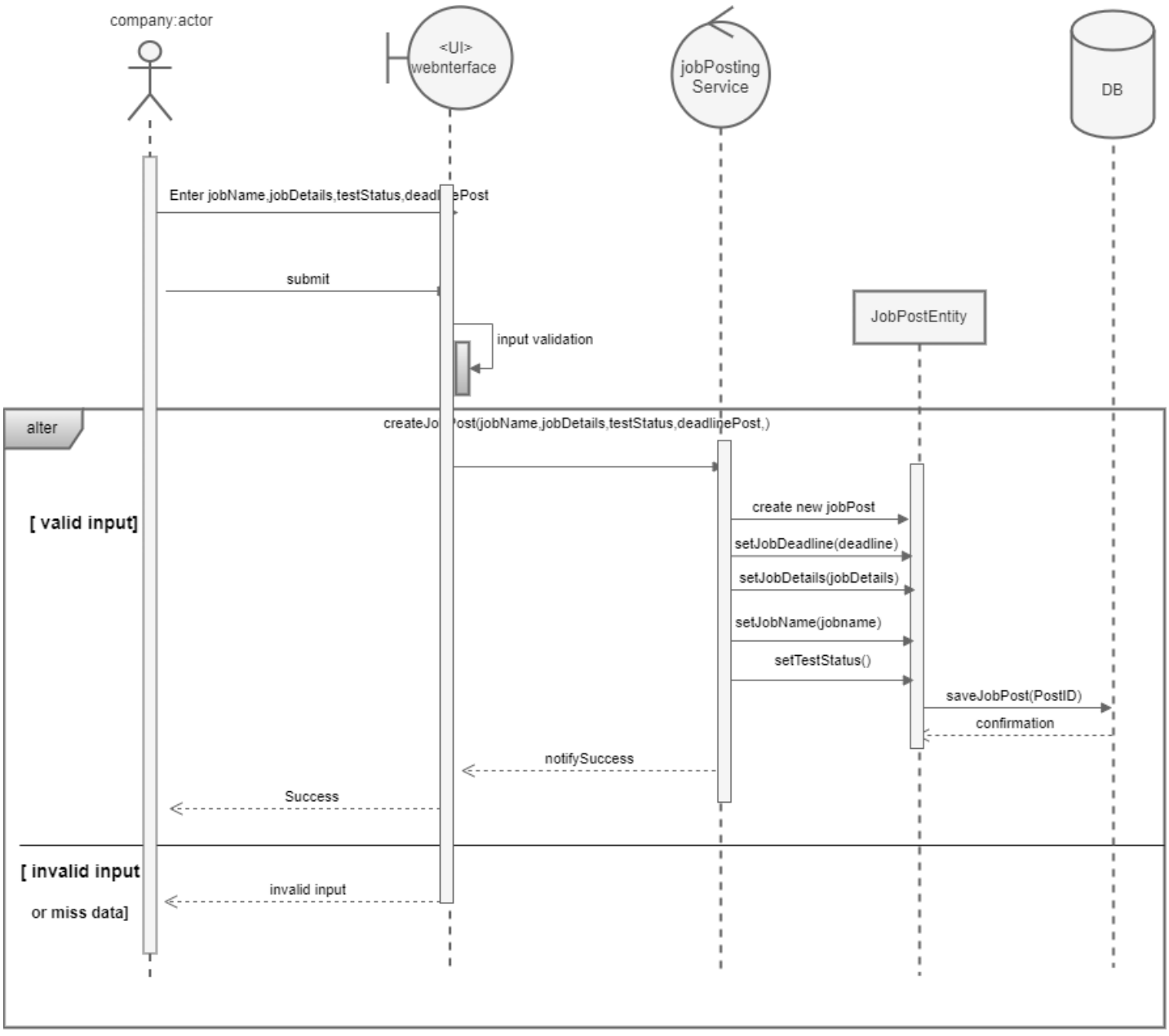
Use case specification:

- Manage Job Post (UC-06)

ID	UC-06
Participating Actors	Company Representative
Flow of Events	1.The company representative accesses the job post management page. 2.The system displays a list of current job posts. 3.The company representative can: • Add a new job post. • Edit an existing job post during 24h. • Remove a job post that is no longer needed. • Close a job post once the position has been filled. • Activate/Deactivate a job post, allowing the representative to temporarily disable the post or reactivate it later. • Set the job post duration so the post remains active for a specific period (e.g., 30 days). 4.The system updates the job post based on the actions taken.
Alternative Flows	A1: Error in job post details, system requests correction.

Entry Conditions	Accesses management page.
Exit Conditions	The operation was completed successfully.

Sequence diagram:



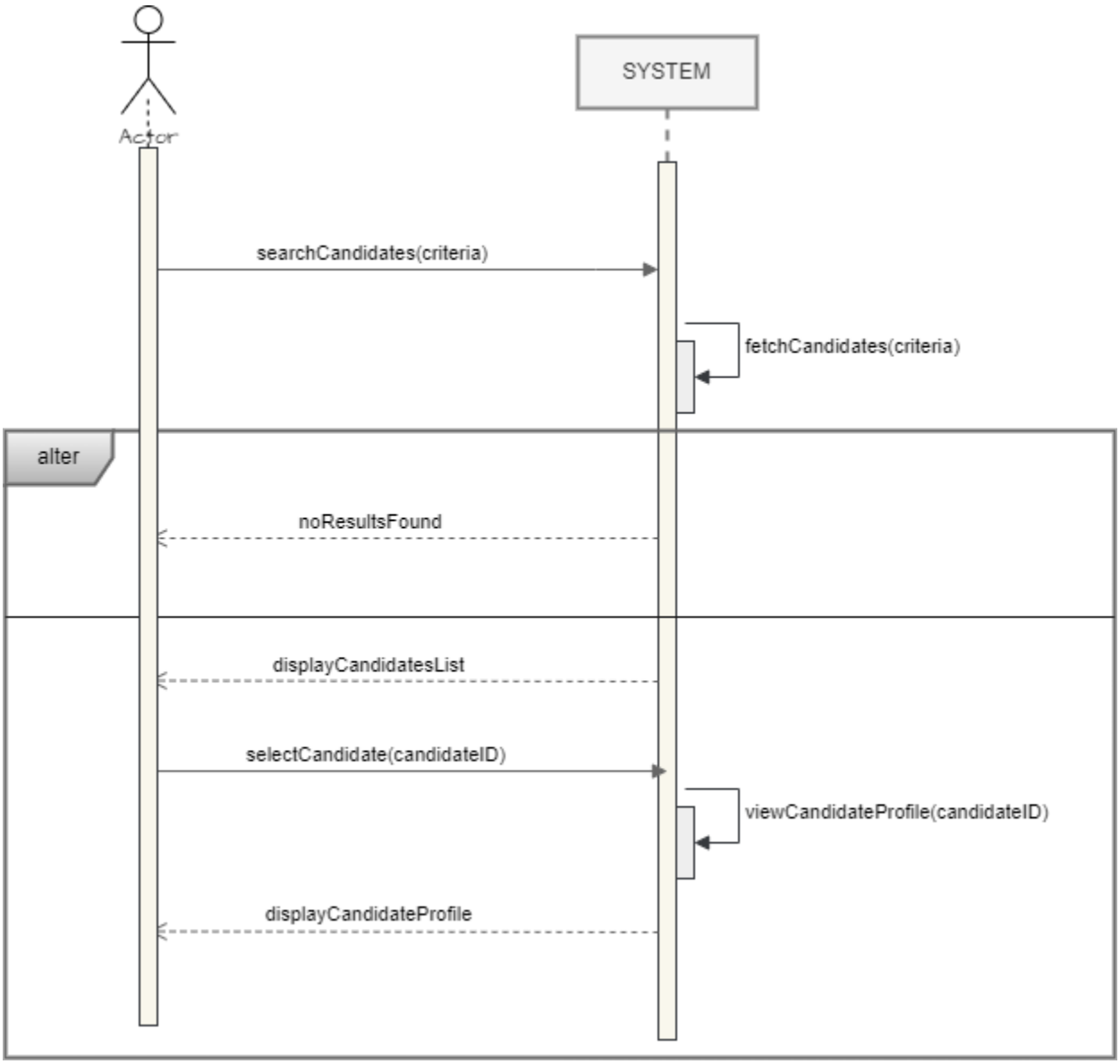
Use case specification:

- Search for Candidates (UC-07)

ID	UC-07
Participating Actors	Company Representative
Flow of Events	1. The company representative opens the candidate search page. 2. The system displays a search form based on criteria (such as skills or others). 3. The company representative enters the required criteria and submits the search. 4. The system displays a list of candidates matching the entered criteria. 5. The company representative can: • View Candidate Profile: If the company representative chooses to view the details of a specific candidate, the system displays the candidate's profile details. • Save Candidate to Shortlist: If the company representative finds a suitable candidate, they can save them to the shortlist for later review.

Alternative Flows	A1: No candidates found, system suggests widening the search criteria.
Entry Conditions	Access search page.
Exit Conditions	Candidates list displayed.

Sequence Diagram:

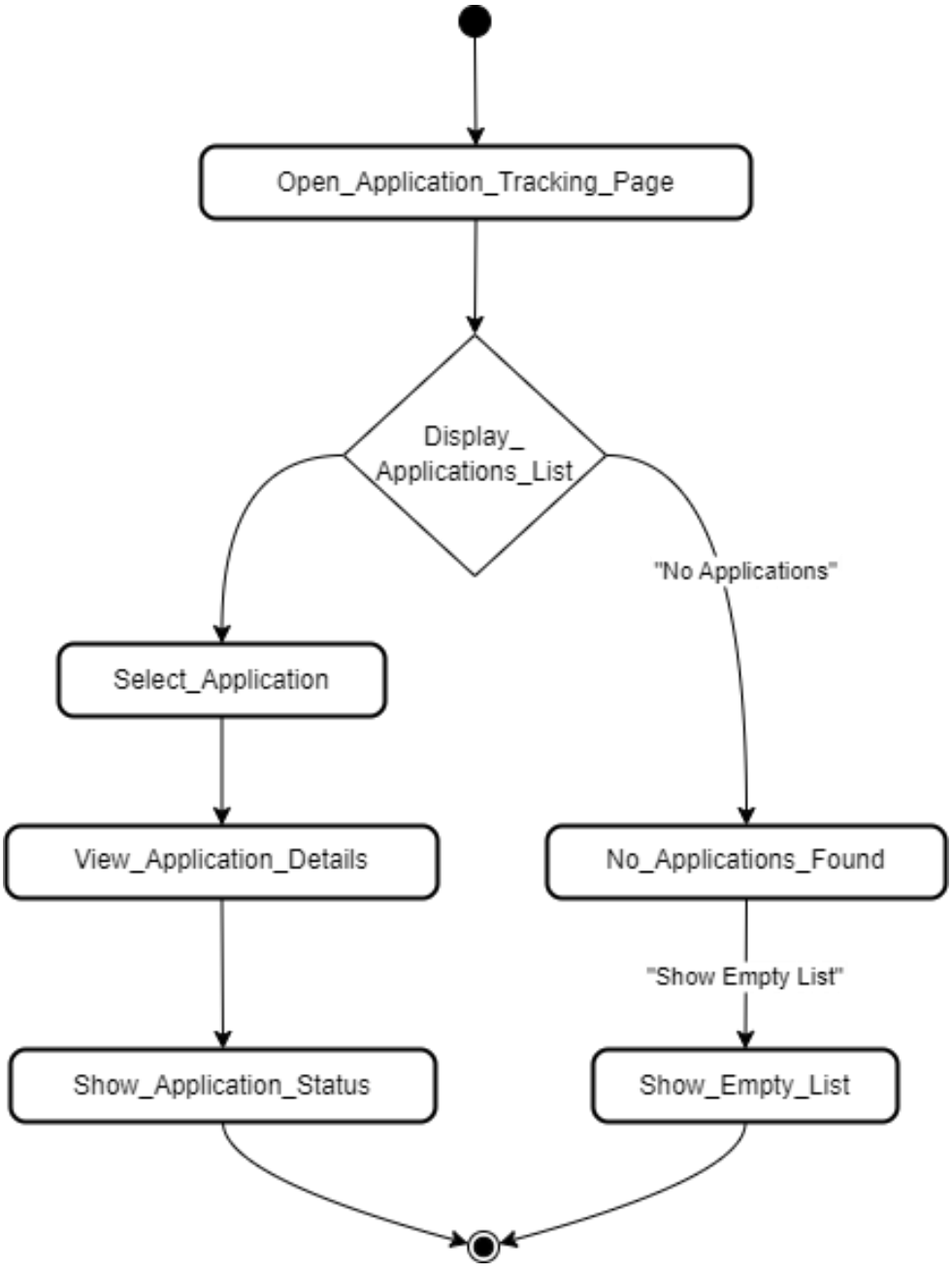


Use case specification:

- Track Job Application (UC-08)

ID	UC-08
Participating Actors	Company Representative
Flow of Events	1. The company representative opens the job application tracking page. 2. The system displays a list of applications received. 3. The representative selects an application to view details. 4. The system shows the status of the application (pending, accepted, rejected).
Alternative Flows	A1: No applications found, system shows empty list.
Entry Conditions	Access application tracking.
Exit Conditions	Application status viewed.

Activity diagram:

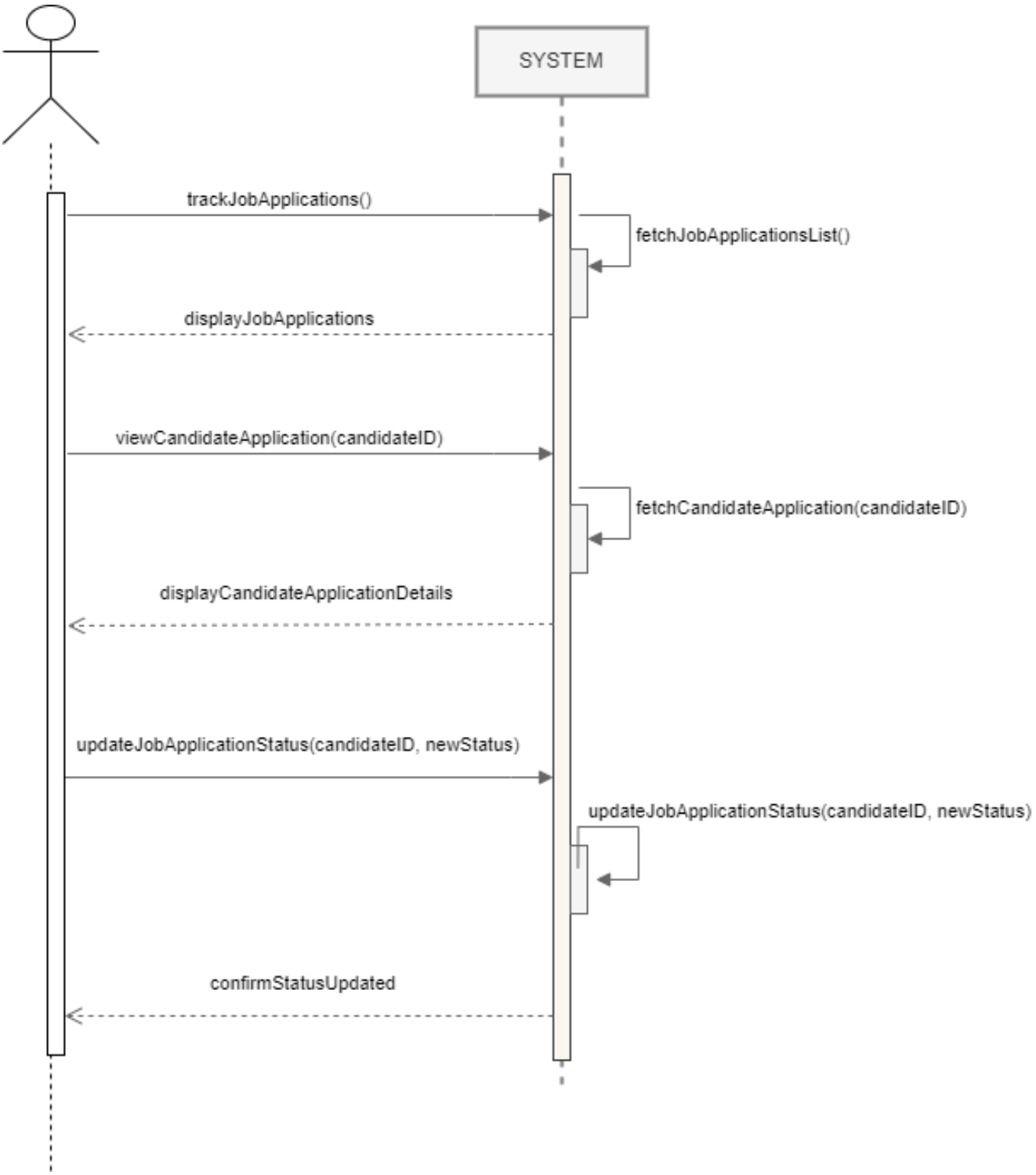


Use case specification:

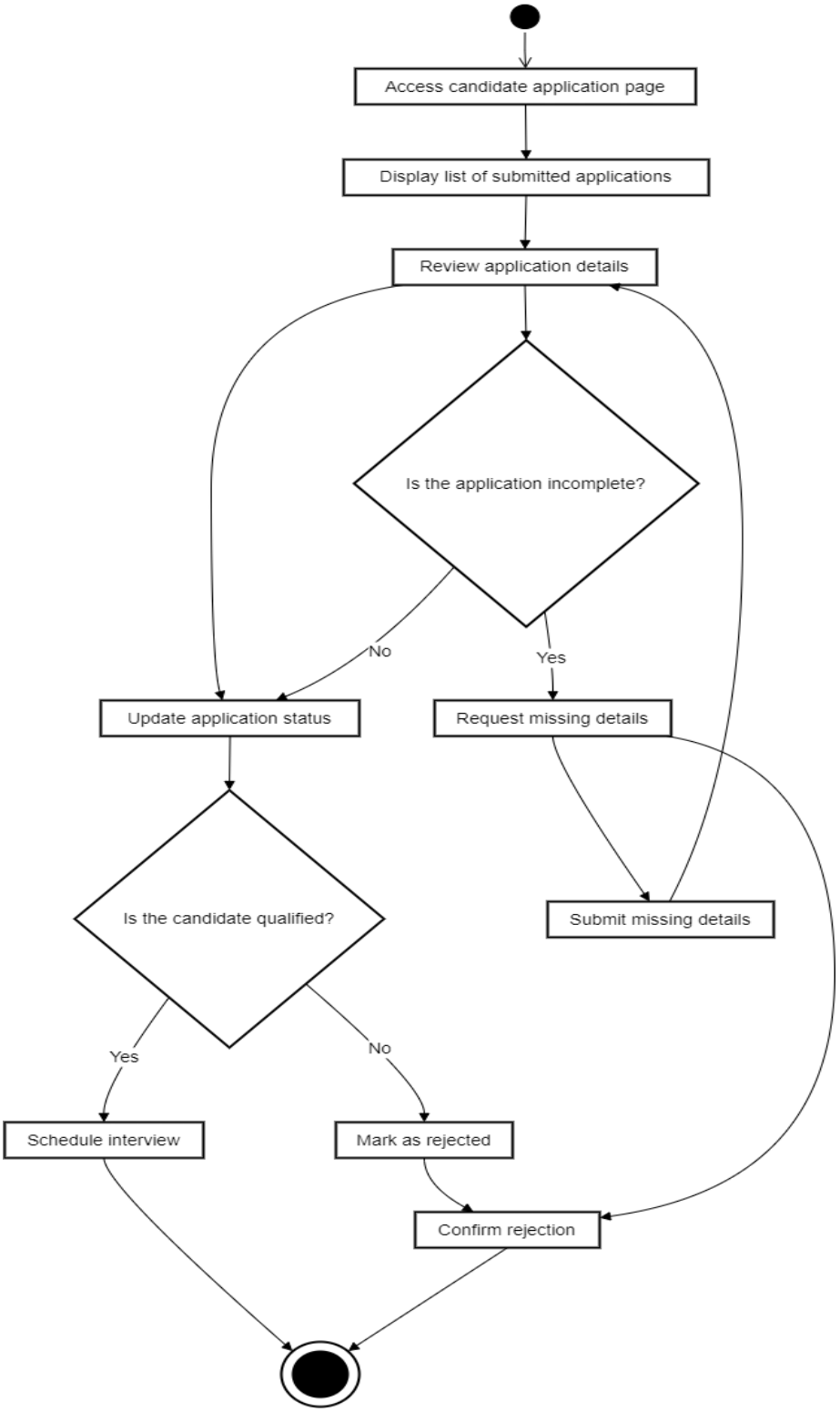
- Manage Candidate Application (UC-08)

ID	UC-08
Participating Actors	Company Representative
Flow of Events	1. The company representative accesses the candidate application page. 2. The system displays a list of submitted applications. 3. The representative reviews the application and updates its status. 4. Schedule Interviews: If the candidate is qualified, the company representative can set up an interview appointment.
Alternative Flows	A1: Application is incomplete, system requests missing details. A2: Candidate is not qualified, system marks as rejected.
Entry Conditions	Access candidate application page.
Exit Conditions	Application status updated

Sequence Diagram:



Activity diagram:



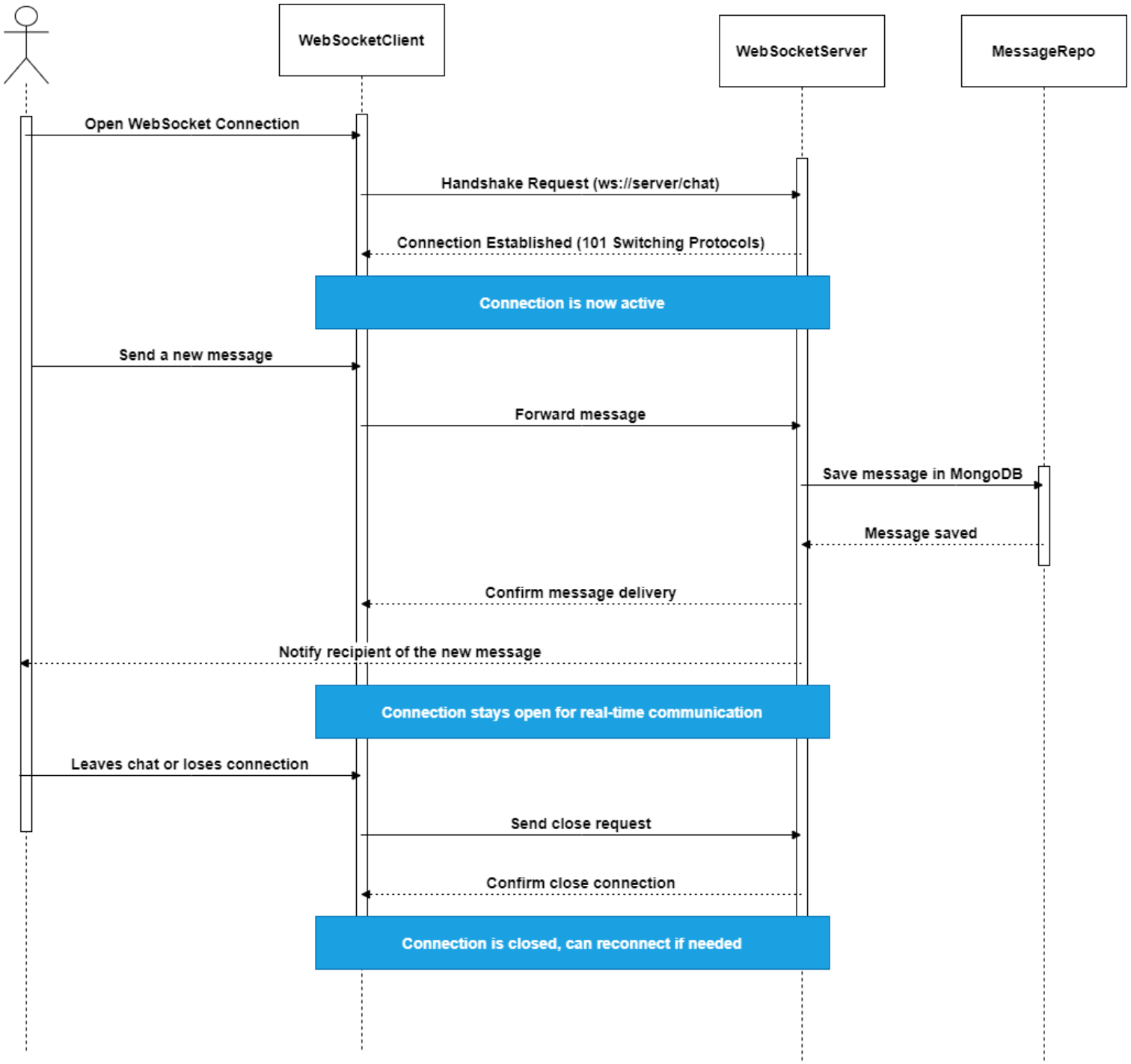
Use case specification:

- Communication Between Job Seeker and Company (UC-09)

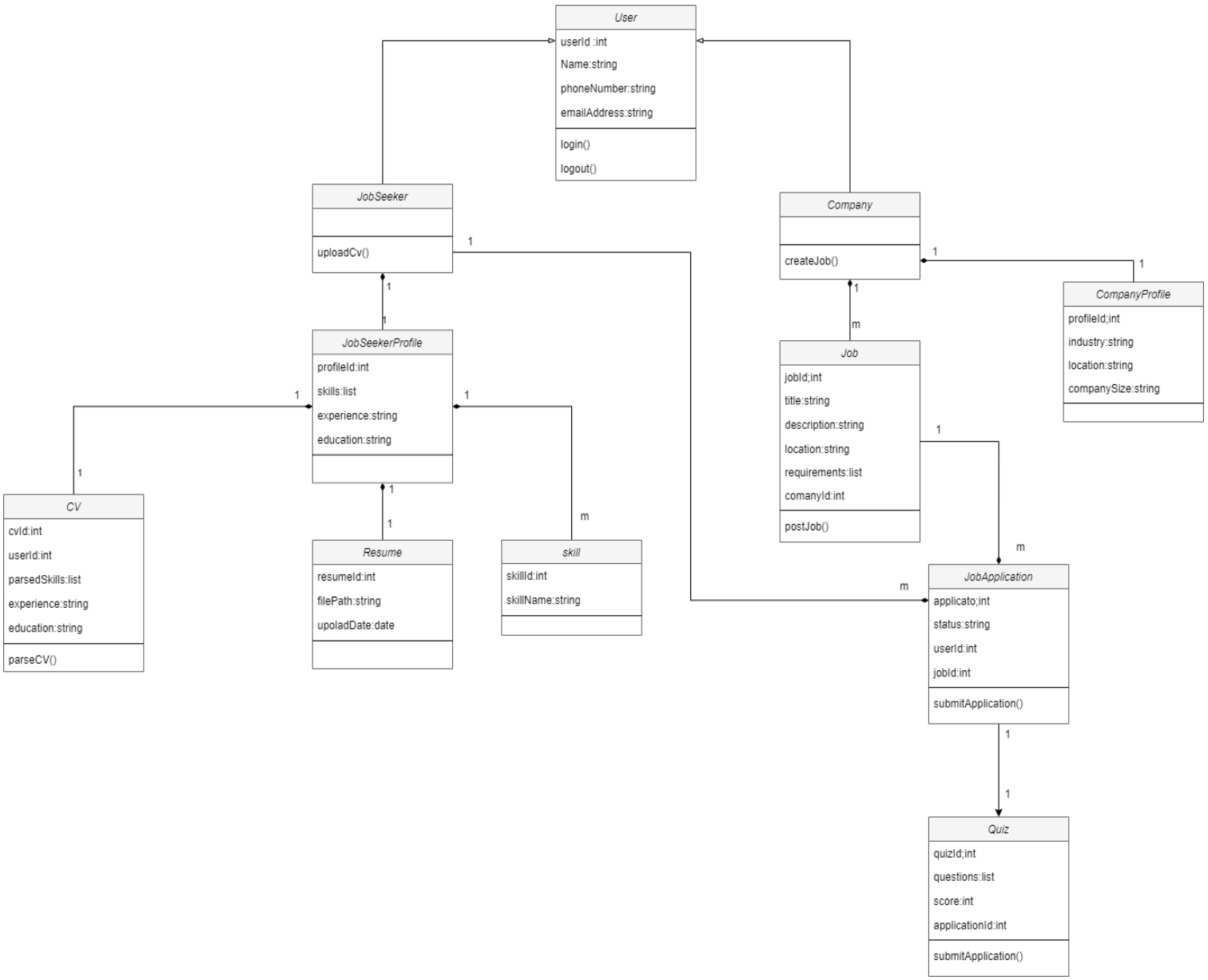
ID	UC-09
Participating Actors	Job Seeker,Company
Flow of Events	1. The job seeker or company logs into the system. 2. The user navigates to the messaging or chat section. 3. The system displays a list of ongoing conversations 4. The user selects an existing conversation or starts a new conversation by searching for the recipient (company or job seeker). 5. The system opens the chat interface, displaying any previous messages if available. 6. The user types a message and sends it. 7. The system processes and sends the message in real-time using WebSocket or HTTP. 8. The recipient receives a notification about the new message. 9. The recipient reads and replies to the message. 10. The system updates the conversation date and timestamps.
Alternative Flows	- A1: If the recipient is offline, the system stores the message and sends it when the recipient comes online.

	- A2: If the recipient has blocked the sender, the system prevents the message from being sent and notifies the sender. - A3: If there is a network issue, the system retries sending the message or notifies the sender of a failure.
Entry Conditions	The user must be logged into the system. The user must have access to the messaging feature.
Exit Conditions	The message is successfully sent and received, with the conversation's date updated.

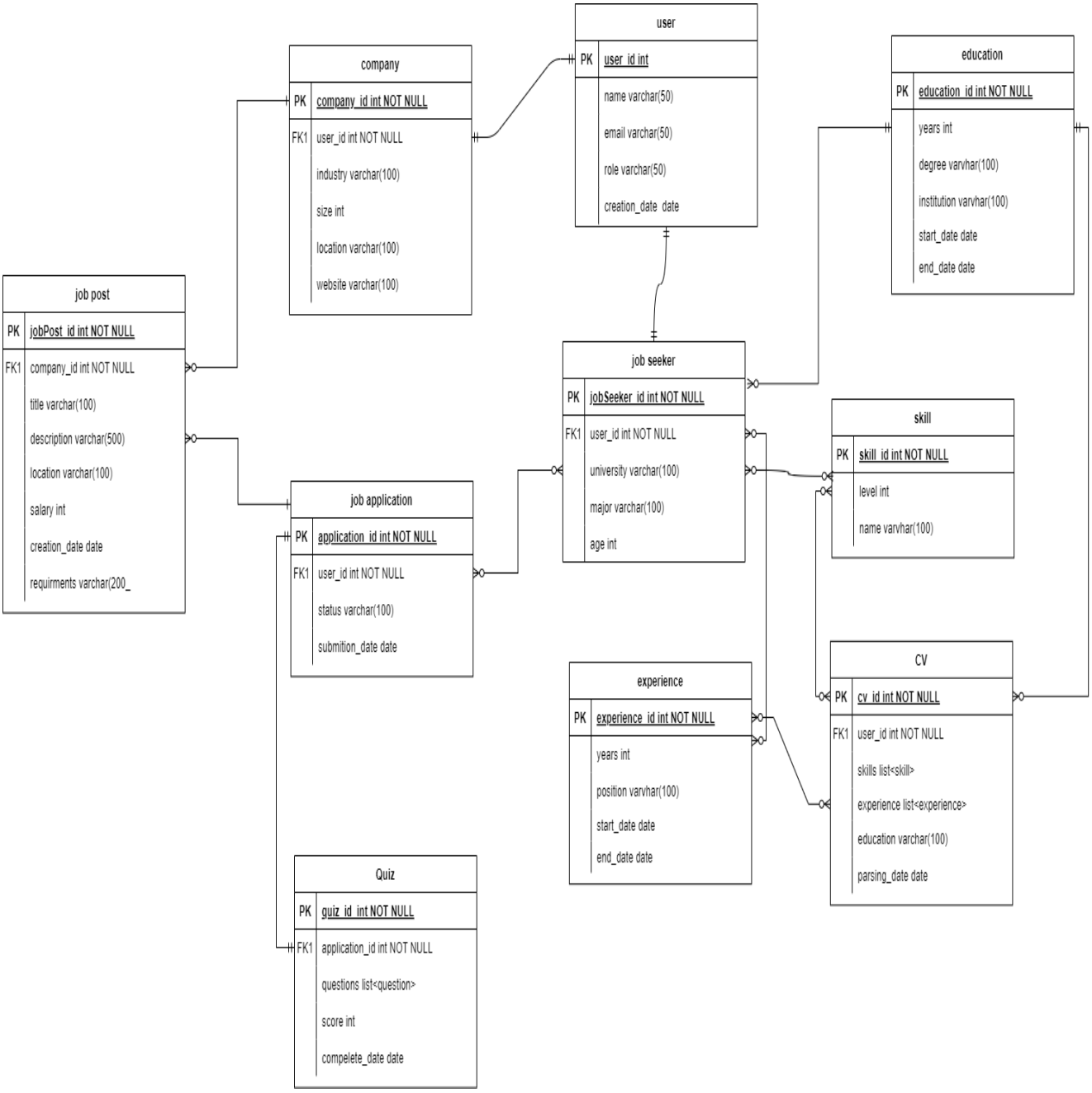
Sequence Diagram:



Analysis class diagram:



4.4.3. Entity Relationship Model Diagram ERD:



4.5. Initial Test Cases:

Test case scenario:		Sce-01: Manage Users (Admin)	
Test case id	Test case title	Test steps	Expected result
Tc-01	Verify successful user management	1. Log in as admin. 2. Navigate to "Manage Users". 3. Add/Edit/Delete a user.	User successfully added/edited/deleted.
Tc-02	Attempt to delete a non-existing user	1. Log in as admin. 2. Navigate to "Manage Users". 3. Try to delete a non-existing user.	Error message "User not found".

Test case scenario:		Sce-02: Manage Profile (Job Seeker)	
Test case id	Test case title	Test steps	Expected result
Tc-03	Update profile information	1. Log in as job seeker. 2. Navigate to "Manage Profile". 3. Edit personal details. 4. Save changes.	Profile successfully updated.
Tc-04	Attempt to save an empty profile	1. Log in as job seeker. 2. Navigate to "Manage Profile". 3. Clear all fields. 4. Save changes.	Error message "Profile fields cannot be empty".

Test case scenario:		Sce-03: Upload CV (Job Seeker)	
Test case id	Test case title	Test steps	Expected result
Tc-05	Upload a valid CV file	1. Log in as job seeker. 2. Navigate to "Upload CV". 3. Choose a valid CV file (PDF/DOC). 4. Click "Upload".	CV uploaded successfully.
Tc-06	Upload an invalid file type	1. Log in as job seeker. 2. Navigate to "Upload CV". 3. Choose an unsupported file format (e.g., PNG). 4. Click "Upload".	Error message "Invalid file format".

Test case scenario:		Sce-04: Search Jobs (Job Seeker)	
Test case id	Test case title	Test steps	Expected result
Tc-07	Perform a job search with valid keywords	1. Log in as job seeker. 2. Enter job title or keyword in the search bar. 3. Click "Search".	Relevant job results displayed.
Tc-08	Perform a job search with invalid keywords	1. Log in as job seeker. 2. Enter random or incorrect keywords. 3. Click "Search".	Error message "No jobs found".

Test case scenario:		Sce-05: Apply for Jobs (Job Seeker)	
Test case id	Test case title	Test steps	Expected result
Tc-09	Apply for a job successfully	1. Log in as job seeker. 2. Search for a job. 3. Click "Apply". 4. Confirm application.	Application submitted successfully.
Tc-10	Apply for a job without a CV	1. Log in as job seeker. 2. Search for a job. 3. Click "Apply" without uploading a CV.	Error message "CV required".

Test case scenario:		Sce-06: View Application Status (Job Seeker)	
Test case id	Test case title	Test steps	Expected result
Tc-11	Check status of submitted job applications	1. Log in as job seeker. 2. Navigate to "Application Status".	Status of all job applications displayed.

Test case scenario:		Sce-07: Manage Job Post (Company)	
Test case id	Test case title	Test steps	Expected result
Tc-12	Post a new job	1. Log in as company. 2. Navigate to "Manage Job Post". 3. Fill in job details. 4. Click "Post".	Job post successfully added.
Tc-13	Delete an existing job post	1. Log in as company. 2. Navigate to "Manage Job Post". 3. Delete an existing job.	Job post successfully deleted.

Test case scenario:		Sce-08: View Job Seeker Profiles (Company)	
Test case id	Test case title	Test steps	Expected result
Tc-14	View job seeker profile	1. Log in as company. 2. Search for a candidate. 3. Click on a candidate profile.	Candidate profile details displayed.

Test case scenario:		Sce-09: Communicate with Candidate (Company)	
Test case id	Test case title	Test steps	Expected result
Tc-15	Send a message to a candidate	1. Log in as company. 2. Search for a candidate. 3. Open messaging section. 4. Send a message.	Message successfully sent.

Test case scenario:		Sce-10: Match with Job (AI)	
Test case id	Test case title	Test steps	Expected result
Tc-16	AI matches candidate with job	1. Log in as job seeker. 2. Upload CV. 3. AI suggests jobs based on profile.	Relevant job suggestions displayed.

Test Case Scenario:		Sce-01: Check Login Functionality.	
Test case id	Test case title	Test steps	Expected result
Tc-17	Check results on entering a valid email and password.	1. Launch the application on the login page. 2. Enter your email and password. 3. Choose “login”.	The login should be successful
Tc-18	Check results on entering an invalid email, or password.	1. Launch the application on the login page. 2. Enter an ID and password. 3. Choose “login”.	Error message “invalid id or password.”
Tc-19	Check results when a user email is empty, and the “login” button is pressed.	1. Launch the application on the login page. 2. Enter a password. 3. Choose “login”.	Error message “a field is missing”

Test case scenario:		Sce-02: Check sign-up functionality.	
Test case id	Test case title	Test steps	Expected result
Tc-20	Check results on completing the sign-up form correctly, and select sign-up	1. Launch the application on the sign-up page. 2. Complete the form of your information. 3. Choose “sign-up”	The sign-up should be successful.
Tc-21	Check results on entering an existing email.	1. Launch the application on the sign-up page. 2. Complete the form of your information. 3. Choose “sign-up”	Error message “the email is already existed”.
Tc-22	Check results when a user chose a weak password (less than 8 characters).	1. Launch the application on the sign-up page. 2. Complete the form of your information. 3. Choose “sign-up”	Error message “weak password!”
Tc-23	Check results when a user email is empty, and the “login” button is pressed.	1. Launch the application on the sign-up page. 2. Complete the form of your information. 3. Choose “sign-up”	Error message “a field is missing”

4.6. Initial RTM Requirements Trackability Matrix:

Req-id	Use cases	analysis	System design	Detailed design	coding	Test cases
RE-FR-JS-1	UC-01: Create Account					TC-01
RE-FR-JS-2	UC-01: Create Account					TC-02
RE-FR-JS-3	UC-01: Create Account					TC-03
RE-FR-JS-4	UC-01: Create Account					TC-04
RE-FR-JS-5	UC-01: Create Account					TC-05
RE-FR-JS-6	UC-01: Create Account					TC-06
RE-FR-JS-7	UC-02: Upload CV					TC-07
RE-FR-JS-8	UC-02: Upload CV, UC-10: CV Parsing					TC-08
RE-FR-JS-9	UC-03: Apply for Jobs, UC-10: Match with Job					TC-09
RE-FR-JS-10	UC-11: Search Jobs					TC-10
RE-FR-JS-11	UC-11: Search Jobs					TC-11
RE-FR-JS-12	UC-03: Apply for Jobs					TC-12
RE-FR-JS-13	UC-03: Apply for Jobs					TC-13

RE-FR-JS-14	UC-07: Track Job Application					TC-14
RE-FR-JS-15	UC-01: Create Account					TC-15
RE-FR-JS-16	UC-11: Search Jobs					TC-16
RE-FR-CO-17	UC-04: Manage Users					TC-17
RE-FR-CO-18	UC-05: Manage Job Post					TC-18
RE-FR-CO-19	UC-06: Search for Candidates					TC-19
RE-FR-CO-20	UC-08: Manage Candidate Application					TC-20
RE-FR-CO-21	UC-06: Search for Candidates					TC-21
RE-FR-CO-22	UC-14: Review Quiz Results					TC-22
RE-FR-CO-23	UC-06: Search for Candidates					TC-23
RE-FR-CO-24	UC-09: Communication Between Job Seeker and Company					TC-24
RE-FR-CO-25	UC-04: Manage Users					TC-25
RE-FR-CO-26	UC-04: Manage Users					TC-26
RE-FR-AI-27	UC-10: Match with Job					TC-27
RE-FR-AI-28	UC-10: CV Parsing					TC-28
RE-FR-AI-29	UC-05: Manage Job Post					TC-29
RE-FR-SYS-30	UC-07: Track Job Application					TC-30

RE-FR-SYS-31	UC-04: Manage Users					TC-31
RE-FR-SYS-32	UC-04: Manage Users					TC-32
RE-FR-SYS-33	UC-04: Manage Users					TC-33
RE-FR-CH-34	UC-09: Communication Between Job Seeker and Company					TC-34
RE-FR-CH-35	UC-09: Communication Between Job Seeker and Company					TC-35
RE-FR-QZ-36	UC-13: Monitor Quizzes					TC-36
RE-FR-QZ-37	UC-12: Take Quiz					TC-37

4.7. AI-Powered Features Analysis

4.7.1. Resume Parsing:

4.7.1.1. Problem Statement

Modern recruitment systems are inundated with large volumes of resumes, which come in various unstructured formats. Manually screening these documents is inefficient, error-prone, and inconsistent. The primary goal is to develop a system that automatically extracts key information—such as a candidate's name, email address, skills, education, and work experience—from these unstructured resumes and converts it into a structured format (e.g., JSON). This structured data can

then be used for candidate matching, analytics, and integration with other parts of the recruitment process.

4.7.1.2. Data Characteristics

I. **Unstructured Nature:**

Resumes are typically provided in formats such as PDF, DOCX, or plain text. Each resume may have a different layout, formatting style, and order of information, making the extraction of structured data challenging.

II. **Key Entities to Extract:**

The essential information typically includes:

- **Personal Details:** Name, email, phone number, and address.
- **Professional Details:** Skills, work experience, and projects.
- **Educational Background:** Degrees obtained, universities attended, and graduation dates.

III. **Noise and Variability:**

Resumes often contain extraneous text (headers, footers, formatting artifacts) that must be removed during preprocessing. Variations in formatting (e.g., multi-column layouts, different fonts or sizes) require robust handling.

4.7.1.3. Challenges

I. **Annotation Complexity:**

To train a robust model, high-quality annotations are required. Creating these annotations is time-consuming, and ensuring that entity spans are correctly aligned with the text tokens is challenging.

II. **Format Variability:**

Because resumes differ widely in structure, the system must be flexible enough to generalize across different layouts while still accurately identifying key entities.

III. **Error Handling:**

Some resumes might have overlapping or misaligned annotations. The model must be designed to either correct or gracefully handle these errors during both training and inference.

IV. **Scalability:**

The system should process large volumes of resumes quickly and accurately. This requires efficient data preprocessing, storage (e.g., using spaCy's DocBin), and a scalable deployment framework such as FastAPI.

4.7.1.4. Requirements

I. **Accuracy:** High precision in extracting candidate information.

II. **Robustness:** Ability to handle diverse resume formats and noisy data.

III. **Efficiency:** Fast preprocessing and real-time inference capabilities.

IV. **Integration:** Structured output in a standard JSON format to enable downstream processes.

4.7.2. Job Recommendation:

4.7.2.1. Problem Definition

Recruitment platforms need to match candidate profiles with job openings accurately. The challenge is to quantify the degree of match based on unstructured text data in resumes and job descriptions.

- **Unstructured Data:**
  Both resumes and job descriptions are typically unstructured texts that require transformation into numerical representations.
- **Objective:**
  Convert text into embeddings that capture semantic meaning and then compute similarity scores to rank candidates against job requirements.
- **Challenges:**
  - **Variability:** Different resumes and job descriptions use varied language and formatting.
  - **High Dimensionality:** Embeddings are high-dimensional vectors, and efficient computation of similarity (using cosine similarity) is critical.
  - **Scalability:** The system should handle a large volume of data and update recommendations dynamically.

4.7.2.2. Requirements

- **Accuracy:**
  The system must accurately represent the semantic content of resumes and job descriptions.

- **Efficiency:**
  Embedding generation and similarity computation must be fast to support real-time recommendations.
- **Integration:**
  The solution should expose a REST API (via FastAPI) for generating embeddings and calculating similarity.
- **Cost Efficiency:**
  Use open-source models (e.g., Sentence Transformers) locally to avoid API costs.

4.7.3. AI-Generated Quiz System.

4.7.3.1. Problem Definition

The primary challenge is to automatically generate test questions that:

- Evaluate candidates on specific technical or practical skills.
- Conform to a fixed output format (JSON) so that downstream systems can easily parse and integrate them.
- Provide consistent and reproducible questions despite the inherent variability in model outputs.

4.7.3.2. Data Characteristics

- **Input Data:**
  The system uses a list of skills as input (e.g., "Python", "Data Science")

along with parameters such as difficulty level and question type (multiple choice or true/false).

- **Output Format:**
  The output must be a well-defined JSON object containing:
  - "question": The question text.
  - "options": A list of four options.
  - "correct_answer": The correct option number (as a string).

4.7.3.3. Challenges

- **Variability in Model Output:**
  Transformer models can include internal reasoning (e.g., <think> sections) or return outputs with inconsistent formatting. Ensuring a stable JSON format requires robust parsing and prompt design.
- **Ensuring Correct Answer Consistency:**
  The correct answer must be given as an option number (e.g., "2"), but model outputs may vary (sometimes providing literal text, numbers, or mixed formats).
- **Prompt Engineering:**
  Crafting a prompt that guides the model to produce consistent, structured output is nontrivial. The prompt must include explicit instructions and examples.
- **Cost and Resource Constraints:**
  While using local, open-source models minimizes cost, ensuring low latency and resource efficiency (e.g., managing GPU/CPU load) is essential for real-time performance.

Chapter5 System Design

5.1. Introduction

This chapter provides a detailed description of the design phase for the Smart HR Recruitment System, focusing on the system's architecture, component interactions, and detailed design. The goal is to create a modular, scalable, and efficient system that connects job seekers with employers while leveraging microservices for job matching, resume parsing, quiz management, and communication functionality. The design phase ensures the system meets both functional and non-functional requirements within the scope of a student project.

5.2. System Architecture

5.2.1. Overview

The system design follows a microservices-based architecture, where each service operates independently and communicates through APIs. This approach ensures modularity, scalability, and ease of maintenance. Each microservice is responsible for a specific functionality and can be deployed, updated, or scaled independently.

The following **non-functional requirements** were considered while designing the system:

5.2.2. Non-Functional Requirements

- **Simplicity**: The microservices are designed to be straightforward, focusing on specific tasks to reduce complexity.
- **Security**: Implement basic user authentication and ensure secure communication between services using HTTPS.

- **Performance**: The system should handle up to 1000 concurrent users and provide responses within 3-5 seconds for most operations.
- **Scalability**: Each microservice can be scaled independently based on demand, ensuring flexibility for future growth.
- **Usability**: The user interface is designed to be intuitive and accessible for all user roles.

5.2.3. Architectural Design

The system consists of the following microservices, each with a dedicated responsibility:

5.2.3.1. Microservices

The system consists of the following microservices, each with a dedicated responsibility:

Architecture Diagram:

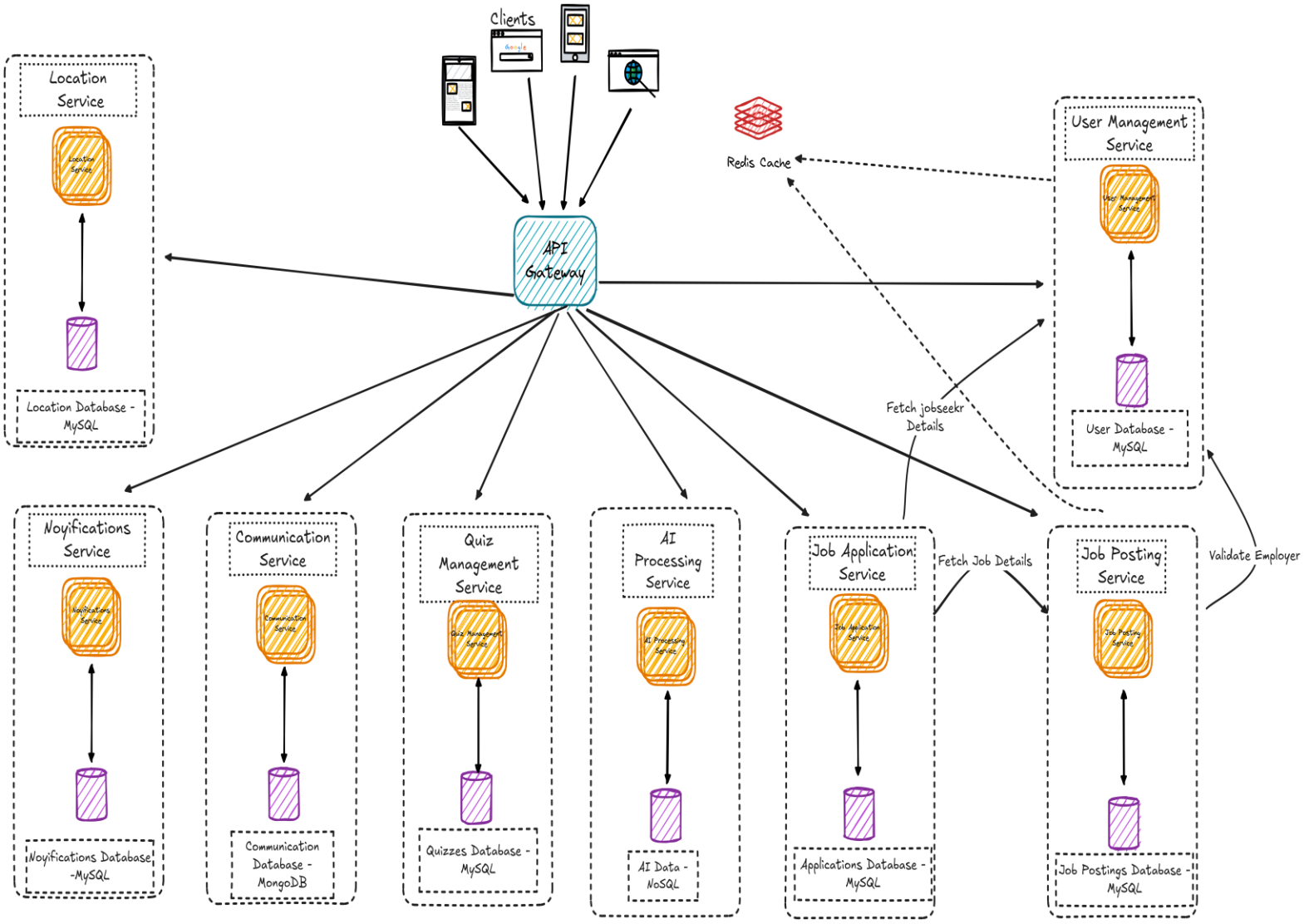


Figure 1 system architecture

- **API Gateway**: Serves as the single entry point for all client requests, routing them to the appropriate microservices.
- **User Management Service**: Handles user registration, authentication, and profile management.
- **Job Posting Service**: Allows employers to post, edit, and delete job listings.

- **Job Application Service**: Manages job applications, including submission and status tracking.
- **AI Processing Service**: Provides AI-powered job matching, resume parsing, and quiz generation support.
- **Quiz Management Service**: Manages quizzes generated for specific jobs, tracks quiz results, and associates them with job applications.
- **Communication Service**: Facilitates real-time chat between job seekers and employers.
- **Notification Service**: Sends real-time notifications to users about application updates, job recommendations, and messages.

5.2.3.2. Data Flow

- **Job Seekers** interact with the API Gateway to access services like profile management, job search, applications, and communication.
- **Employers** use the API Gateway to post jobs, review applications, manage candidate communications, and quizzes.
- The **AI Processing Service** analyzes job descriptions and resumes to provide personalized job recommendations and generate quizzes.
- The **Notification Service** sends updates based on triggers from other services, such as application submissions or quiz completions.

5.2.3.3. Communication Between Microservices

The microservices communicate using **RESTful APIs**, ensuring loose coupling. For example:

- When a job seeker applies for a job, the **Job Application Service** notifies the **Notification Service** to send a confirmation message.

- The **AI Processing Service** fetches data from the **User Management Service** and the **Job Posting Service** to generate job recommendations.
- The **Quiz Management Service** fetches job details from the **Job Posting Service** and collaborates with the **AI Processing Service** to generate quizzes.
- The **Communication Service** validates user identities through the **User Management Service** and retrieves application details from the **Job Application Service** to allow chats.

5.3. Detailed Design

5.3.1. Database Design

Each microservice has its own database to ensure data encapsulation and independence. Key database designs include:

- **User Management Database**: Stores user credentials, profiles, and roles.
- **Job Posting Database**: Contains job details, including title, description, company, and location.
- **Job Application Database**: Tracks applications, including applicant ID, job ID, and status.
- **AI Processing Database**: Stores processed data, such as extracted keywords, job matches, and quiz content.
- **Quiz Management Database**: Stores quizzes, quiz questions, and results.
- **Communication Database**: Tracks messages exchanged between users.
- **Notification Database**: Stores notification logs and statuses.

5.3.2. Component Interaction

Key interactions between microservices include:

- The **API Gateway** routes client requests to the appropriate service.
- The **Notification Service** is triggered by events in the **Job Application Service**, **Quiz Management Service**, and **Communication Service**.
- The **AI Processing Service** interacts with the **User Management Service** and **Job Posting Service** to generate job recommendations and quizzes.
- The **Quiz Management Service** associates quiz results with job applications and updates the **Job Application Service**.
- The **Communication Service** retrieves job application details from the **Job Application Service** to enable valid chats between job seekers and employers.

5.4. Design Constraints

- **Basic AI Implementation**: The AI module uses simple keyword matching and rule-based algorithms for job matching and quiz generation to reduce complexity and computational requirements.
- **Concurrent Users**: The system is designed to support up to 1000 concurrent users during testing, with scalability options for future expansion.
- **Simplified Chat System**: The Communication Service supports basic real-time messaging without advanced features like media sharing or typing indicators.

5.5. Conclusion

The **Smart HR Recruitment System** leverages a microservices architecture to ensure modularity, scalability, and ease of maintenance. Each service focuses on a specific functionality, such as job management, quiz handling, or communication,

simplifying development and deployment. This design is practical and achievable for a student project, providing a strong foundation for implementation and testing in subsequent phases.

5.6. Detailed Design for Each Service

5.6.1. User Management Service

The **User Management Service** is a fundamental component of the **Smart HR Recruitment System**, responsible for managing **user authentication, registration, and profile management**. It enables job seekers, companies, and administrators to securely access the platform and manage their accounts**.** The service ensures seamless integration with other system components, facilitating smooth interactions across various functionalities.

5.6.1.1. Role of User Management Service in the System

The **User Management Service** is responsible for handling **user-related operations**, ensuring **secure authentication and role-based access control**. It enables job seekers to **create and manage profiles**, while companies can **manage their corporate accounts**.

This service interacts with multiple other components:

- **Job Posting Service**: Ensures **only verified companies** can post job listings.
- **Job Application Service**: Allows job seekers to **apply for jobs** using their profiles.
- **Communication Service**: Enables real-time **chat between authenticated users**.
- **Notification Service**: Sends **alerts** about **job applications, interviews, and profile updates**.

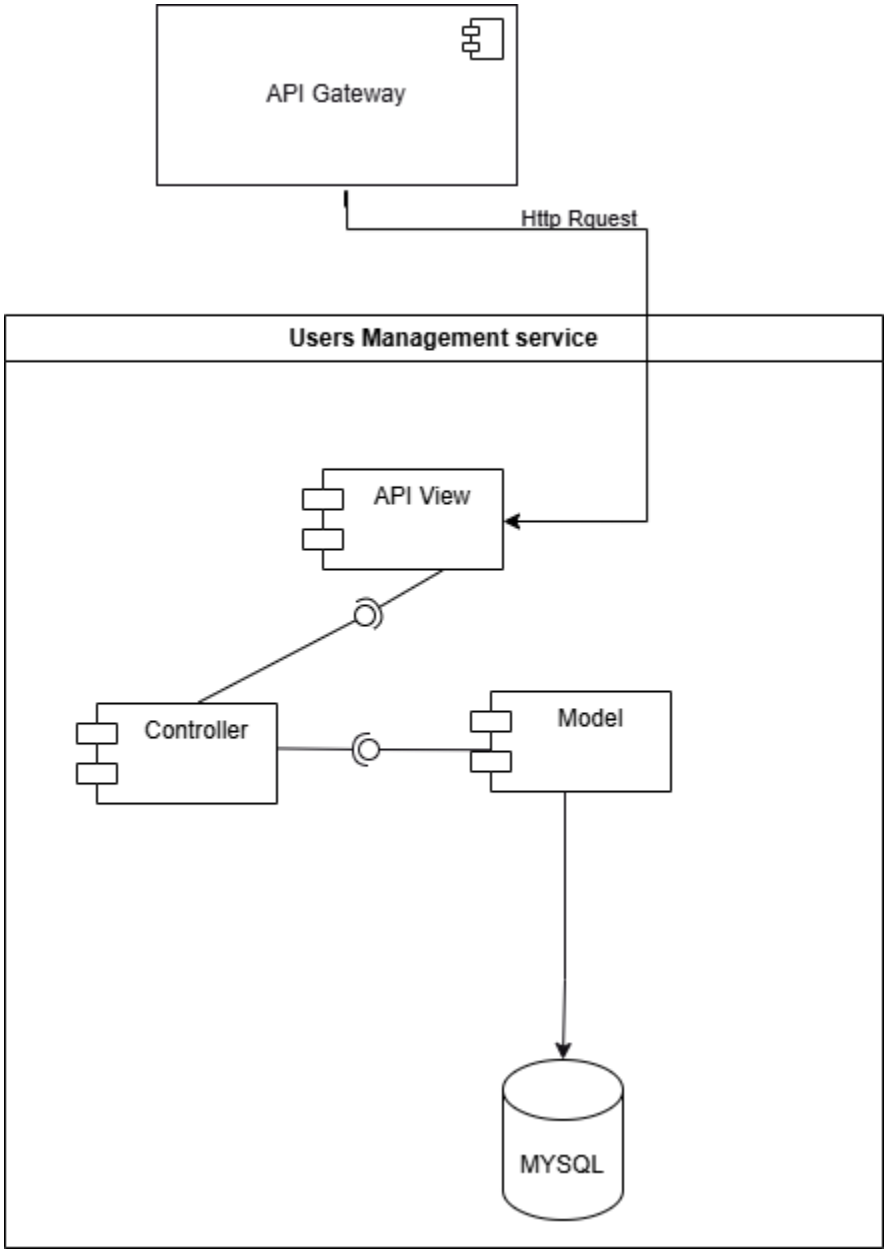
By serving as the **foundation of user identity and access control**, the **User Management Service** plays a crucial role in ensuring security, data integrity, and smooth interactions between job seekers and employers.

5.6.1.2. Responsibilities

- **User Registration**: Allows new users (job seekers, companies, and admins) to register and create an account.
- **Authentication & Authorization**: Implements **secure login mechanisms** and **role-based access control**.
- **Profile Management**: Enables users to update personal details, upload profile pictures, and manage preferences.
- **Company Profile Management**: Allows employers to manage company details and job postings.
- **Password Management**: Provides users with secure password change and recovery mechanisms.
- **User Verification**: Ensures data integrity and prevents unauthorized access through email verification and security tokens.

With these functionalities, the **User Management Service** guarantees a secure and seamless experience for all users.

5.6.1.3. Component diagram:



5.6.1.4. Components

- **API Gateway**:
    - The **API Gateway** acts as a single entry point for external client requests. It routes HTTP requests related to user management to this service.
    - It ensures abstraction, security, and efficient request routing.
- **User Management Service**:
    - This service handles the core logic for user management operations. It consists of the following components:

a. API View:

- The **API View** serves as the interface for handling incoming HTTP requests from the API Gateway.
- It validates incoming requests, processes them, and sends responses.
- It calls the **Controller** for executing business logic.

b. Controller:

- The **Controller** contains the business logic of the service.
- It processes data received from the **API View** and interacts with the **Model** to fetch or manipulate data.

For example:

It validates user credentials for login.

It processes user profile updates.

c. Model:

1. The **Model** represents the database schema and provides an abstraction for the MySQL database.
2. It defines the structure of user-related data (e.g., email, password, profile details) and ensures data consistency.
3. For example:
    1. It defines the User class for storing user data.
    2. It includes relationships like OneToOne and ManyToMany for entities like JobSeeker or Company.

d. MySQL Database:

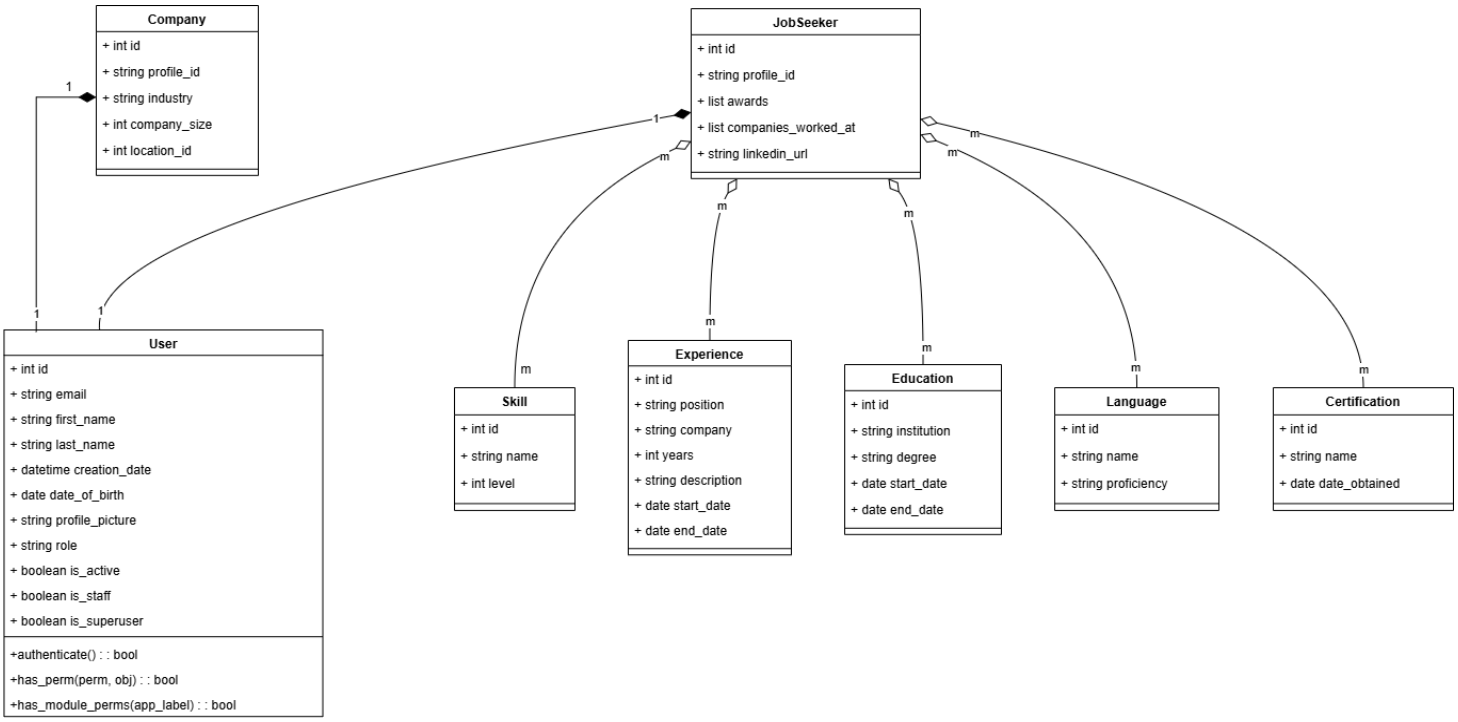
4. The MySQL database is used to store persistent data for this service.
5. It contains tables for user details, roles, and related information.

5.6.1.5. Interactions

- **API Gateway to API View**:
    - The API Gateway routes user-related HTTP requests to the **API View** of the User Management Service.
- **API View to Controller**:
    - The **API View** forwards validated requests to the **Controller**, which applies business logic to handle the request.
- **Controller to Model**:
    - The **Controller** interacts with the **Model** to fetch or manipulate user data stored in the database.
- **Model to Database**:

- The **Model** directly interacts with the MySQL database to perform operations such as querying, updating, or deleting user data.

5.6.1.6. Class Diagram:



5.6.1.7. Design Patterns

- **MVC Pattern**: The service uses the **Model-View-Controller (MVC)** architecture:
  - **Model**: Handles data storage and database interaction.
  - **View (API View)**: Acts as the interface for client communication.
  - **Controller**: Implements the business logic for processing requests.

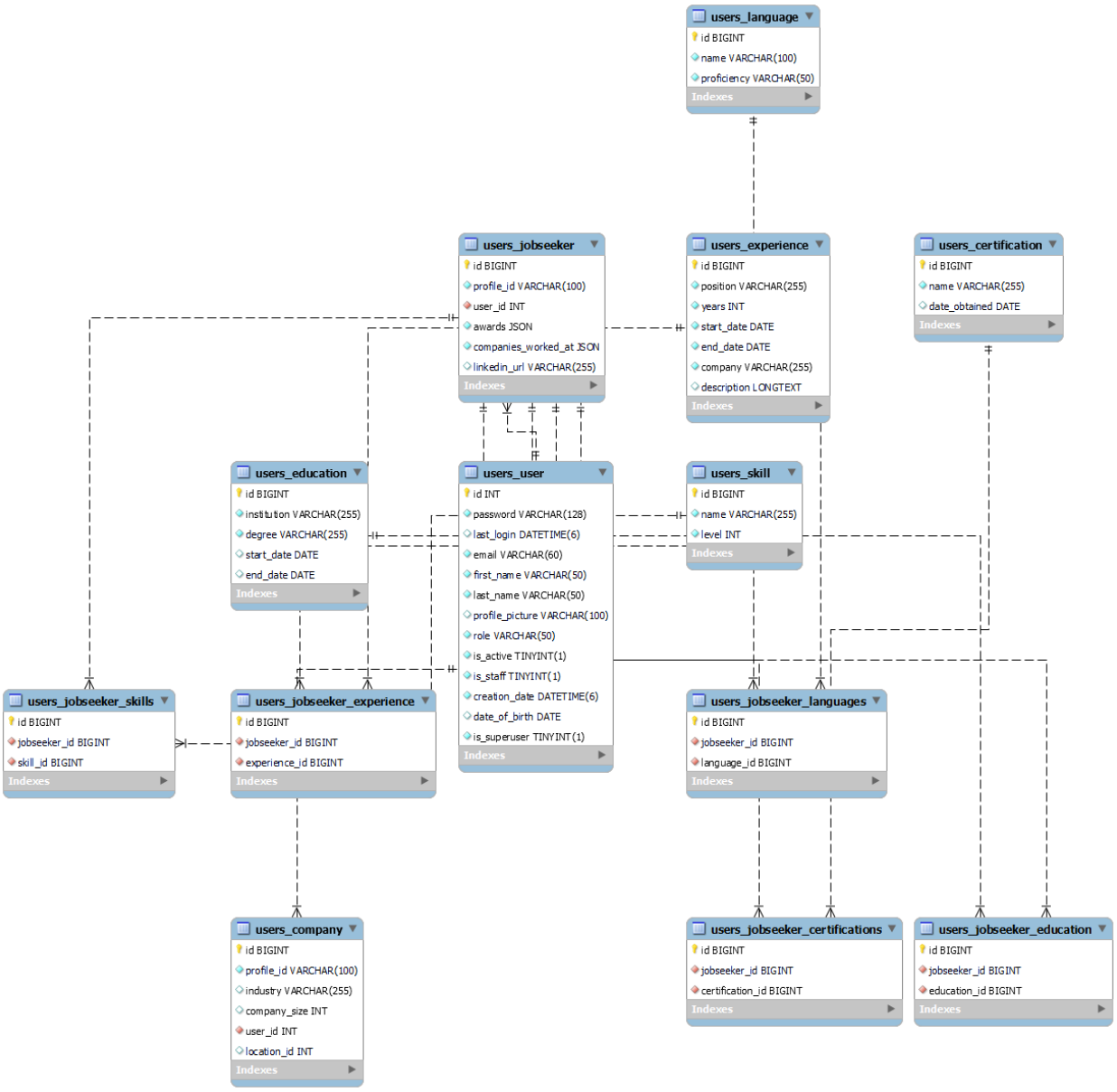
5.6.1.8. Data Flow

- A client sends a request (e.g., POST /register) via the API Gateway.

- The API Gateway forwards the request to the **API View** in the User Management Service.
- The **API View** validates the input and forwards it to the **Controller**.
- The **Controller** processes the request and interacts with the **Model** to access the database.
- The **Model** performs the required operation (e.g., create a new user, update profile).
- The **Controller** returns a response to the **API View**, which sends it back to the client via the API Gateway.

5.6.1.9. Database

The database is a fundamental part of the User Management Service, as it stores and manages data related to users, job seekers, companies, and authentication credentials. This service relies on MySQL, a relational database management system (RDBMS), to ensure structured data storage, high performance, and integrity. Additionally, Redis is used as a caching layer to enhance authentication speed and session management.


users_language
id BIGINT
name VARCHAR(100)
proficiency VARCHAR(50)
Indexes
users_jobseeker
id BIGINT
profile_id VARCHAR(100)
user_id INT
awards JSON
companies_worked_at JSON
linkedin_url VARCHAR(255)
Indexes
users_experience
id BIGINT
position VARCHAR(255)
years INT
start_date DATE
end_date DATE
company VARCHAR(255)
description LONGTEXT
Indexes
users_certification
id BIGINT
name VARCHAR(255)
date_obtained DATE
Indexes
users_education
id BIGINT
institution VARCHAR(255)
degree VARCHAR(255)
start_date DATE
end_date DATE
Indexes
users_user
id INT
password VARCHAR(128)
last_login DATETIME(6)
email VARCHAR(60)
first_name VARCHAR(50)
last_name VARCHAR(50)
profile_picture VARCHAR(100)
role VARCHAR(50)
is_active TINYINT(1)
is_staff TINYINT(1)
creation_date DATETIME(6)
date_of_birth DATE
is_superuser TINYINT(1)
Indexes
users_skill
id BIGINT
name VARCHAR(255)
level INT
Indexes
users_jobseeker_skills
id BIGINT
jobseeker_id BIGINT
skill_id BIGINT
Indexes
users_jobseeker_experience
id BIGINT
jobseeker_id BIGINT
experience_id BIGINT
Indexes
users_jobseeker_languages
id BIGINT
jobseeker_id BIGINT
language_id BIGINT
Indexes
users_company
id BIGINT
profile_id VARCHAR(100)
industry VARCHAR(255)
company_size INT
user_id INT
location_id INT
Indexes
users_jobseeker_certifications
id BIGINT
jobseeker_id BIGINT
certification_id BIGINT
Indexes
users_jobseeker_education
id BIGINT
jobseeker_id BIGINT
education_id BIGINT
Indexes

5.6.1.10. API Endpoints

Method	Endpoint	Description
GET	/users/	List all users or create a new user
GET/PUT/DELETE	/users/<int:pk>/	Retrieve, update, or delete a specific user by ID
GET/POST	/jobseekers/	List all job seekers or create a new job seeker
GET/PUT/DELETE	/jobseekers/<int:pk>/	Retrieve, update, or delete a specific job seeker by ID
GET/POST	/companies/	List all companies or create a new company
GET/PUT/DELETE	/companiesByUserId/<int:pk>/	Retrieve, update, or delete a company by user ID
GET/PUT/DELETE	/companiesByCompanyId/<int:pk>/	Retrieve, update, or delete a company by company ID
POST	/users/change-password/<int:pk>/	Change the password of a user
POST	/register/	Register a new user
POST	/parse-cv/	Parse CV for extracting job seeker details

5.6.2. Job Posting Service

The Job Post Service is a key component of the comprehensive smart recruitment system designed to facilitate interaction between companies and job seekers. The service aims to provide a dynamic and automated platform for companies to post available job openings, enabling job seekers to view and apply for them with ease. By leveraging this service, the communication efficiency between the two parties is enhanced, accelerating the overall recruitment process.

5.6.2.1. Role of Job Post Service in the System

The Job Post Service is responsible for publishing job openings provided by companies. These job listings are then available for job seekers to browse and choose according to their skills and experience. In addition, the service provides the option for companies to modify or delete job postings as needed. As a result, it serves as a crucial link between companies seeking employees and individuals looking for job opportunities.

This service is not isolated in the system; it is integrated with several other services. It is directly connected to the User Management Service, which manages the data of both companies and job seekers. It also interacts with the Job Application Service, allowing job seekers to apply for the positions they are interested in. Therefore, the "Job Post Service" plays a central role in improving the recruitment experience for both companies and job seekers.

5.6.2.2. Responsibilities

The "Job Post Service" is responsible for a variety of core tasks to ensure the efficient management and availability of job listings for all stakeholders involved, including:

1. Adding a New Job Posting: Responsible for allowing companies to post new job openings by providing comprehensive details such as the title, description, requirements, and salary.
2. Viewing All Job Postings: Ensures that users (both companies and job seekers) can view the list of available job openings in the system.
3. Job Search: Facilitates job seekers in searching for job opportunities using various filters such as location, job category, or job type.
4. Updating Job Postings: Allows companies to modify or update the details of existing job postings as necessary.
5. Deleting a Job Posting: Enables companies to remove job postings that are no longer relevant or needed.

And more functionalities are available to further enhance the experience for both companies and job seekers.

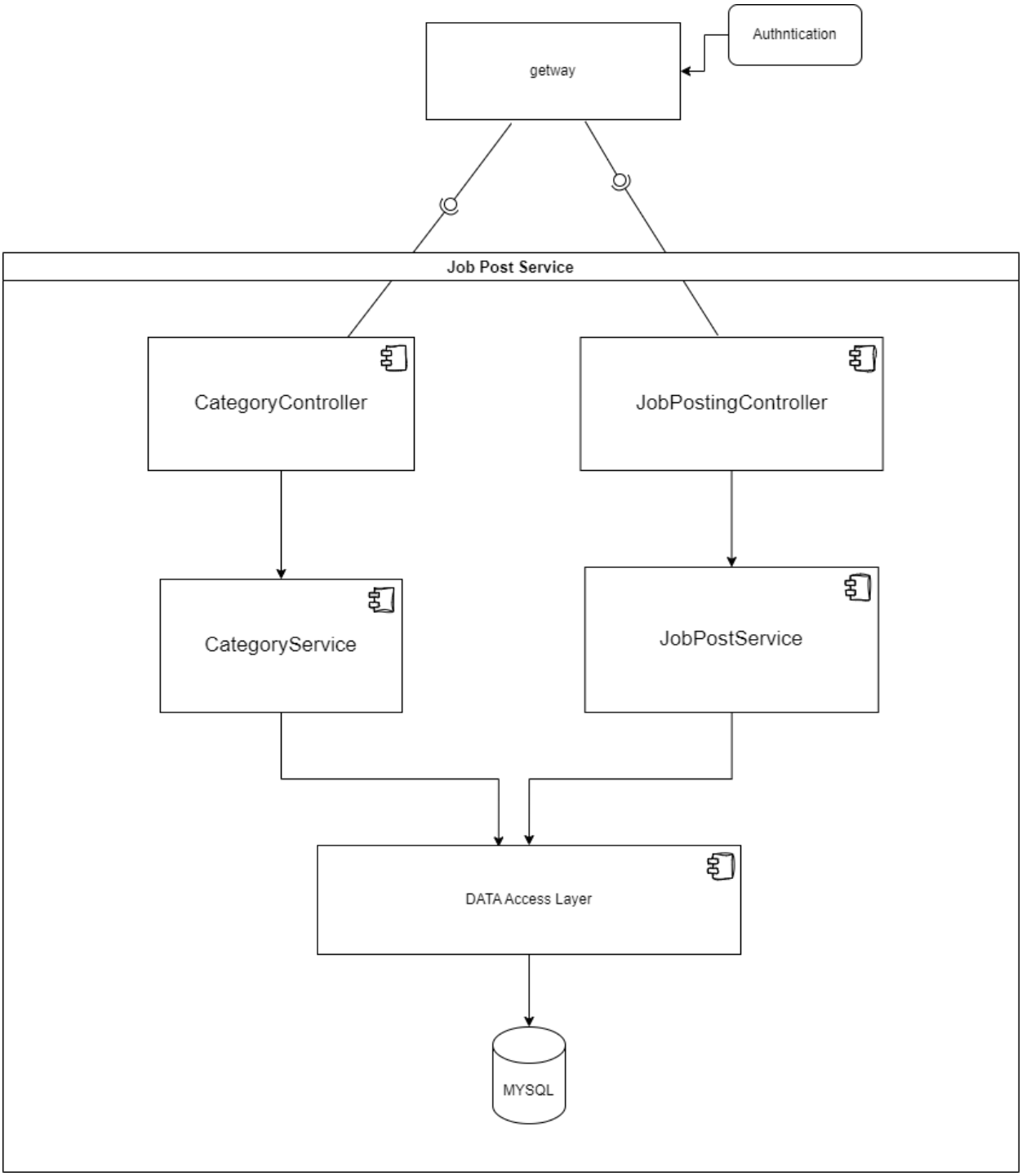
5.6.2.3. Component Diagram

The Component Diagram is one of the diagram types in Unified Modeling Language (UML), used to represent the structure of software systems at the component level and their interactions. This diagram illustrates how the system is divided into independent components and how these components interact with each other through interfaces or ports.

In the Job Posting Service system, the component diagram is used to represent the relationships between services, repositories, controllers, and entities within the system. This diagram provides a clear understanding of how the code is organized and helps development teams define the responsibilities of each component, facilitating system development and maintenance.

Objectives of the Component Diagram in this project:

- Illustrate the general architecture of the Job Posting Service system.
- Define the relationships between different software components.
- Facilitate the understanding of data flow paths between entities.
- Assist developers in separating tasks between different services, achieving scalability and flexibility.


Authntication
getway
Job Post Service
CategoryController
JobPostingController
CategoryService
JobPostService
DATA Access Layer
MYSQL

5.6.2.4. Application structure:

The Job Post Service application is based on a Microservices architecture, which allows the division of services into small, independent components, making development, scaling, and maintenance easier. The system relies on an API Gateway to route requests, along with an Authentication mechanism to ensure secure access to the services.

The application is designed according to the MVC (Model-View-Controller) pattern, where the three layers are separated to ensure code flexibility and ease of management. The architecture consists of:

- Controllers: Serve as the interface between the user and the service, receiving requests, processing them, and passing them to the appropriate services.
- Services: Contain the business logic of the system, executing operations related to job posts and categories before interacting with the database.
- Data Access Layer: Includes entities and repositories that use JPA to interact with the database, ensuring efficient data management regardless of the database technology used.

The general architecture for future services built using Spring Boot will follow the same structure, ensuring consistency and reliability across the system.

5.6.2.5. Components

The CategoryController is responsible for handling category-related requests. It acts as an intermediary between the Gateway and the CategoryService, ensuring that requests for creating, updating, or deleting categories are efficiently processed and forwarded to the appropriate service.

Similarly, the JobPostingController manages job posting requests, receiving them from the Gateway and directing them to the JobPostService. This ensures that job advertisements can be created, updated, or removed as needed.

The CategoryService contains the business logic for managing categories. It interacts with the Data Access Layer, which includes the Model and Repository, to perform database operations such as adding, updating, and retrieving category data.

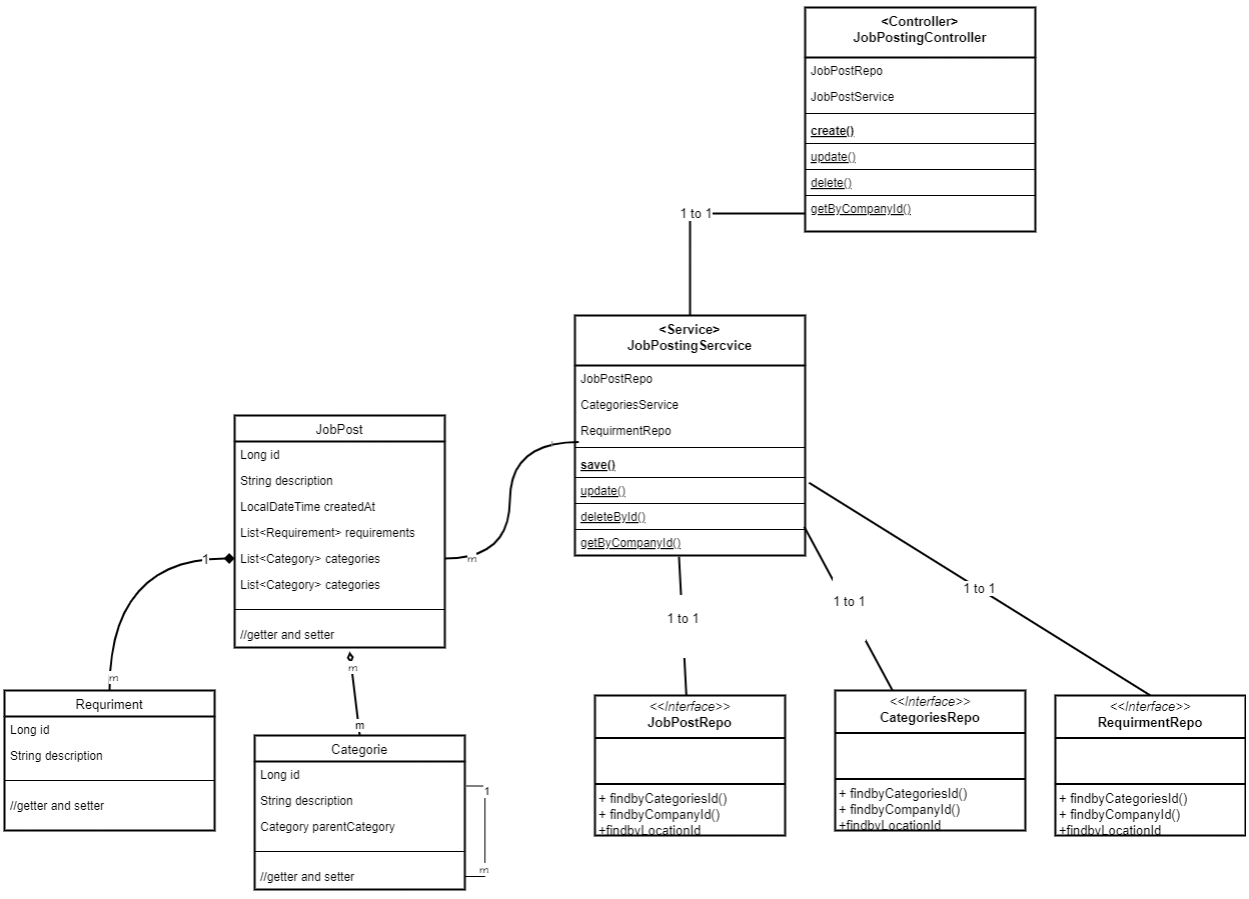
The JobPostService plays a similar role for job postings, managing the logic required to create, modify, or delete job advertisements. It relies on the Data Access Layer to execute these operations, ensuring data consistency and proper management.

At the core of data management, the Data Access Layer consists of Model and Repository components and utilizes JPA (Java Persistence API) for efficient database interaction. The Model defines the structure of category and job post entities, while the Repository facilitates CRUD (Create, Read, Update, Delete) operations on the database. By leveraging JPA with MySQL, this layer ensures seamless object-relational mapping (ORM), efficient query execution, and data persistence across the system.

To further illustrate the structure and relationships within the Job Post Service application, a Class Diagram is provided. This diagram details the key classes, their attributes, methods, and the relationships between them. It serves as a blueprint for understanding how the system's components interact at the code level, ensuring clarity for developers and stakeholders alike.

5.6.2.6. Class Diagram:

The Class Diagram for the Job Post Service application highlights the core classes and their interactions. Below is a breakdown of the main classes and their responsibilities:

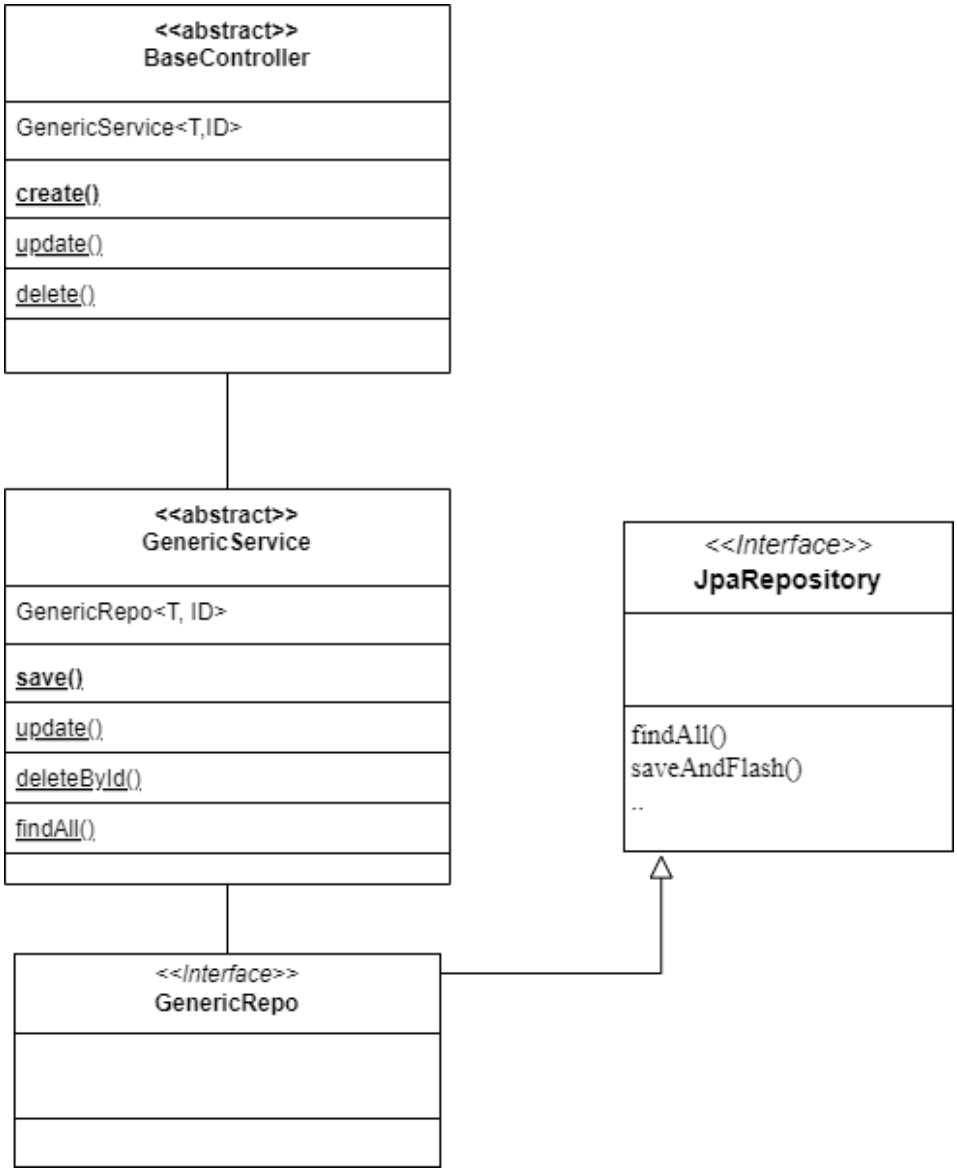


5.6.2.7. Design Patterns

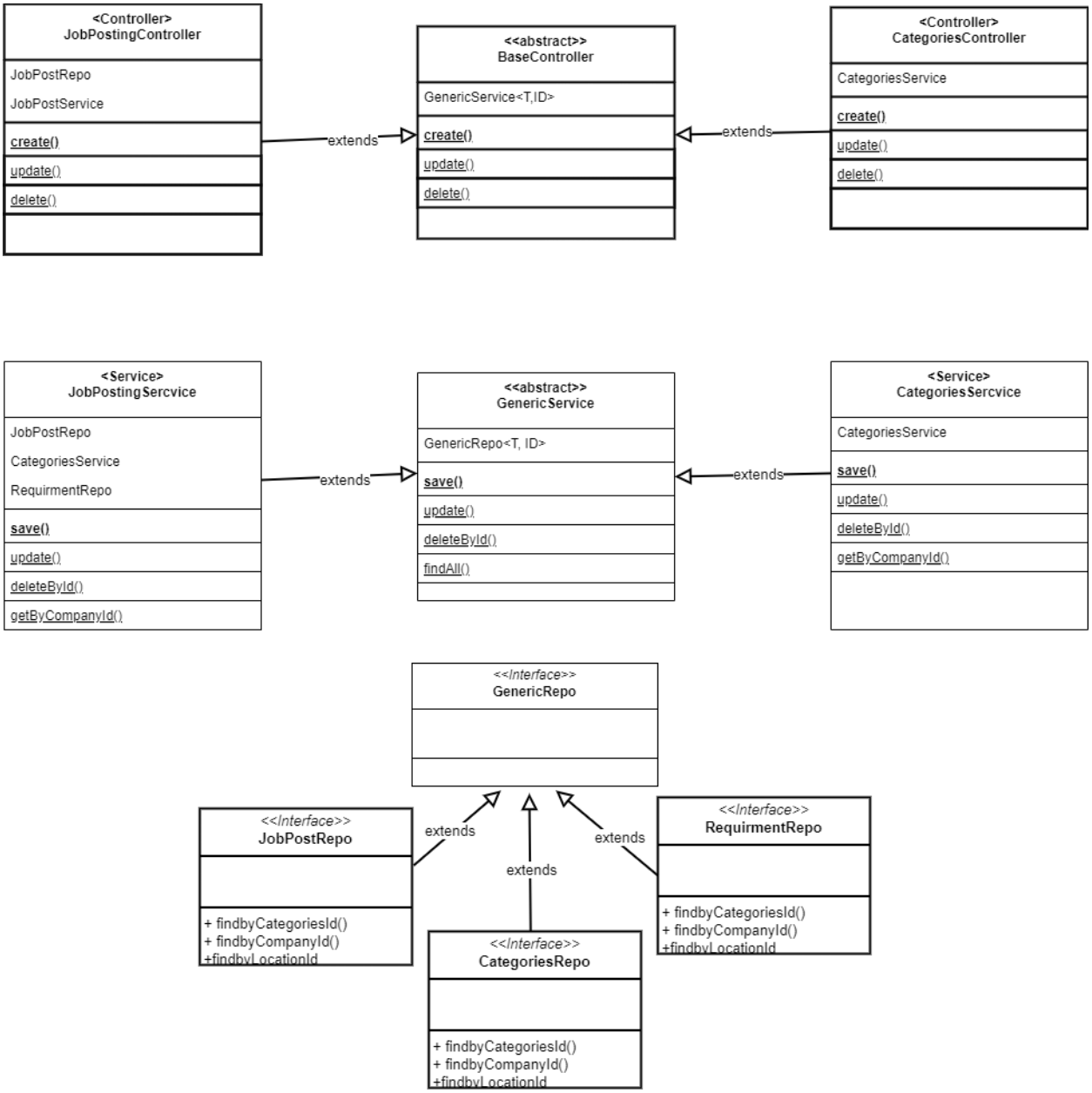
In the design of **Job Post Service**, multiple **design patterns** have been utilized to ensure a flexible and reusable architecture. These patterns help improve the maintainability and scalability of the service while reducing code duplication.

One of the patterns used is the **Template Method Pattern**, which allows for the creation of a common logic structure while enabling customization of certain steps in subclasses. To achieve this, **Generic Programming** has been applied, enabling the development of reusable components that work with multiple data types without the need to duplicate code.

The service architecture is designed so that all **controllers** extend from the base class **BaseController**, all **services** extend from **GenericService**, and **repositories** extend from **GenericRepo**, providing a unified framework for managing data across different entities in the system.

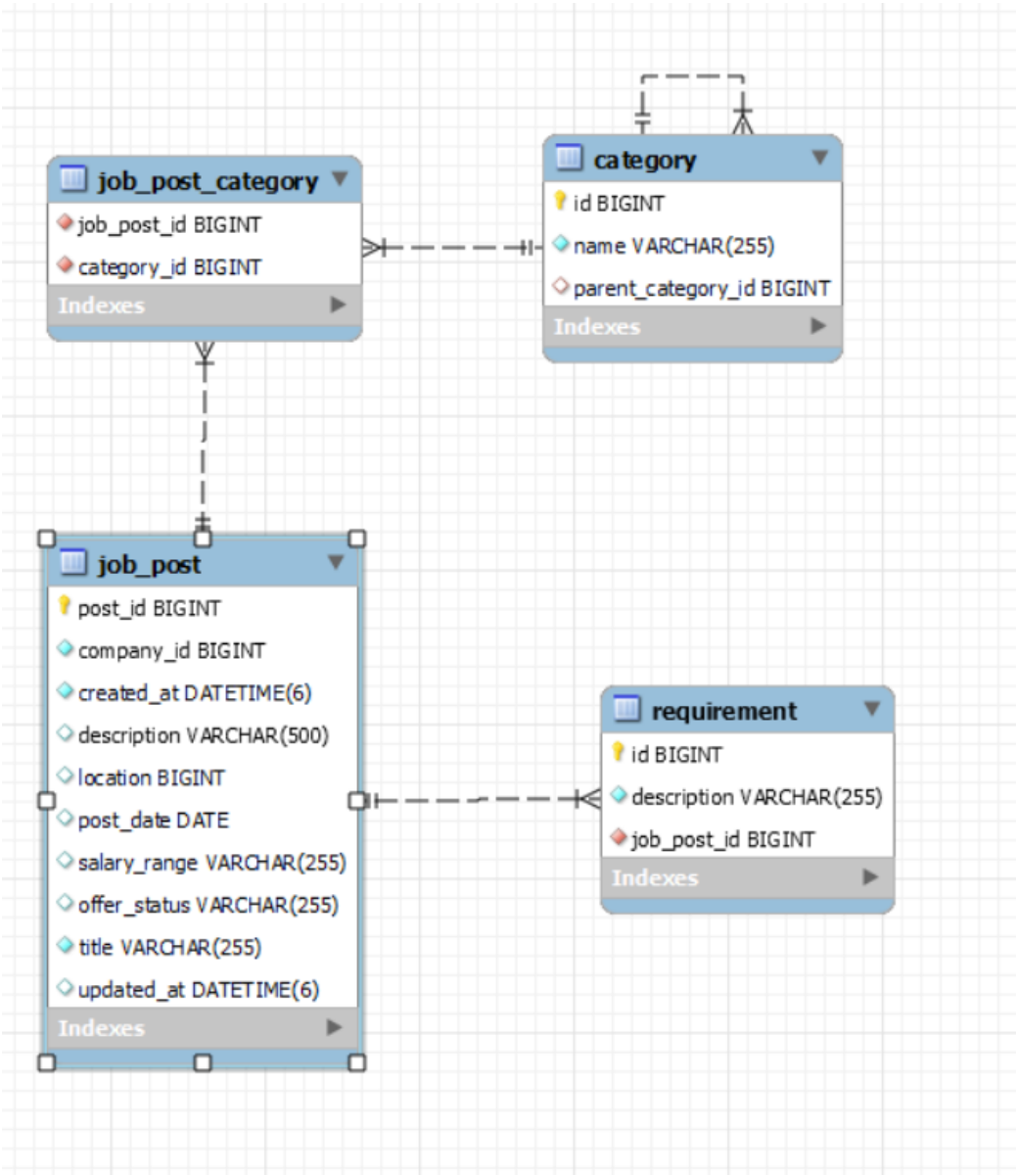

<<abstract>>
BaseController
GenericService<T,ID>
create()
update()
delete()
<<abstract>>
GenericService
GenericRepo<T, ID>
save()
update()
deleteById()
findAll()
<<Interface>>
JpaRepository
findAll()
saveAndFlash()
..
<<Interface>>
GenericRepo

5.6.2.8. Relationship with other classes:



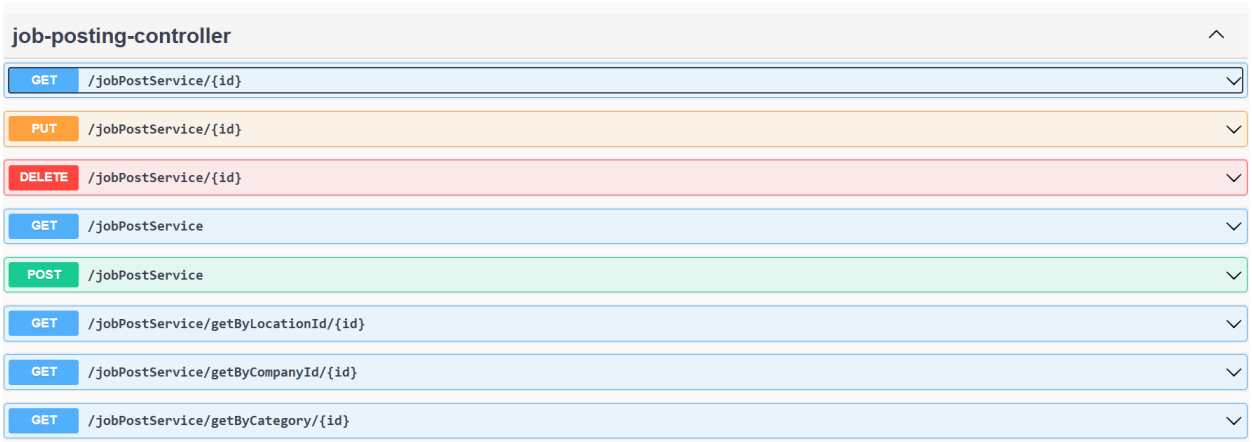
5.6.2.9. Database

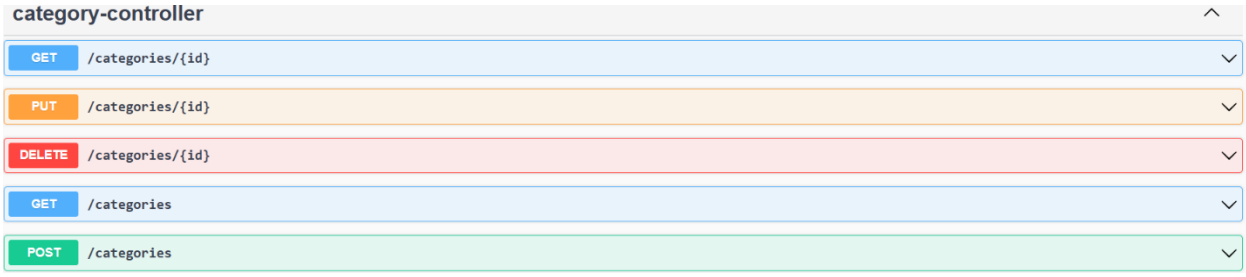
The database plays a crucial role in the **JobPost_Service** system, where all data related to job posts, companies, and job requirements are stored and managed. The system utilizes **MySQL** as its relational database management system (RDBMS), ensuring high performance and flexibility in data handling.



5.6.2.10. API Endpoints

The **Job Post Service** provides a set of RESTful APIs to manage job postings, ensuring seamless interaction between job seekers, companies, and the system. Below is an overview of the key endpoints available in the service.





5.6.3. Communication Service

The **Communication Service** is one of the core components of the system, playing a crucial role in enabling real-time interaction between users. This service is designed to ensure seamless information exchange between job seekers and employers, facilitating recruitment processes and providing an efficient means of direct communication.

The service is built using **Spring Boot** and leverages **WebSockets**, allowing for bidirectional communication between users in real time. Additionally, messages are stored in **MongoDB**, ensuring they can be retrieved when needed.

5.6.3.1. Role of the Communication Service in the System

The **Communication Service** acts as the backbone of the messaging system, offering a reliable communication channel between different system entities. Through this service, job seekers and employers can exchange messages quickly and efficiently, making hiring, interviews, and job offer negotiations more convenient.

Furthermore, the service integrates with other system components, such as the **User Management Service** and the **Job Posting Service**, ensuring a seamless and well-connected user experience.

5.6.3.2. Responsibilities

The **Communication Service** is responsible for various key functions to ensure its efficiency and stability, including:

1. **Managing Real-Time Messaging**
    - Providing an instant communication channel via **WebSockets** and **STOMP**, enabling real-time message exchange.
    - Supporting the sending and receiving of text messages between job seekers and employers.
2. **Message Security**
    - Encrypting messages and securing data transmission between users.
3. **Message Routing and Delivery**
    - Managing message routing to the intended recipients based on chat IDs.
    - Ensuring message order consistency and preventing message loss during transmission.
4. **Message Storage and Retrieval**
    - Using **MongoDB** to store chat history, allowing users to access their previous conversations at any time.
    - Supporting message search functionality for easy retrieval of information.

5.6.3.3. Data Flow in WebSocket Communication

1. Opening the WebSocket Connection

The frontend establishes a WebSocket connection with the backend through the /ws endpoint. The STOMP protocol is used to facilitate and manage the communication between the client and server.

2. User Subscription to Message Channels

Once the WebSocket connection is established, the client subscribes to a dedicated message queue, such as /user/queue/messages, to receive messages directed specifically to them.

3. Sending Messages from the Client

When a user sends a message, it is transmitted via WebSocket to the server through the /app/chat.sendMessage endpoint. The message typically includes:

- Sender's ID
- Recipient's ID
- Chat ID
- Message content

4. Message Processing on the Server

Upon receiving the message at /chat.sendMessage, the server:

- Validates the data
- Stores the message in the database
- Determines the recipient for message delivery

5. Forwarding the Message to the Recipient

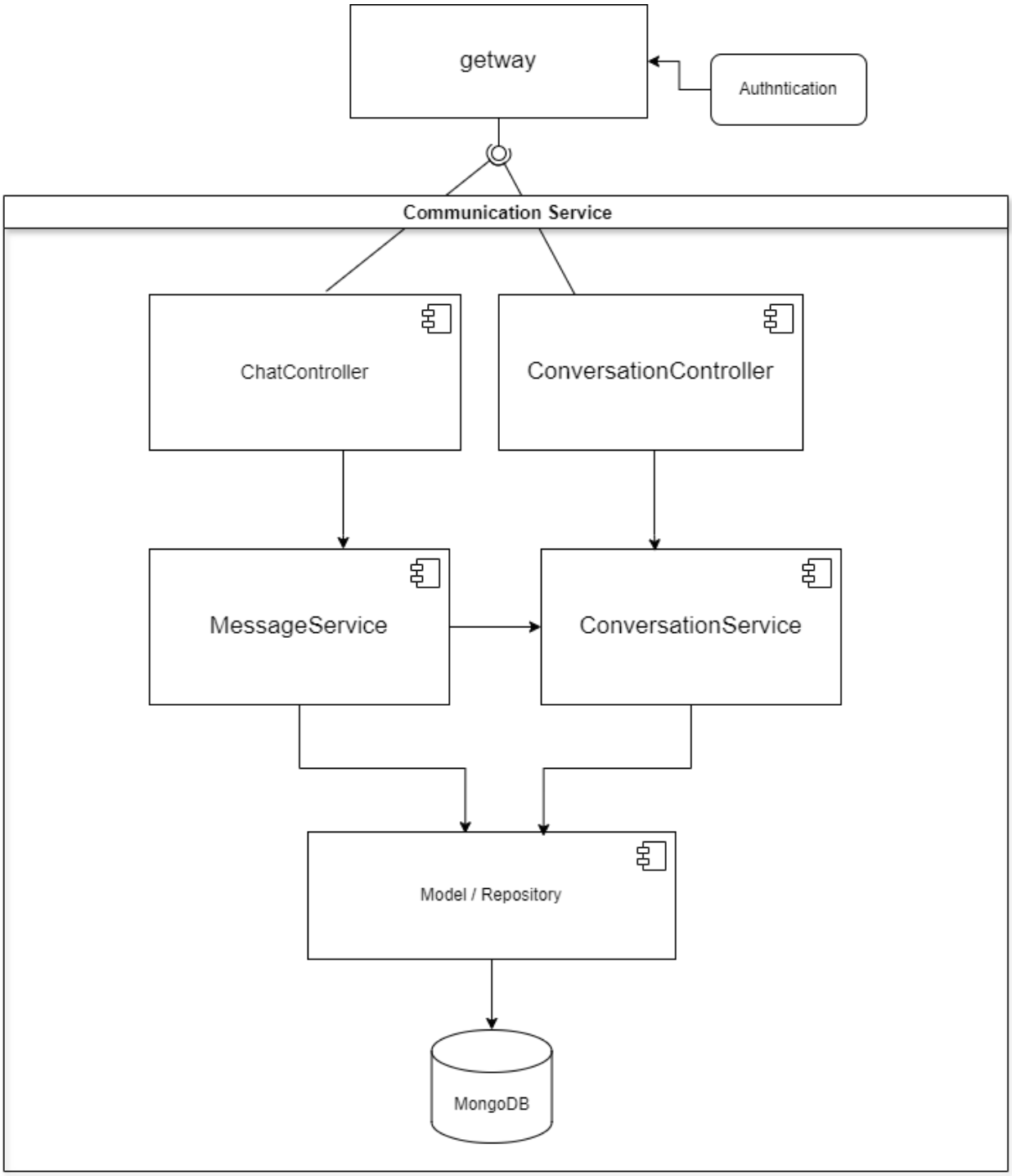
After processing, the server forwards the message to the intended recipient via WebSocket on the /user/{recipientId}/queue/messages endpoint, ensuring direct delivery.

6. Receiving and Displaying Messages on the Client

The recipient, having subscribed to /user/queue/messages, receives the message in real time and displays it on the frontend interface without requiring a page refresh.

This flow ensures efficient, real-time communication between users using WebSocket and STOMP.

5.6.3.4. Component Diagram



5.6.3.5. Components

1. **ChatController**

   The ChatController is responsible for handling requests related to chats, such as sending messages or retrieving chat history. This component receives requests from the Gateway and forwards them to the MessageService to process the logical operations related to messages. This ensures that all chat-related requests are processed quickly and efficiently.
2. **ConversationController**

   The ConversationController manages requests related to conversations, such as creating new conversations or adding participants. It receives requests from the Gateway and directs them to the ConversationService to execute the required operations. This ensures that all requests related to conversations are processed correctly and systematically.
3. **MessageService**

   The MessageService contains the logic for managing messages, such as sending, editing, or deleting them. This component interacts with the Model/Repository to access the database and perform operations like saving or retrieving messages. This ensures that all operations related to messages are handled smoothly and efficiently.
4. **ConversationService**

   Similarly, the ConversationService contains the logic for managing conversations, such as creating them or updating their status. This component also interacts with the Model/Repository to access the database and perform operations like saving or retrieving conversations. This ensures

that all operations related to conversations are performed consistently and systematically.

5. **Model/Repository**
   The Model/Repository layer is responsible for managing the data related to messages and conversations. It contains Models that define the data structure, such as messages and conversations, and provides Repositories to interact with the database and perform CRUD (Create, Read, Update, Delete) operations. MongoDB is used as the database to store the data in a flexible and efficient manner.
6. **MongoDB**
   MongoDB serves as the main database for storing data related to messages and conversations. This database stores data such as individual messages, conversation details, and participants, providing high performance and flexibility in handling real-time data.

5.6.3.6. Explanation of the Relationship Between Components Using DI

In the Communication Service , the relationships between internal components are managed using the principle of Dependency Injection. This principle allows components to be decoupled from the dependencies they require, making the system more flexible and testable. For
example, ChatController and ConversationController depend
on MessageService and ConversationService, respectively. However, instead of creating these services internally, they are "injected" through
the Constructor or Setter.
Similarly, MessageService and ConversationService depend

on Model/Repository to access data, and these dependencies are also injected. This approach ensures that each component focuses solely on its own logic without worrying about how its dependencies are created, enhancing maintainability and reusability.

This applies to what was mentioned previously in Job Post Service.

Secondary Components Overview:

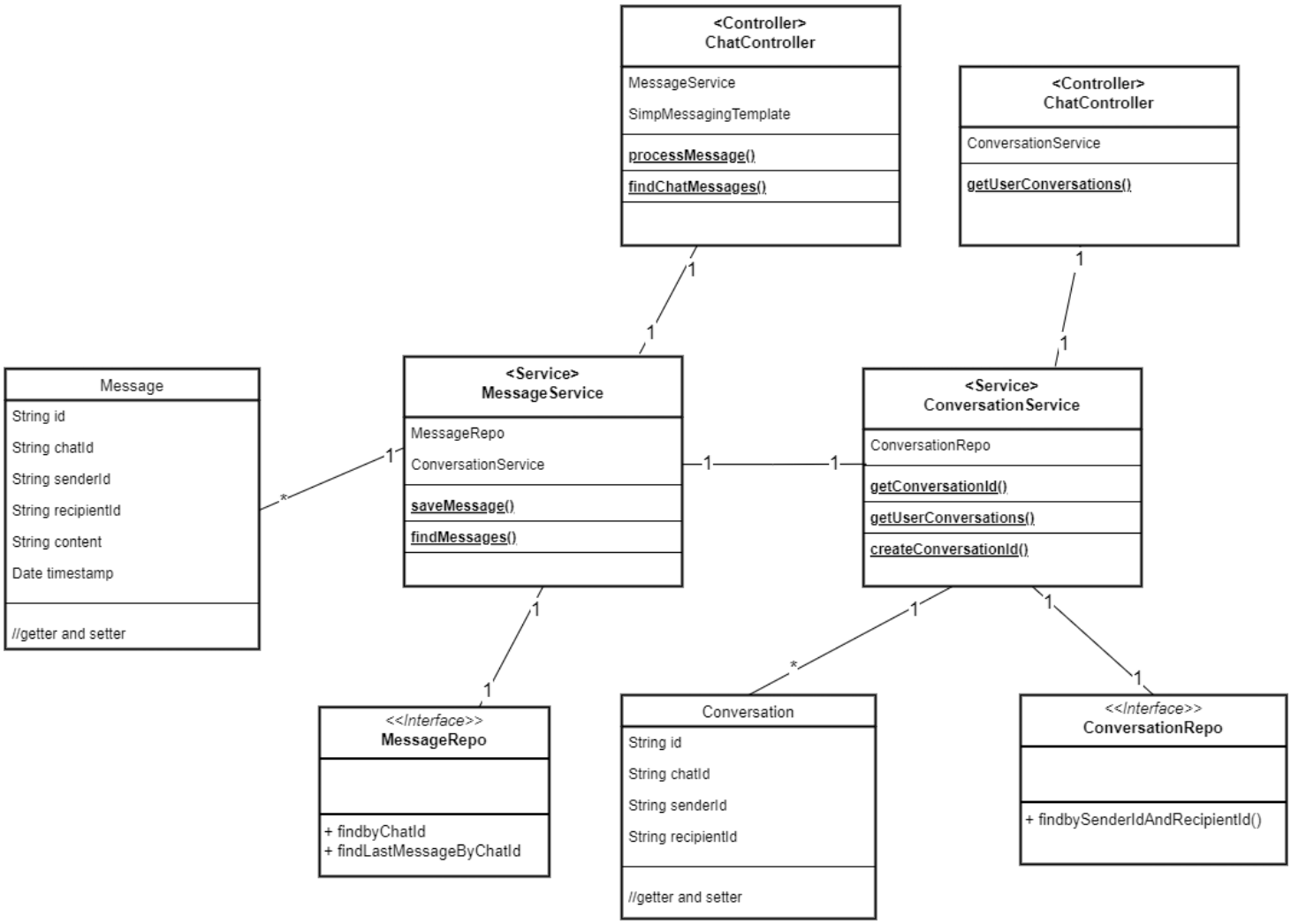
1. config >> SocketConfig
   The SocketConfig component is responsible for configuring WebSocket settings and managing the overall connection setup. It ensures that the WebSocket server is properly initialized, and it manages the STOMP protocol to facilitate communication between the client and server. This configuration component also handles aspects like authentication, security, and connection policies, making sure that the communication is established and maintained efficiently.

2. DTO (Data Transfer Objects)
   - MessageDTO: The MessageDTO is a data transfer object that represents the structure of a message being sent or received. It encapsulates details such as the sender's ID, recipient's ID, chat ID, message content, and timestamp, allowing messages to be transmitted in a structured format across the system.
   - ChatNotificationDTO: The ChatNotificationDTO is used for sending notifications to users regarding new messages or updates in their conversations. This DTO contains information such as the notification type, the user to be notified, and the content of the

notification, ensuring that users are alerted in real-time about important activities in their chat.

- ConversationDTO: The ConversationDTO represents the structure of a conversation. It holds details like the participants, the conversation's status, and metadata related to the conversation. This DTO is used to transfer conversation data between different layers of the application, such as when creating new conversations or retrieving conversation details.

5.6.3.7. Class Diagram



5.6.3.8. Database

MongoDB has been used as the primary database for the communication service to store and manage data related to messages and conversations using **documents**. **MongoDB** offers flexibility and ease in handling real-time data, making it ideal for

chat applications. Instead of using tables and rows as in relational databases, data is stored in **documents** that represent data objects, making it easier to handle non-static or dynamic data structures

Conversation in Document


```
_id: ObjectId('679e5193c8756d7fdd62ff47')
chatId : "ABC_AMER"
senderId : "ABC"
recipientId : "AMER"
_class : "com.example.Communication_Service.Model.Conversation"
```

```
_id: ObjectId('679e5193c8756d7fdd62ff48')
chatId : "ABC_AMER"
senderId : "AMER"
recipientId : "ABC"
_class : "com.example.Communication_Service.Model.Conversation"
```


Message in Document


```
_id: ObjectId('679cfa7da2964039cbf4605f')
chatId : "ABC_AMER"
senderId : "AMER"
recipientId : "ABC"
content : "HI"
timestamp : 2025-01-31T16:29:49.879+00:00
_class : "com.example.Communication_Service.Model.Message"
```

```
_id: ObjectId('679e47608134d8053335728e')
chatId : "AMER_YASSER"
senderId : "YASSER"
recipientId : "AMER"
encryptedMessage : "q/1y/SGAKMaS61uP2cxo8YCg0EKH"
aesKeyForRecipient : "khodplnOP5ThAQX6kV3A0JvKzDaJ6HIsfMugrbmu9ozBIp1dTt/QgW7Ggccl5PpmlsFonl…"
_class : "com.example.Communication_Service.Model.Message"
```

5.6.3.9. Comparison Between Relational and Non-Relational Databases for Communication Service

Criterion	Relational Database	Non-Relational Database (MongoDB)
Structure	Fixed tables with relationships between tables	Flexible documents storing data as JSON objects
Flexibility	Less flexible; structure must be defined in advance	More flexible; data can be added or modified easily without affecting others
Performance	May suffer performance issues when handling large amounts of data or heterogeneous data	High performance in handling large and dynamic real-time data
Scalability	Vertical scaling (adding more powerful machines)	Horizontal scaling (adding more servers easily)
Data Integrity	Supports data integrity through foreign keys and relationships	No strict data integrity enforcement; stores heterogeneous data
Ideal Use Case	Applications requiring complex relationships between data (e.g., banking systems)	Applications needing flexibility and speed in real-time data processing (e.g., chat applications)

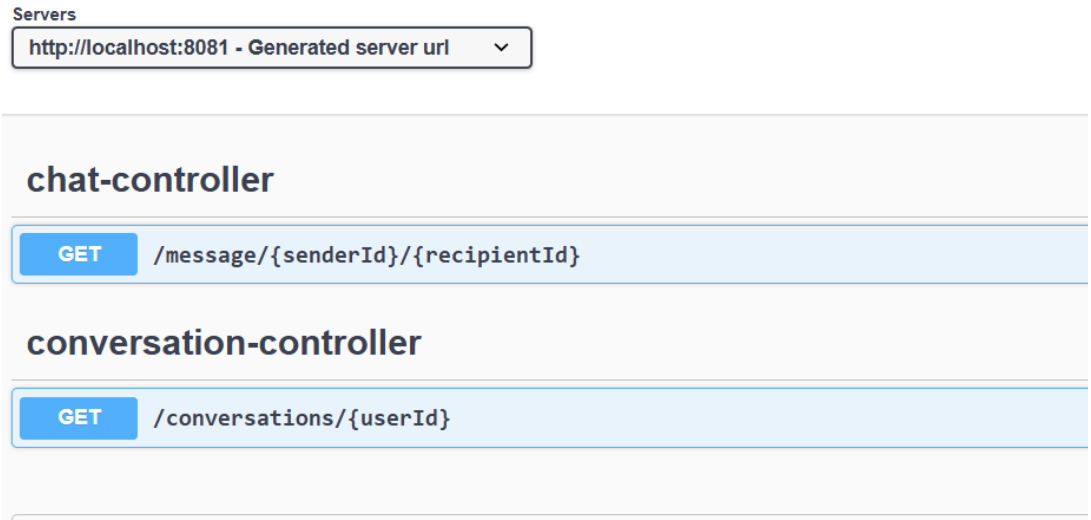
For the **Communication Service**, using **MongoDB** (a non-relational database) is an ideal choice due to its flexibility in handling dynamic and heterogeneous data, as

well as its high performance in real-time data processing, such as messages and conversations. Relational databases, on the other hand, are more suited for applications that require complex relationships between data but are less efficient in handling fast-changing and unstructured data like that in chat applications.

5.6.3.10. API Endpoints

API endpoints are an essential part of the communication service, allowing interaction between the client and the server. Some endpoints use REST API for operations like sending messages and retrieving conversations, while others use WebSocket like the /chat endpoint, providing real-time interaction through STOMP. These non-REST endpoints help improve performance and provide a seamless experience without needing to refresh the page.

REST end Points:



5.6.4. Quiz management Service

The "Quiz management Service" is a core component of the overall system designed to manage and deliver quizzes to job applicants as part of a recruitment process. This service is primarily responsible for handling the creation, updating, and management of quizzes, which are then linked to specific job postings. It acts as the bridge between job applications and the assessment mechanism, enabling companies to evaluate the skills of applicants more effectively.

The service provides an interface for administering quizzes, including the setting of questions, assigning scores, and defining the total score for the quiz. Additionally, it ensures that the applicants can only access quizzes related to the job positions they apply for, streamlining the process of job application evaluation.

5.6.4.1. Role of the Quiz management Service in the System

The role of the Quiz management Service is crucial for ensuring that job applicants are assessed based on their skills and knowledge relevant to the position they are applying for. Its main functions include:

- Providing quizzes: It ensures that applicants are presented with the correct quiz based on the job posting they are applying for.
- Tracking performance: The Quiz management Service helps track the progress and scores of applicants, making it easy to evaluate their performance.
- Administering assessments: The service allows administrators to create and update quizzes, including defining the questions, answers, and time limits.

- Integrating with other services: It connects with other microservices such as the "Job Posting Service" and "User Management Service" and "Job Application Service " to ensure that the right quiz is given to the right candidate at the right time.

5.6.4.2. Responsibilities

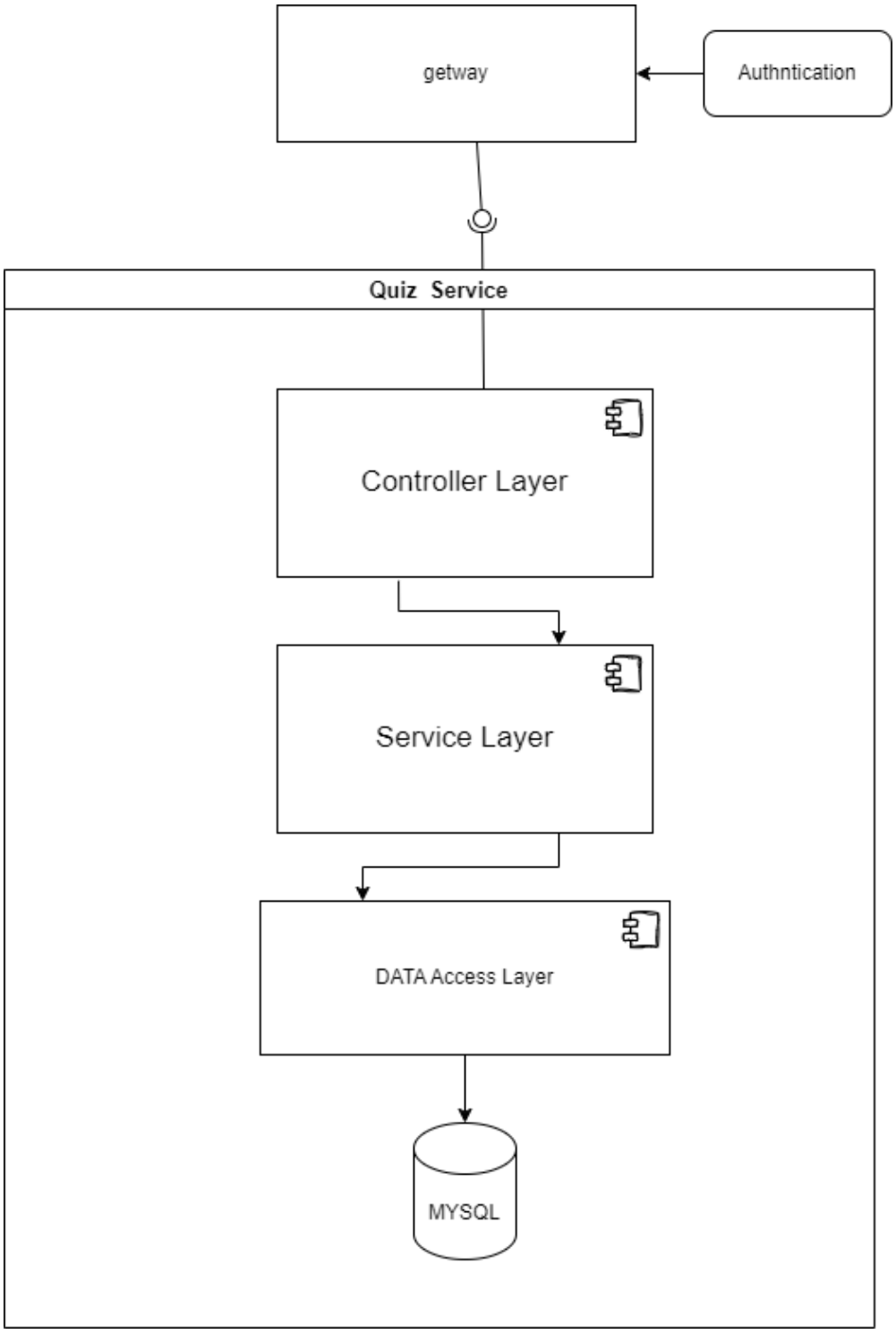
The Quiz management Service has a set of well-defined responsibilities within the system, which can be outlined as follows:

1. Quiz Creation and Management:
    - The service allows for the creation of quizzes related to job posts.
    - It ensures that quizzes can be updated, modified, or deleted by the appropriate user role (typically an admin).
2. Handling Questions:
    - It is responsible for adding, removing, and updating questions within a quiz.
    - The service manages the association of questions with the quiz and ensures they are linked correctly.
3. Time Management:
    - The Quiz management Service allows for the setting of time limits on quizzes, ensuring that applicants complete the quiz within a predefined period.
4. Score Calculation:
    - The service calculates and manages the total score for a quiz, ensuring that each applicant's performance is scored based on their answers.

- It also supports the calculation of percentages or other relevant metrics based on the applicant's answers.

5. Quiz Access Control:
    - It ensures that quizzes are only accessible to applicants who have applied for specific job postings.
    - The service verifies the eligibility of the user to access a particular quiz and prevents unauthorized access.

5.6.4.3. Component Diagram



5.6.4.4. Components

1. Controller Layer:

The Controller Layer handles HTTP requests and serves as the interface between the user and the system. When requests are received from the Gateway, this layer delegates them to the appropriate services in the Service Layer. Its primary tasks include creating quizzes, retrieving quiz details, and submitting quiz answers.

2. Service Layer:

The Service Layer contains the business logic for managing quizzes. It processes requests from the Controller Layer and performs operations such as quiz creation, updating, and deletion. Additionally, it interacts with the Data Access Layer to retrieve or persist quiz data securely.

3. Data Access Layer:

The Data Access Layer manages the persistence and retrieval of quiz-related data. It includes Models that define the structure of quiz data, such as questions, answers, and results. It also provides Repositories for performing CRUD (Create, Read, Update, Delete) operations on the database. This layer uses MySQL as the primary database to store quiz data.

4. MySQL:

MySQL serves as the primary database for storing quiz-related data. It stores data such as quiz questions, user answers, and quiz results. MySQL provides reliable and efficient data management for the application, ensuring optimal performance for data storage and retrieval.

A more detailed explanation of each layer will be provided in the Class Diagram section.

5.6.4.5. Class Diagram and Design Patterns

The Class Diagram provides a detailed structure of relationships and entities within the system. It is divided into **two main sections:**

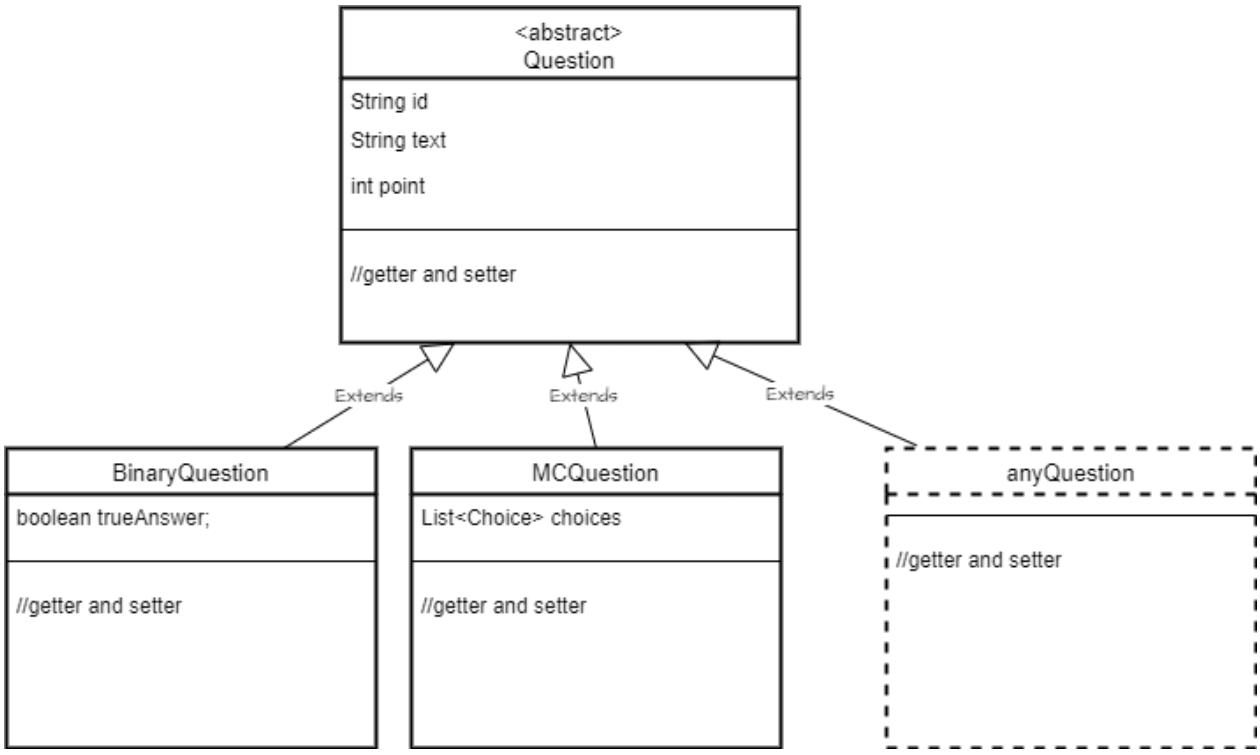
1. **Quiz Management Section**: This section deals with classes related to managing quizzes, including the Quiz class that handles properties such as title, description, company ID, and total score. It also includes relationships with other classes like Question to manage individual questions and Category for categorization.

   - **Abstract Question Class and Its Subtypes**

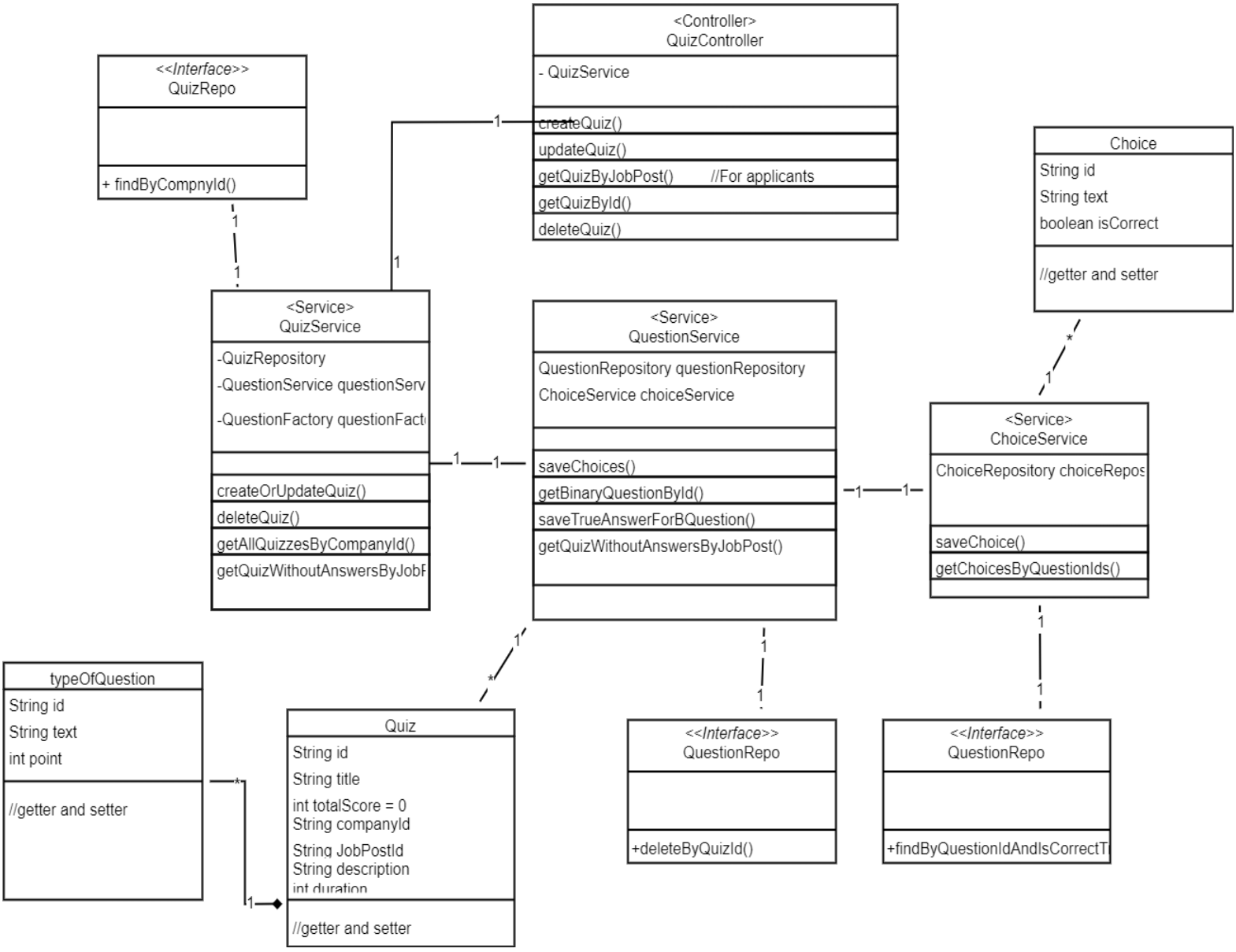
To enhance flexibility and maintainability, the Question class is designed as an abstract class. This allows for the creation of different types of questions while enforcing a common structure. Various question types inherit from Question, such as:

- MultipleChoiceQuestion: Supports multiple predefined answer options with a single correct choice.
- BinaryQuestion: Represents questions with only two possible answers (true or false).
- ShortAnswerQuestion: Allows users to input free-text responses that may require manual or AI-based evaluation.

By using an abstract class, the system ensures consistency across different question types while enabling future extensions without modifying existing code. Each subtype can implement its own validation and scoring mechanisms, making the system more scalable and adaptable.



5.6.4.6. Class Diagram



○ **Using the Factory Pattern in the Project**

In this project, the Factory Pattern is applied as a primary tool for creating objects of different types of questions in a flexible and organized manner. The factory pattern is one of the structural design patterns aimed at simplifying the object creation process, so that users don't need to know the details of how to create the objects but only need to specify the type of object they want to create.

The core concept of the factory pattern in this project is the use of the QuestionFactory class, which is responsible for creating objects of different types of questions based on the type of question provided. For example, when the input question type is "MULTIPLE_CHOICE", the factory creates an object of the MCQuestion class, which represents a multiple-choice question. If the question type is "TRUE_FALSE", the factory creates an object of the BinaryQuestion class, which represents a true/false question. If an unsupported question type is provided, an exception is thrown to indicate the unknown type.

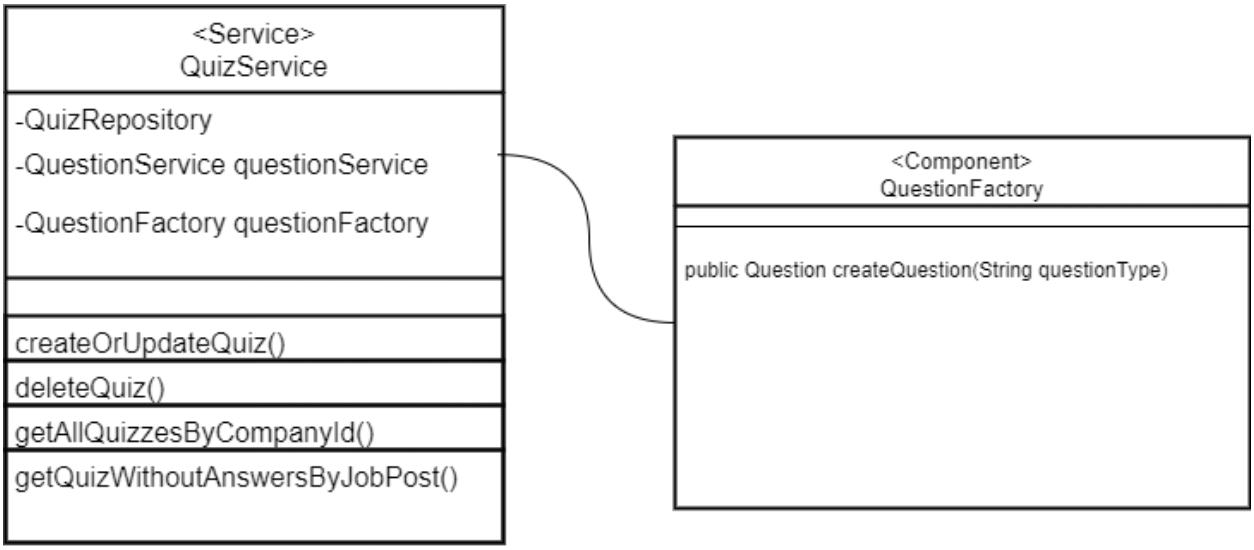
The main benefit of using the factory pattern in the project is separating the question creation process from the rest of the system. Instead of writing complex logic to create different types of questions in various places in the code, a unified factory is used to provide a standardized way to create questions, making the code clearer and easier to maintain. Additionally, the factory pattern allows for the easy addition of new question types, as new classes can be added and the factory modified to handle new types without impacting other parts of the system.

Benefits of applying the factory pattern in this project:

- **Flexibility and Expansion**: New types of questions can be added easily by creating new question classes and modifying the factory to handle them without changing other parts of the code.

- **Abstraction**: The factory pattern provides developers with a unified interface to create various types of questions without needing to know the detailed implementation of how they are created.
- **Better Code Management**: The factory helps improve code organization by centralizing the question creation logic in one place, making the system easier to maintain and develop further.

Thus, the factory pattern is a powerful tool for simplifying question management in the project, enhancing organization and flexibility, and making the system easily extensible in the future.



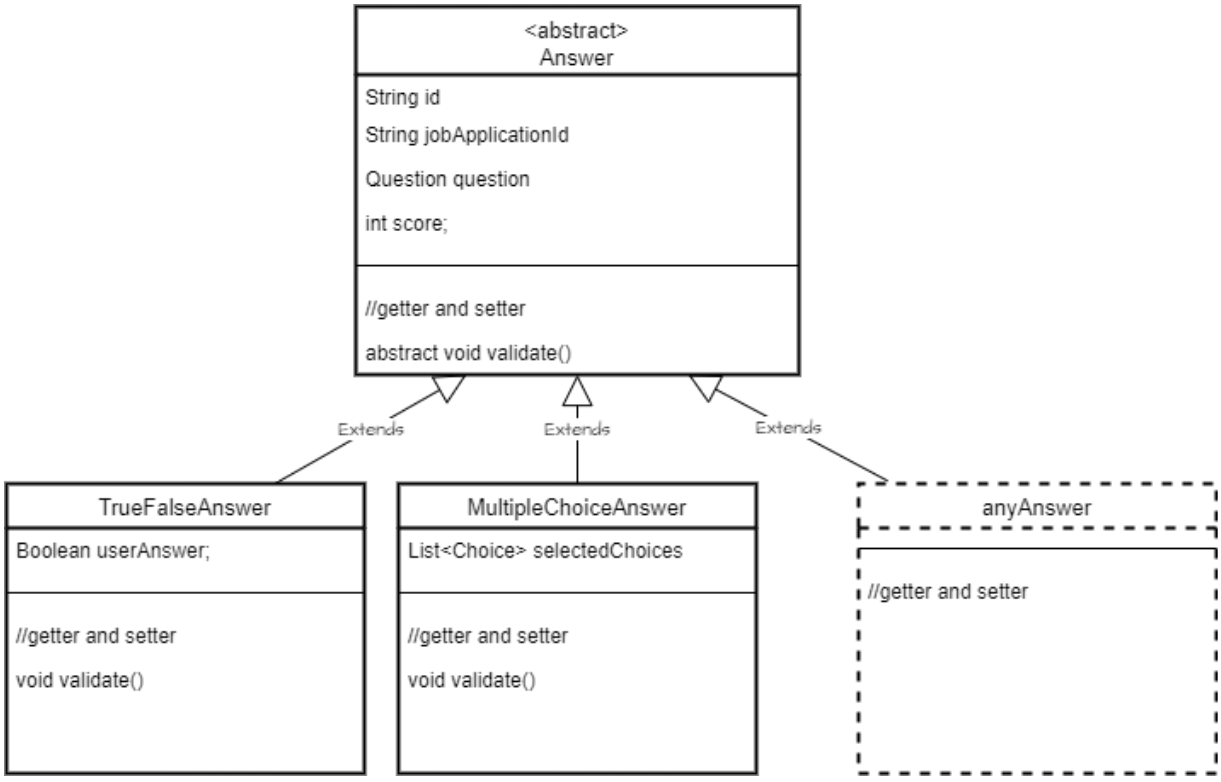
2. **Exam and Answer Management Section:** This section handles classes like Answer that store user responses, and JobApplication that links answers to specific job applications. Classes such as AnswerService and AnswerRepository handle the processing and storage of answers.

- **Abstract Answer Class and Its Subtypes**

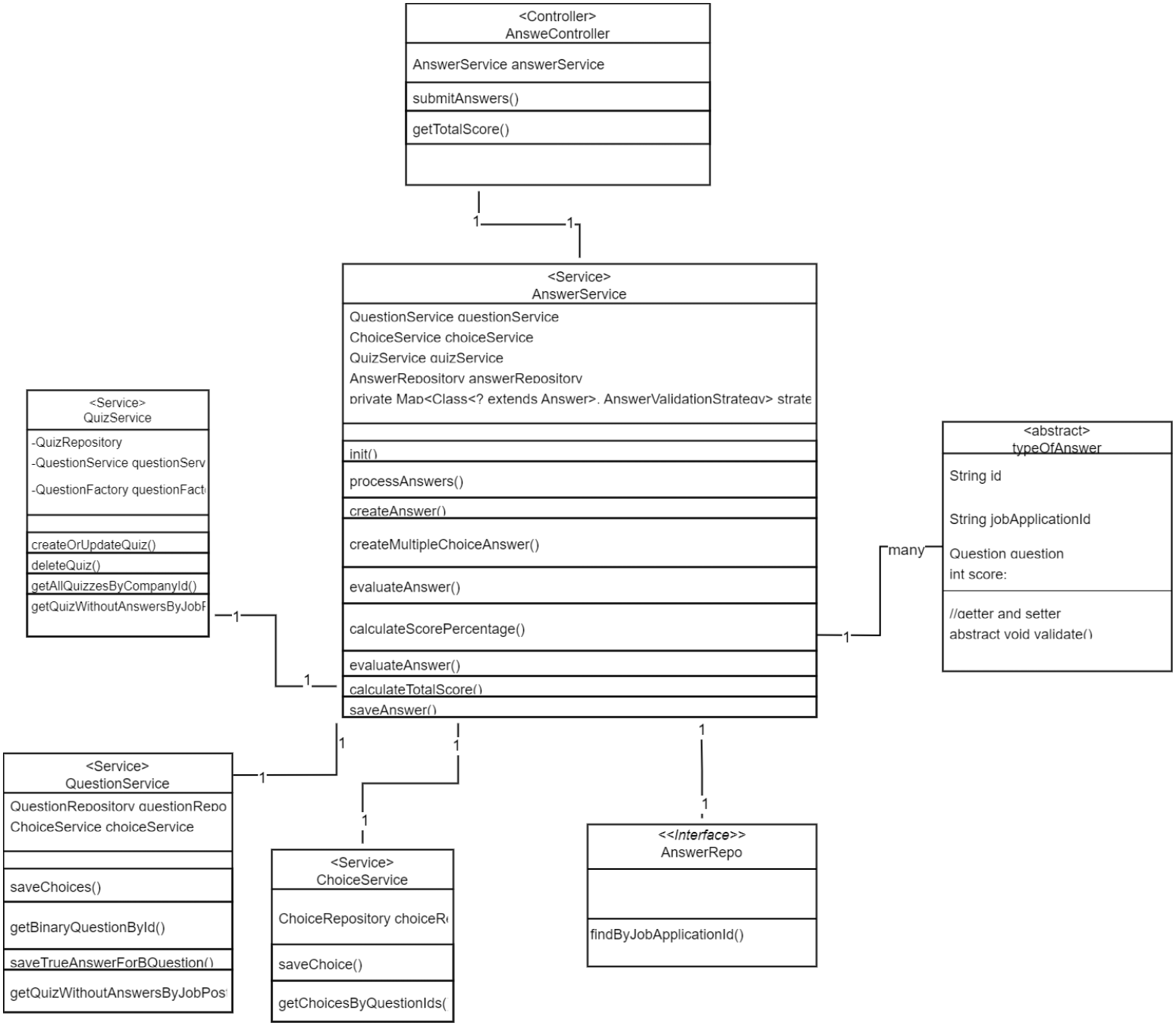
To ensure flexibility and maintain a consistent structure, the Answer class is defined as an abstract class. This approach allows for different types of answers while enforcing a unified structure across all responses. Various answer types inherit from Answer, including:

- **MultipleChoiceAnswer**: Stores the selected option for multiple-choice questions and validates it against the correct choice.
- **TrueFalseAnswer**: Captures responses for true/false questions and evaluates correctness.
- **ShortAnswerResponse**: Holds free-text answers provided by users, which may require manual review or AI-based assessment.

By using an abstract class for answers, the system remains extensible, allowing new answer types to be introduced without modifying existing logic.



○ Class Diagram



- **2.3 Strategy Pattern in the Project**

In this project, the Strategy Pattern is applied to organize the process of validating answers based on the question type. This pattern helps separate the validation logic for each question type, allowing the appropriate strategy for answer validation to be chosen at runtime. It provides great flexibility and makes the project easier to maintain and extend.

Concept of Strategy Pattern:

The Strategy Pattern is a behavioral design pattern that allows selecting an algorithm or behavior (in this context, the answer validation mechanism) at runtime. Instead of using multiple conditional statements or complex code with various validation methods, the Strategy Pattern enables defining a family of algorithms (answer validation mechanisms) and making them interchangeable.

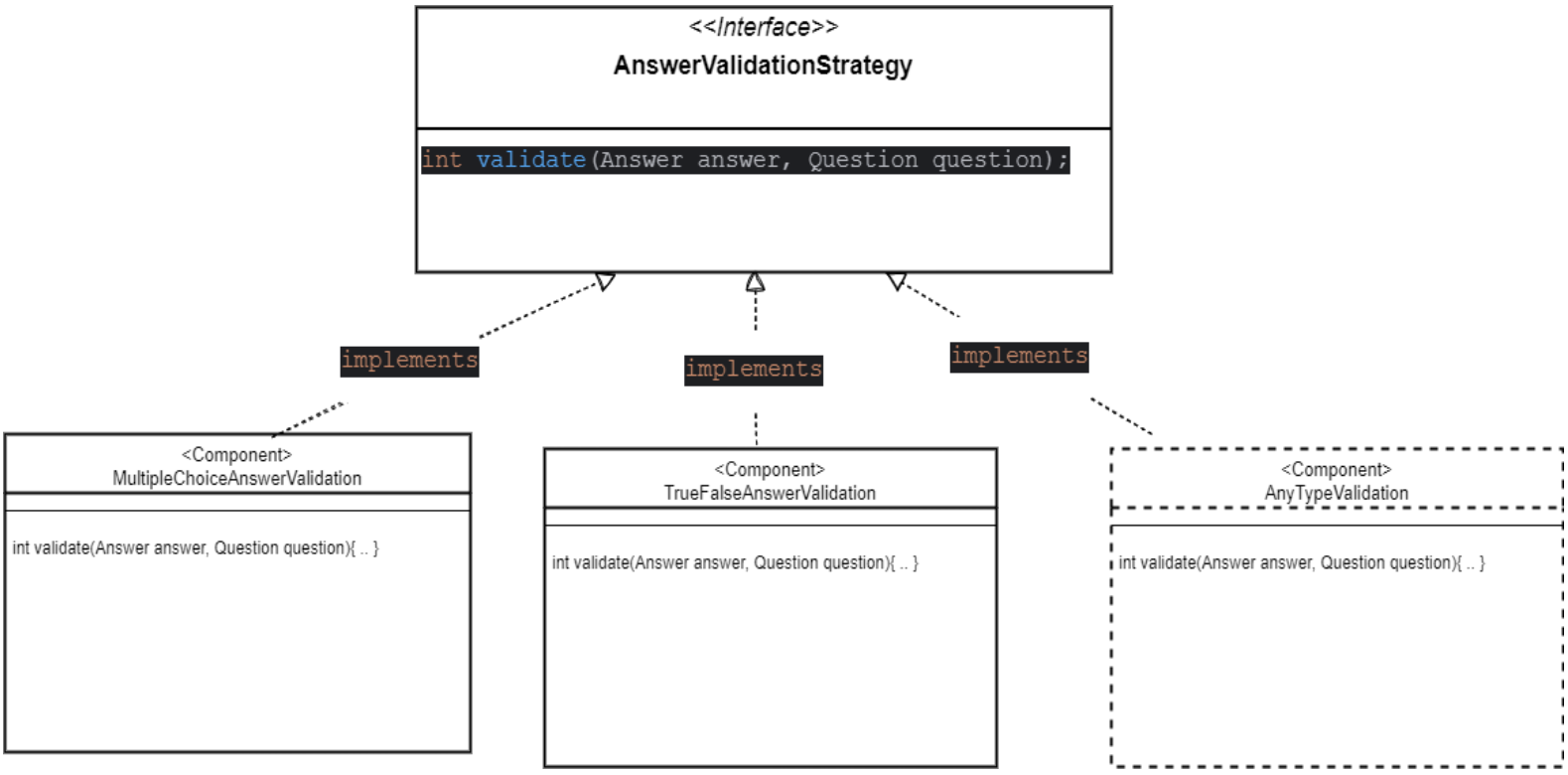
Applying the Strategy Pattern in the Project:

- AnswerValidationStrategy Interface:
  This interface defines a contract for the answer validation mechanism. All strategies for validating answers for different types of questions implement this interface. The contract includes a validate method that takes an answer and a question and returns the score based on the validation result.
- MultipleChoiceAnswerValidation Strategy:
  This strategy is responsible for validating answers for multiple-choice questions. The answer is validated by comparing the options selected by the user with the correct choices in the question. This strategy calculates the score based on the number of correct answers selected by the user.
- TrueFalseAnswerValidation Strategy:
  This strategy is specialized in validating answers for true/false questions. It

compares the user's input answer with the correct answer for the binary question. If the answer is correct, it awards the specified points.

Benefits of the Strategy Pattern in this Project:

- Separation of Concerns:
  The validation logic for each question type is separated into its own class, making the code more organized and easier to understand. Each class is responsible for executing a specific validation logic.
- **Flexibility**:
  New validation strategies for other types of questions can be added without affecting the existing code. The AnswerValidationStrategy interface remains unchanged, and new validation strategies can be introduced easily.
- **Scalability**:
  As new question types are added to the system, new validation strategies can be created without needing to modify existing code. This helps to scale the system smoothly and efficiently.
- **Reusability**:
  Validation strategies can be reused throughout different parts of the application, ensuring consistency in the validation logic across various question types.



2.4 Service Operation Mechanism

1. Initializing Validation Strategies

   When the AnswerService is initialized, the @PostConstruct annotation is used to set up a strategyMap that links each answer type (such as True/False or Multiple Choice) with its corresponding validation strategy. The appropriate strategy is determined based on the type of answer submitted by the user.

2. Processing Answers

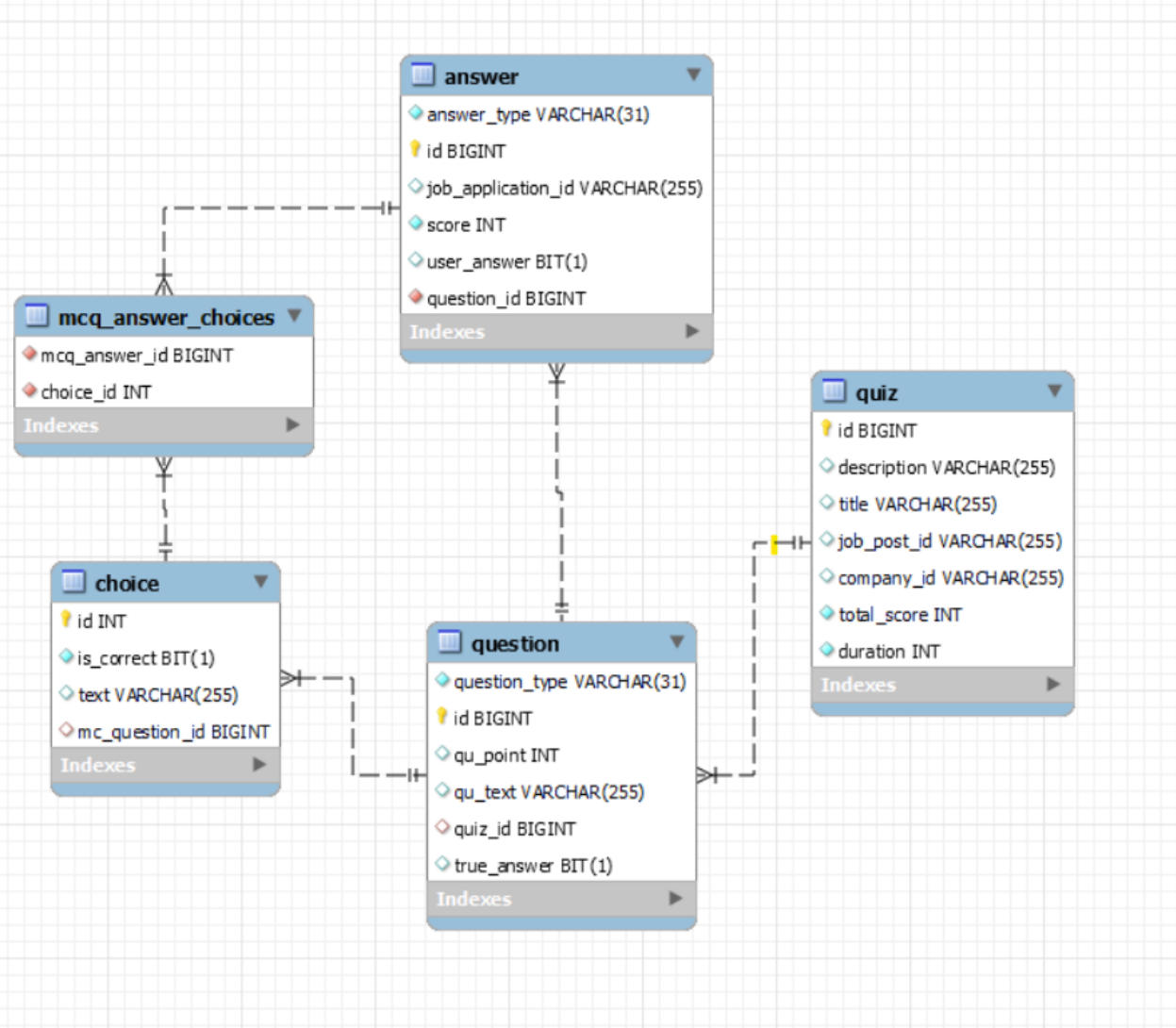
   The AnswerRequestDTO containing a collection of answers submitted by users is received. For each answer, the corresponding question is fetched using questionService.getOpQuestionById. Based on the answer type, the appropriate answer is created:

- If the answer type is TRUE_FALSE, a TrueFalseAnswer is created.
- If the answer type is MULTIPLE_CHOICE, a MultipleChoiceAnswer is created by selecting the correct choices through choiceService.

3. Validating the Answer and Calculating Points
   After creating the appropriate answer, the strategy specified for that answer type is used to validate the answer. The validation depends on the type of question:
   - For True/False questions, the submitted answer is compared with the correct answer defined in the question.
   - For Multiple Choice questions, the submitted answer is compared with the correct choices for the question, and points are calculated based on the number of correct answers.
4. Saving the Answer
   After validating the answer and calculating the score, the answer is saved in the database using answerRepository.save(answer).
5. Calculating the Overall Score
   After processing all answers, the overall score for the user in the quiz is calculated. This is done by summing the points for each answer and comparing them with the total points available for the quiz. The result is calculated as a percentage of the total points.
6. Final Score Calculation
   In the calculateScorePercentage method, the points for all answers submitted by the user in the quiz are summed, and the percentage score is calculated based on the total available points for the quiz.

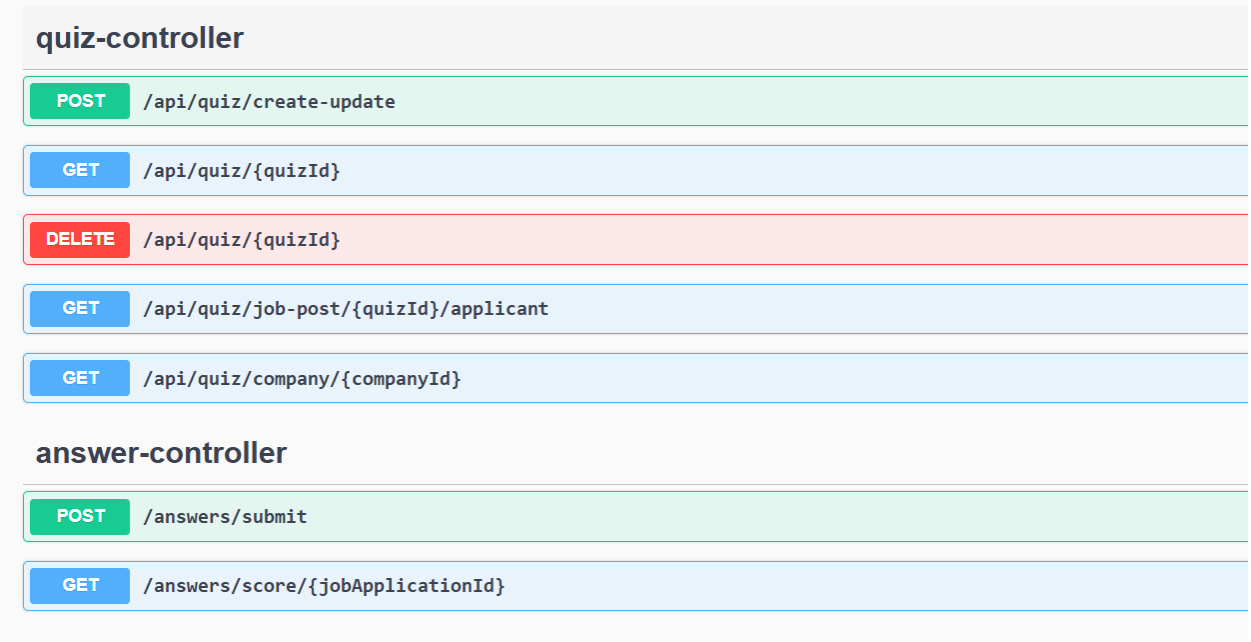
5.6.4.7. Database:

The database is essential in the quiz application, where all data related to quizzes, questions, and answers are stored. The system uses MySQL to ensure good performance and flexibility in data management.



5.6.4.8. API Endpoints

The Quiz Application provides a set of RESTful APIs to manage quizzes, questions, and answers, ensuring smooth interaction between users, questions, and the system. Below is an overview of the key endpoints available in the application.



5.6.5. Job Application Service

The **Job Application Service** is a core component of the **Smart HR Recruitment System**, responsible for **handling job applications, tracking their progress, and managing hiring decisions**. It allows **job seekers to submit applications and employers to review and manage applicants** efficiently.

5.6.5.1. Role of Job Application Service in the System

The **Job Application Service** is responsible for managing the **entire lifecycle of job applications**. It provides an interface for job seekers to **apply for jobs** and for employers to **review, shortlist, and update the status of applications**.

This service is closely integrated with:

User Management Service: Ensures that **only authenticated users** can apply for jobs.

Job Posting Service: Retrieves **job details** before processing applications.

AI Processing Service: Uses **AI-based job matching** to suggest suitable jobs for applicants.

Notification Service: Sends alerts about **application status updates**.

By enabling structured **job applications and employer reviews**, this service improves **hiring efficiency and transparency** in the recruitment process.

5.6.5.2. Responsibilities

Job Application Submission: Allows job seekers to **apply for job listings** using their profiles.

Application Tracking: Enables job seekers to **track the status** of their applications (e.g., Pending, Reviewed, Accepted, Rejected).

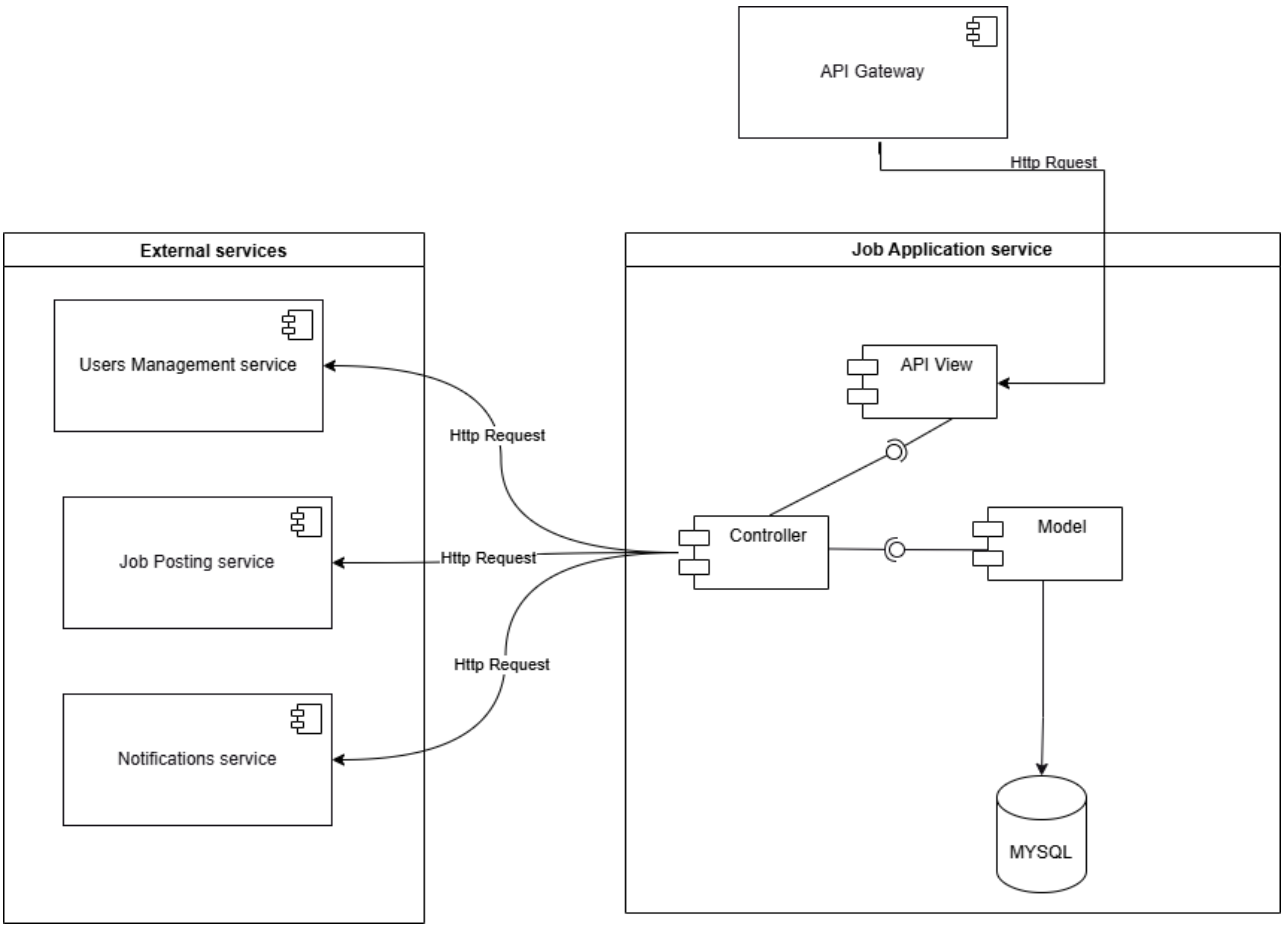
Employer Application Review: Allows employers to **view, filter, and shortlist applicants** for their job postings.

Status Updates: Employers can **update application statuses**, notifying candidates about progress.

Applications by Location: Enables companies to **filter applicants based on geographical preferences**.

By ensuring a **structured and transparent application process**, the **Job Application Service** enhances the overall recruitment experience for both job seekers and employers.

5.6.5.3. Component diagram:



5.6.5.4. Components

- **API Gateway**:
  - Handles external client requests (e.g., from the frontend).
  - Routes requests related to job applications (e.g., POST /apply, GET /applications) to the Job Application Service.
- **API View**:
  - Receives HTTP requests from the API Gateway.
  - Validates input data (e.g., ensures job_id and user_id are provided).
  - Forwards valid requests to the **Controller** for business logic execution.
- **Controller**:

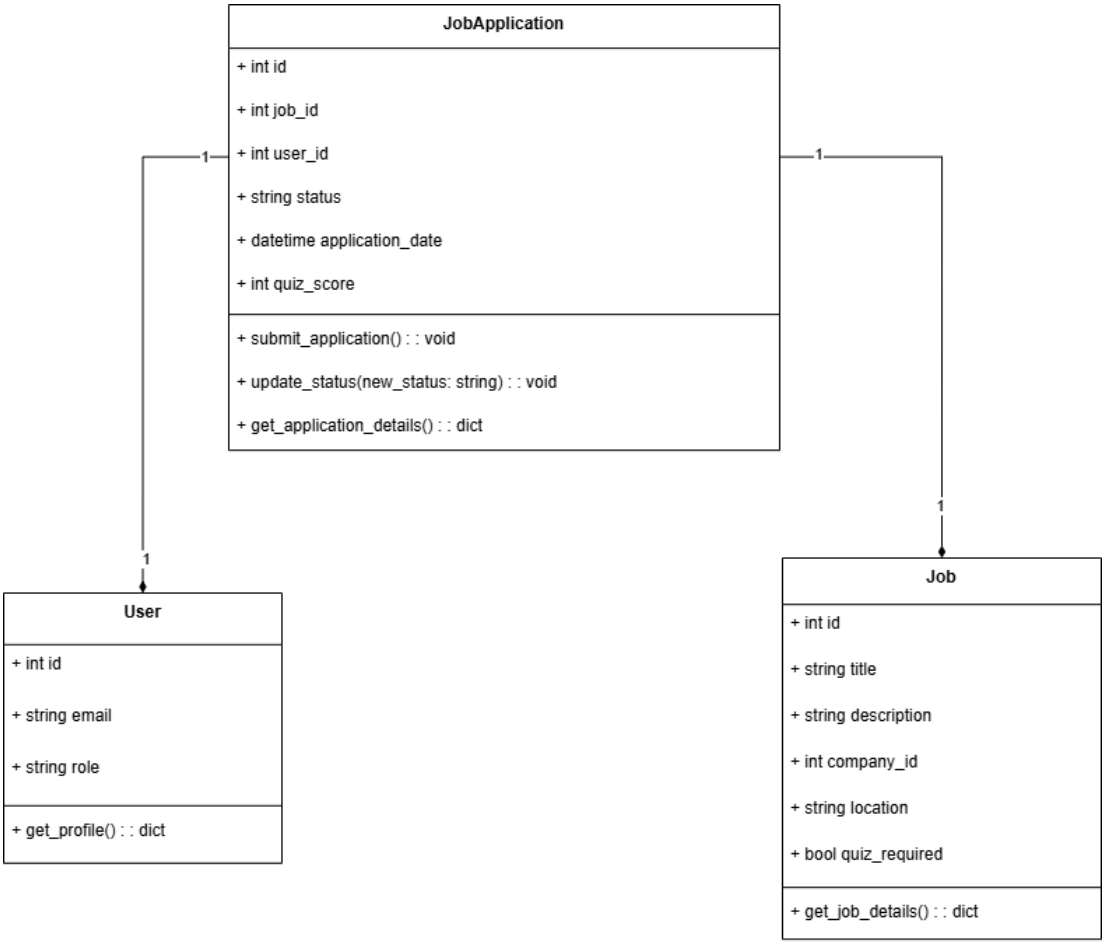
    - Implements the core business logic of the service, such as:
        - Submitting a job application.
        - Fetching application details for a user or job.
        - Updating the application status.
    - Interacts with external services like the **User Management Service**, **Job Posting Service**, and **Notification Service**.
- **Model**:
    - Represents the database schema for the Job Application entity.
    - Contains methods for creating, updating, and retrieving job application records.
- **Job Application Database**:
    - A MySQL database that stores job application records, including:
        - id (Primary Key).
        - job_id (Foreign Key referencing the Job Posting Service).
        - user_id (Foreign Key referencing the User Management Service).
        - status (e.g., submitted, reviewed, accepted, rejected).
        - application_date.
        - quiz_score
- **External Services**:
    - **User Management Service**:
        - Provides user information for applications (e.g., user_id validation).
    - **Job Posting Service**:
        - Provides job details (e.g., job_id validation, job title, and description).
    - **Notification Service**:

        - Sends notifications when an application is submitted, updated, or reviewed.

5.6.5.5. Data Flow

- **Submit Application**:
    - A job seeker submits an application via the frontend.
    - The request is routed through the API Gateway to the **API View**.
    - The **API View** validates the input and forwards it to the **Controller**.
    - The **Controller**:
        - Fetches user details from the **User Management Service**.
        - Validates job details with the **Job Posting Service**.
        - Saves the application to the database through the **Model**.
        - Triggers the **Notification Service** to send a confirmation message.
- **Update Application Status**:
    - An employer reviews an application and updates its status (e.g., from submitted to reviewed).
    - The **Controller** updates the status in the database and triggers a notification.
- **Fetch Applications**:
    - A job seeker or employer retrieves applications.
    - The **Controller** fetches the data from the database and, if needed, enriches it with information from the **User Management Service** or **Job Posting Service**.

5.6.5.6. Class Diagram:



5.6.5.7. API Endpoints

Method	Endpoint	Description
POST	/apply/	Apply for a job
GET	/application/<int:application_id>/	Retrieve job application status
.0PUT	/application/<int:application_id>/update/	Update job application status
GET	/job-seeker/<int:job_seeker_id>/applications/	Retrieve all applications by a job seeker
GET	/job/<int:job_id>/applications/	Retrieve all applications for a job posting
GET	/applications/by-location/	Retrieve applications by location

5.6.6. Notification Service

5.6.6.1. Responsibilities:

- Sends notifications for various events.

5.6.6.2. Components:

5.6.6.3. Class Diagram:

5.6.7. Location Service

The **Location Service** is an essential part of the **Smart HR Recruitment System**, managing geographical data such as countries and cities. It enables job seekers to specify location preferences and employers to filter candidates based on location.

5.6.7.1. Role of Location Service in the System

The **Location Service** is responsible for storing and managing geographical data that supports location-based job searches and employer-candidate matching.

This service interacts with:

- **Job Posting Service**: Ensures jobs are associated with specific locations for better filtering.
- **Job Application Service**: Helps companies search for candidates based on geographical preferences.
- **User Management Service**: Stores location data for job seekers and employers.

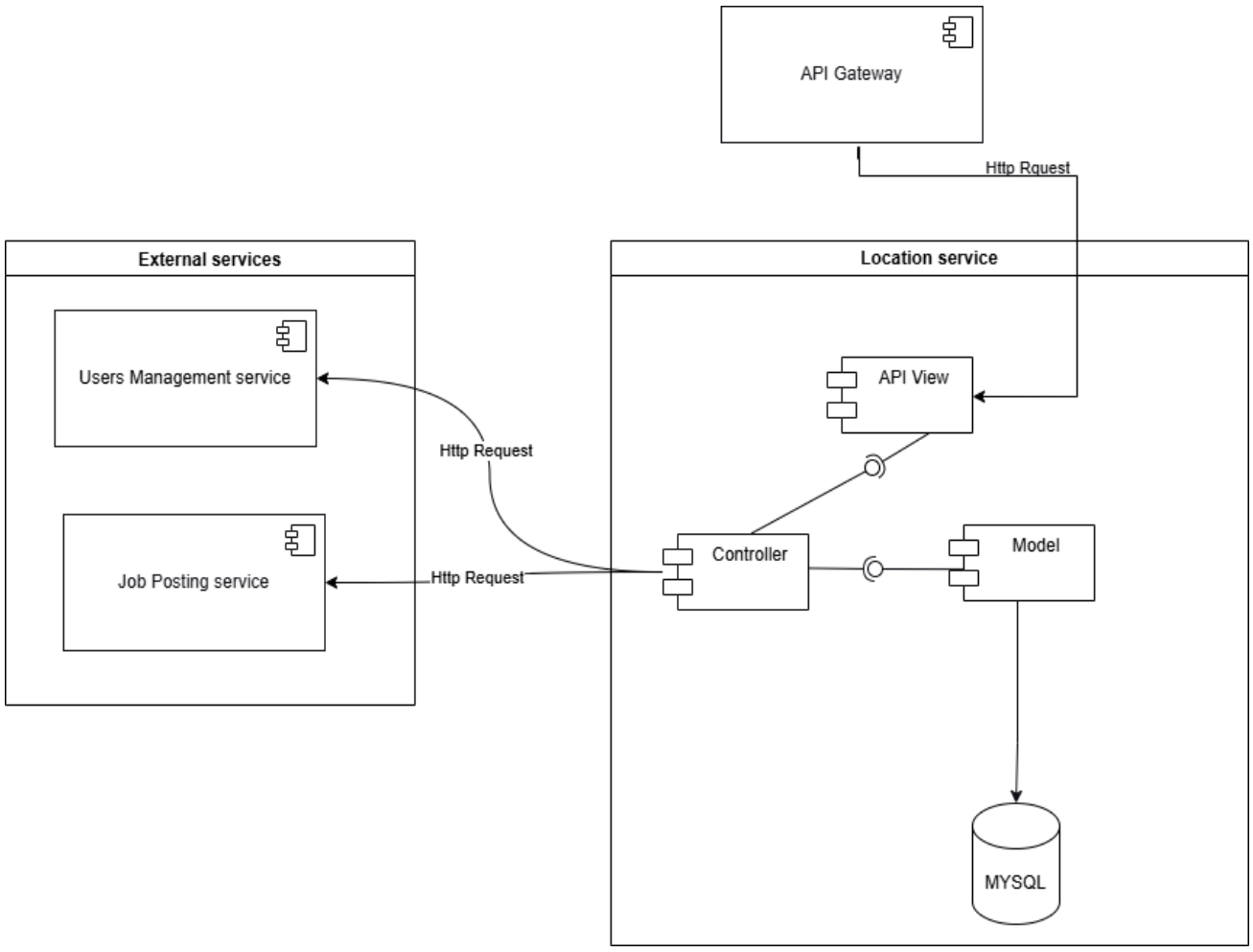
By ensuring accurate **location-based job filtering**, the **Location Service** enhances the system's ability to connect employers with nearby talent.

5.6.7.2. Responsibilities

1. **Storing Country & City Data**: Maintains an up-to-date database of countries and cities.
2. **Location-Based Job Searching**: Enables job seekers to filter jobs by preferred location.
3. **Employer Candidate Filtering**: Allows employers to search for applicants based on location proximity.
4. **Seamless Integration**: Ensures that all job listings and user profiles include accurate location data.

By providing structured and accurate location-based data, the Location Service plays a vital role in improving job matching efficiency and regional hiring processes.

5.6.7.3. Components diagram



5.6.7.4. Components

1. API Gateway

- Handles external client requests (e.g., from the frontend and other microservices).
- Routes location-related requests (e.g., GET /locations/countries, GET /locations/cities) to the **Location Service**.

2. API View

- Receives HTTP requests from the **API Gateway**.
- Validates input data (e.g., ensures a country name or city name is provided).
- Forwards valid requests to the **Controller** for business logic execution.

3. Controller

- Implements the core business logic of the **Location Service**, such as:
  - Retrieving all available countries.
  - Fetching cities associated with a given country.
  - Adding new country or city records.
  - Enforcing data validation rules (e.g., country codes must be unique).
- Interacts with the **Model** to store and retrieve data.

4. Model

- Represents the **database schema** for Country and City entities.
- Defines relationships between **Country** and **City** (One-to-Many).
- Contains methods for:
  - Creating and updating country/city records.
  - Fetching all available locations.

- Deleting cities if a country is removed.

5. External Services

- **Job Posting Service**:
  - Fetches location data when a company posts a job.
  - Ensures job postings include valid locations from the **Location Service**.
- **User Management Service**:
  - Associates users with their **country and city** for location-based job recommendations.

5.6.7.5. Data Flow

1. Fetching Available Locations

- A **client** (frontend or another service) requests available locations via GET /locations/countries.
- The request is routed through the **API Gateway** to the **API View**.
- The **Controller** fetches data from the **Model**, which queries the **Location Database**.
- The **Controller** returns the list of available countries to the client.

2. Adding a New City

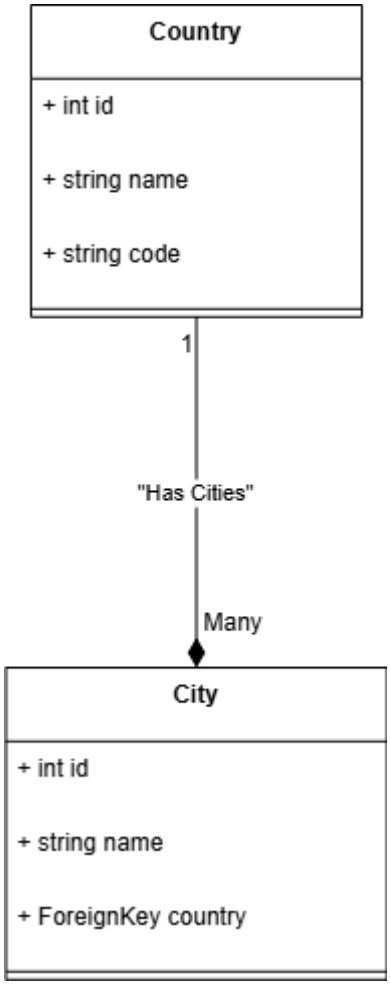
- A request (POST /locations/cities) is sent with a city name and its associated country.
- The **API View** validates the request and forwards it to the **Controller**.
- The **Controller** ensures:

    - The **country exists** before adding a city.
    - The **city is unique within the country**.
- The **city is stored** in the **Location Database**.

3. Deleting a Country

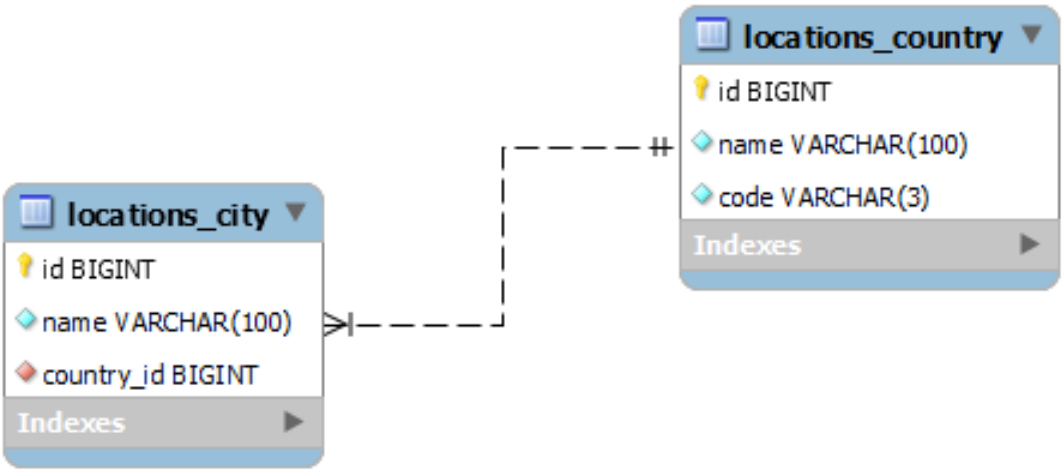
- If a **country is deleted**, all **related cities are also removed** (on_delete=models.CASCADE).
- Ensures **data integrity** by preventing orphaned city records.

5.6.7.6. Class Diagram:



5.6.7.7. Database

- A **MySQL** database that stores location data, including:
  - **Country Table**:
    - id (Primary Key).
    - name (Unique, indexed).
    - code (ISO 3166-1 Alpha-3 country code).
  - **City Table**:
    - id (Primary Key).
    - name (City name).
    - country_id (Foreign Key referencing Country).
    - Unique constraint: **Cities must be unique per country**.



5.7. AI Processing Service

5.7.1. Resume Parsing:

5.7.1.1. Architecture

The CV parsing module is organized into several layers, each addressing a specific stage of the pipeline:

1. **Data Collection & Annotation Layer:**

- **Input:** Resumes in multiple formats (PDF, DOCX, plain text).
- **Annotation:** Human experts annotate these resumes using tools like Prodigy or spaCy's annotation tool. Annotations are saved with key entity labels such as NAME, EMAIL, SKILLS, EDUCATION, etc.
- **Output:** An annotated dataset that serves as the ground truth for training.
- **Dataset:** After searching, we relied on a database containing more than 1000 CVs that were annotated, and permission was obtained from the owner of this data.
- **Link:** https://drive.google.com/drive/folders/1S29kzv3qKsAgfTYDcLRwq2_F1H1lXwBt?usp=sharing

II. **Preprocessing & Data Preparation Layer:**

- **Cleaning:** Remove extraneous characters, line breaks, and formatting artifacts.
- **Tokenization:** Use spaCy's tokenizer to segment the text into tokens while preserving the annotation offsets.
- **Normalization:** Standardize text (e.g., converting to lowercase, handling Unicode characters) so that the model receives consistent input.
- **DocBin Creation:** Convert annotated documents into spaCy's DocBin format, which is optimized for model training and efficient disk storage.

III. **Model Training Layer:**

- **Configuration:**

  Generate a configuration file (using spacy init fill-config) tailored to the project's requirements. This file includes the model architecture, training hyperparameters, and paths to the training and development datasets.

- **Training:**

  Train the NER model using the prepared DocBin files with a command

  - **Metrics:** During training, monitor losses, precision, recall, and F1-scores to ensure the model learns to extract entities correctly.
  - **Error Logging:** Log errors (e.g., when entity spans cannot be created) to refine the annotation process or adjust model parameters.

IV. **Integration & Deployment Layer:**

- **Model Loading:** Load the trained spaCy model in a FastAPI application.
- **API Endpoints:**

  Provide endpoints for:

  - **Plain Text Parsing:** Accepts raw text resumes and returns structured JSON.
  - **PDF Parsing:** Extracts text from PDFs using libraries like PyMuPDF (fitz) and processes the text with the loaded model.

- **Output Structure:**

  The API returns a JSON object with standard keys such as name, email, skills, education, etc., making it easy to integrate with downstream systems like applicant tracking systems (ATS) or matching engines.

5.7.1.2. Detailed Component Design

Preprocessing Component

Text Cleaning:

Use regular expressions and string manipulation to remove noise (e.g., headers, footers, page numbers).

Tokenization:

The spaCy tokenizer splits the text into tokens while maintaining the exact character offsets required for accurate annotation alignment.

Normalization:

Convert text to a consistent case (usually lowercase) and remove non-ASCII characters to avoid processing errors.

Entity Extraction Component

Model Architecture:

A custom NER model built on spaCy, possibly leveraging transformer architectures for better context understanding.

Training Data Handling:

Use DocBin to store training examples. This allows efficient loading and processing during model training.

Error Handling:

Implement checks to ensure that entity spans do not overlap and that missing spans are logged for later correction.

API Integration Component

Endpoint Design:

Two main endpoints are provided:

/extract-data: For plain text resumes.

/extract-data-from-pdf: For PDF resumes.

PDF Text Extraction:

Use PyMuPDF (fitz) to extract text from PDF files, ensuring the file pointer is reset and text is correctly decoded.

Output Formatting:

The extracted entities are mapped into a JSON schema, ensuring that even if certain entities are missing, the output structure remains consistent.

Data Flow

Input:

The system accepts a resume (either as plain text or as a PDF file).

Preprocessing:

The text is cleaned, tokenized, and normalized.

Entity Extraction:

The preprocessed text is passed through the NER model to identify and extract entities.

Output Structuring:

The extracted data is formatted into a consistent JSON object.

API Response:

The JSON object is returned to the client, ready for integration into the recruitment platform.

5.7.2. Job Recommendation

5.7.2.1. System Architecture

The job recommendation module is composed of three primary layers:

- **Embedding Generation Layer:**
  - **Input:** Candidate profiles and job descriptions as raw text.
  - **Process:** Use a pre-trained Sentence Transformer (e.g., all-MiniLM-L6-v2) to generate semantic embeddings.
  - **Output:** High-dimensional numerical vectors representing the text.
- **Similarity Calculation Layer:**
  - **Process:** Compute cosine similarity between candidate embeddings and job embeddings.
  - **Output:** A similarity score (typically scaled to a percentage) that quantifies the match between candidate and job.
- **API and Integration Layer:**
  - **API Endpoints:**
    - **Generate Embedding Endpoint:** Accepts text input and returns a serialized embedding.
    - **Calculate Similarity Endpoint:** Accepts two embeddings (candidate and job) and returns a similarity percentage.
  - **Output Format:** JSON responses that can be easily integrated with front-end interfaces or other systems.

5.7.2.2. Component Design

Embedding Generation Component

- **Model:**
  Use SentenceTransformer('all-MiniLM-L6-v2') for generating embeddings.
- **Serialization:**
  Use Python's pickle module (and base64 encoding) to serialize embeddings for storage in relational databases like MySQL.

Similarity Calculation Component

- **Method:**
  Use cosine similarity to measure the similarity between two embeddings.
- **Scaling:**
  The similarity score, which ranges from -1 to 1, is scaled to a percentage scale [0, 100] for interpretability.

API Integration Component

- **FastAPI Endpoints:**
  Expose endpoints for:
  - **Generating embeddings:** Accept raw text, generate an embedding, and return it in a serialized (base64-encoded) form.
  - **Calculating similarity:** Accept two serialized embeddings and compute their cosine similarity.

5.7.2.3. Data Flow

- **Input:**
  The client submits a candidate's resume or job description as text.
- **Embedding Generation:**
  The text is passed to the Sentence Transformer model to generate an embedding.

- **Embedding Storage/Transfer:**
  The embedding is serialized and, if needed, stored in a database or transferred to the similarity calculation endpoint.
- **Similarity Computation:**
  Two embeddings (from a candidate and a job) are compared using cosine similarity.
- **Output:**
  A JSON response returns the similarity percentage.

5.7.3. AI-Generated Quiz System.

5.7.3.1. System Architecture

The Test/Question Generation module is divided into three primary layers:

- **Input Layer:**
  Accepts parameters from the user via a REST API. Inputs include:
  - **Skill:** The topic or technical area.
  - **Difficulty:** E.g., "easy", "medium", "hard".
  - **Question Type:** E.g., "multiple_choice", "true_false".
  - **Number of Questions:** How many questions to generate.
  - **Model:** The local model to use (e.g., "deepseek-r1:1.5b").
- **Processing Layer:**
  - **Prompt Construction:**
    Constructs a detailed prompt for the model that includes strict formatting requirements and an explicit JSON example.
  - **Model Inference:**
    Calls the local model via the Ollama client to generate question text.

    - **Output Parsing:**
      Uses regular expressions and JSON parsing to extract a structured JSON object from the raw model output.
- **Output Layer:**
    - Returns a JSON response containing the generated questions.
    - Ensures that the output is consistent and contains the keys: "question", "options", and "correct_answer".
- **Component Design**

Prompt Construction

- The prompt explicitly instructs the model to output JSON in a fixed format, including an example.
- It emphasizes:
    - The number of options (4).
    - That the correct answer must be provided as an option number (as a string).

Output Parsing

- The parser removes extraneous sections (such as <think> blocks) and markdown formatting.
- It uses a regex pattern to extract the JSON block, then loads it into a Python dictionary.
- Fallback defaults are provided if the JSON cannot be parsed.

API Integration

- The system is exposed via FastAPI endpoints.

- One endpoint handles plain text input (for direct question generation), and additional endpoints can be created if needed.
- The final output is returned as a JSON object for easy integration with front-end systems.

Chapter 6 Practical Implementation

6.1. Introduction

This chapter details the practical implementation of the Smart HR Recruitment System, covering the technologies and tools used, system interfaces, and AI-based functionalities. The project follows a microservices architecture, allowing modular development, scalability, and independent deployment of services. Each service is responsible for a specific function, ensuring maintainability and efficiency.

The implementation is divided into the following sections:

- Backend Development (Django, FastAPI, Spring Boot)
- Frontend Development (React.js with TypeScript)
- Database Management (MySQL, MongoDB, Redis)
- Real-Time Features (WebSockets, Redis Pub/Sub)
- Deployment & Version Control (Docker, GitHub)
- AI-Powered Features (Resume Parsing, Job Matching, Quiz Generation)

6.2. Used Tools

6.2.1. Backend Technologies

Django & Django REST Framework (DRF)

Django is a high-level Python web framework that encourages rapid development with built-in security, authentication, and database management features. Django REST Framework (DRF) extends Django, providing tools for building RESTful APIs.

Usage in Project:

- User Management Service (Handles user authentication, profile management)
- Job Application Service (Handles applications and status tracking)
- Location Service (Manages job seeker locations for better search results)

Key Features:

- Model-View-Controller (MVC) architecture
- Built-in authentication
- Fast database querying with ORM (Object-Relational Mapping)

FastAPI

FastAPI is a high-performance web framework that provides asynchronous capabilities, making it ideal for AI-based tasks.

Usage in Project:

- Acts as the backbone of the application by exposing REST endpoints for all services (CV Parsing, Job Recommendation, and AI-Generated Quiz System).

Key Features:

- Asynchronous processing for high-speed AI tasks
- Auto-generated OpenAPI documentation
- Provides asynchronous request handling for fast and scalable API performance.
- Simplifies API development with automatic data validation and interactive documentation.
- Built-in request validation with Pydantic

Spring Boot

Spring Boot is a Java-based framework that simplifies microservice development.

Usage in Project:

- Job Posting Service (Handles job listings and employer accounts)
- Communication Service (Handles chat functionality and real-time messaging)
- Quiz Management Service (Manages job-related quizzes and candidate assessments)

Key Features:

- Spring Security for authentication & authorization
- Spring Data JPA for database interaction
- REST API support for seamless interaction with frontend

6.2.2. Frontend Technologies

React.js with TypeScript

React.js is a JavaScript library used for creating interactive user interfaces, while TypeScript enhances React by introducing static typing, which improves code quality and maintainability.

Usage in Project:

- Job Seeker & Employer Dashboards
- Job Search & Application Portal
- Real-time Chat Interface

Key Features:

- Component-based architecture for reusable UI elements
- Strong typing with TypeScript to reduce runtime errors
- State management with React Context API

6.2.3. Database Management

MySQL

MySQL is an open-source relational database management system (RDBMS) known for its reliability and scalability.

Usage in Project:

- User Management Service ⟶ Stores user details, roles, and profiles
- Job Posting Service ⟶ Stores job listings and company details
- Job Application Service ⟶ Tracks applications and statuses
- Quiz Management Service ⟶ Stores quiz questions and results
- Location Service ⟶ Manages job locations

Key Features:

- ACID compliance for data consistency
- Foreign key relationships for structured data storage

MongoDB

MongoDB is a NoSQL document-based database that allows flexible data storage.

Usage in Project:

- AI Processing Service ⟶ Stores AI-generated job matches and parsed resumes
- Communication Service ⟶ Stores chat history and messages

Key Features:

- JSON-like document structure
- High scalability for handling AI-related data

Redis

Redis is an in-memory key-value store used for caching and real-time data storage.

Usage in Project:

- User Management Service ⟶ Caches frequently accessed user profiles
- Communication Service ⟶ Stores WebSocket connections for real-time chat

Key Features:

- Sub-millisecond latency for fast data retrieval
- Supports Pub/Sub messaging for real-time chat

6.2.4. AI Used Technologies and Tools

SpaCy

SpaCy is a Natural Language Processing (NLP) library optimized for fast text processing.

Usage in Project:

- Resume Parsing: Extracts skills, education, and experience from uploaded resumes.
- build, train, and deploy a custom Named Entity Recognition (NER) model that extracts key candidate information

Key Features:

- Named Entity Recognition (NER)
- Dependency parsing for structured text processing

PyMuPDF (fitz)

A Python binding for MuPDF, a lightweight PDF and XPS viewer.

Usage in Project:

- CV Parsing module to extract text from PDF documents.

Key Features:

- Extracts text from PDF resumes so that the spaCy model can process them.
- Efficiently handles PDF text extraction from various file formats and layouts.

scikit-learn

machine learning library in Python.

Usage in Project:

- Similarity calculation module

- Specifically used in the Job Recommendation module to compute cosine similarity between embeddings.

Key Features:

- Offers a suite of metrics and utilities (like cosine similarity) for comparing high-dimensional vectors.
- Helps quantify the match between candidate profiles and job descriptions.

Regular Expressions (re)

Python module for working with regular expressions.

Usage in Project:

- Utilized across various modules (especially in the AI-Generated Quiz System) to clean up and extract structured data from raw text output (e.g., removing markdown formatting or internal reasoning text).

Key Features:

- Provides a flexible and powerful way to parse and validate text, ensuring that the output adheres to a consistent structure

Ollama Client

A client library for interfacing with local transformer-based models (e.g., Deepseek-r1:1.5b).

Usage in Project:

- Integrated within the AI-Generated Quiz System to generate test questions from a local model.

Key Features:

- Allows local deployment of advanced NLP models, avoiding external API costs.
- Supports real-time question generation with customizable prompts and strict output formatting.

Sentence Transformers

A library that provides pre-trained transformer models (e.g., all-MiniLM-L6-v2) for generating embeddings from text. In the job recommendation module, it converts candidate profiles and job descriptions into high-dimensional vectors used for similarity computations.

Usage in Project:

- Job Matching: Compares job seeker profiles with job descriptions.
- Provides efficient, ready-to-use models for real-time embedding generation.

Key Features:

- Cosine similarity for relevance scoring
- Pre-trained models for accurate embeddings

6.2.5. Deployment & Source Control

Docker

Docker is a containerization tool that allows microservices to run in isolated environments.

Usage in Project:

- Each microservice runs inside a Docker container.
- Docker Compose is used for orchestrating services.

GitHub

GitHub is a Git-based source control system that enables collaborative software development.

Usage in Project:

- Manages backend, frontend, and AI processing services.
- Continuous Integration/Deployment (CI/CD) pipelines.

Key Features:

- Branching strategy for collaborative development.
- Pull requests & code reviews for quality assurance.
- GitHub Actions for automated CI/CD pipelines.

6.3. AI-Powered Features Implementation

6.3.1. Resume Parsing:

- **Data Preparation & Model Training** (Conceptual Steps):
  - **Annotation:**
    Annotators label resumes: Data taken after annotation is ready
  - **DocBin Creation:**
    Convert the labeled data into spaCy's DocBin with a custom script.
- **Train/Dev Split:**
  A portion of the data is reserved for testing/validation.
  - **spaCy Training:**
  - python -m spacy train config.cfg --output output_directory --paths.train train_data.spacy --paths.dev test_data.spacy --gpu-id 0
- **FastAPI Endpoints:**
  - @app.post("/extract-data-from-pdf")
  - def extract_data_from_pdf(file: UploadFile = File(...)):.....

6.3.2. Job Recommendation

- **Embedding Generation:**
  The endpoint /generate-embedding/ takes raw text and uses SentenceTransformer to generate an embedding. The embedding is then serialized using pickle and base64-encoded for safe transmission or storage.
- **Similarity Calculation:**
  The endpoint /calculate-similarity/ accepts two base64-encoded embeddings (one for the candidate and one for the job). It decodes and deserializes these embeddings and computes the cosine similarity, converting the score to a percentage.

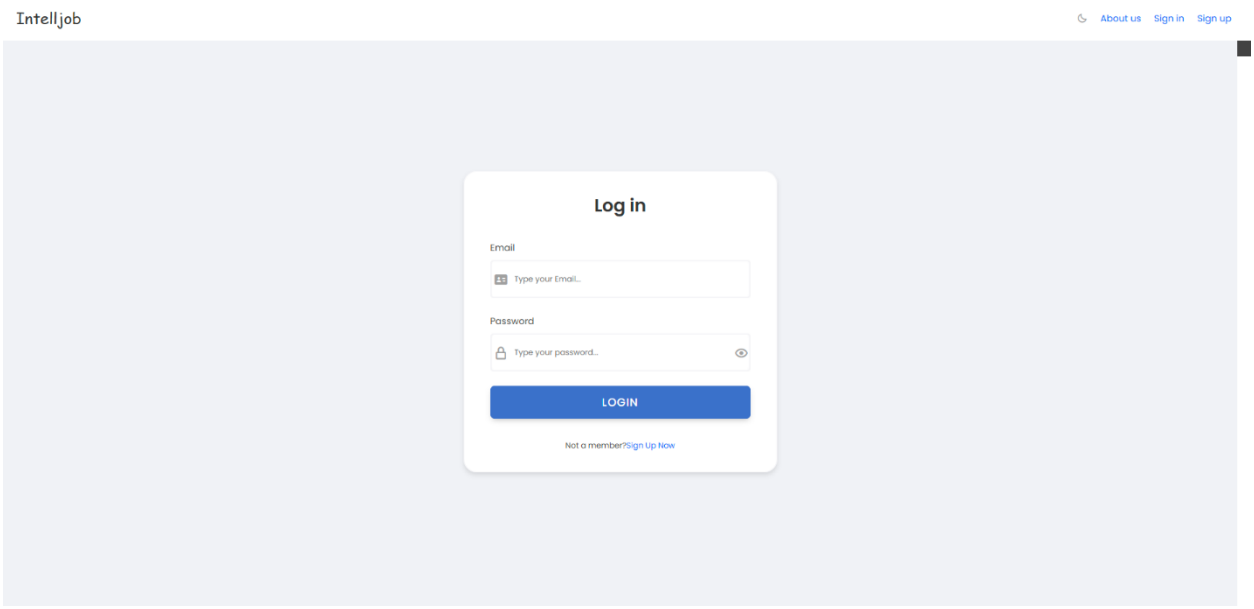
- **Integration:**
  These endpoints can be called from a client (or another service) to first generate embeddings and later compare them, forming the basis of a job recommendation system.

6.3.3. AI-Generated Quiz System.

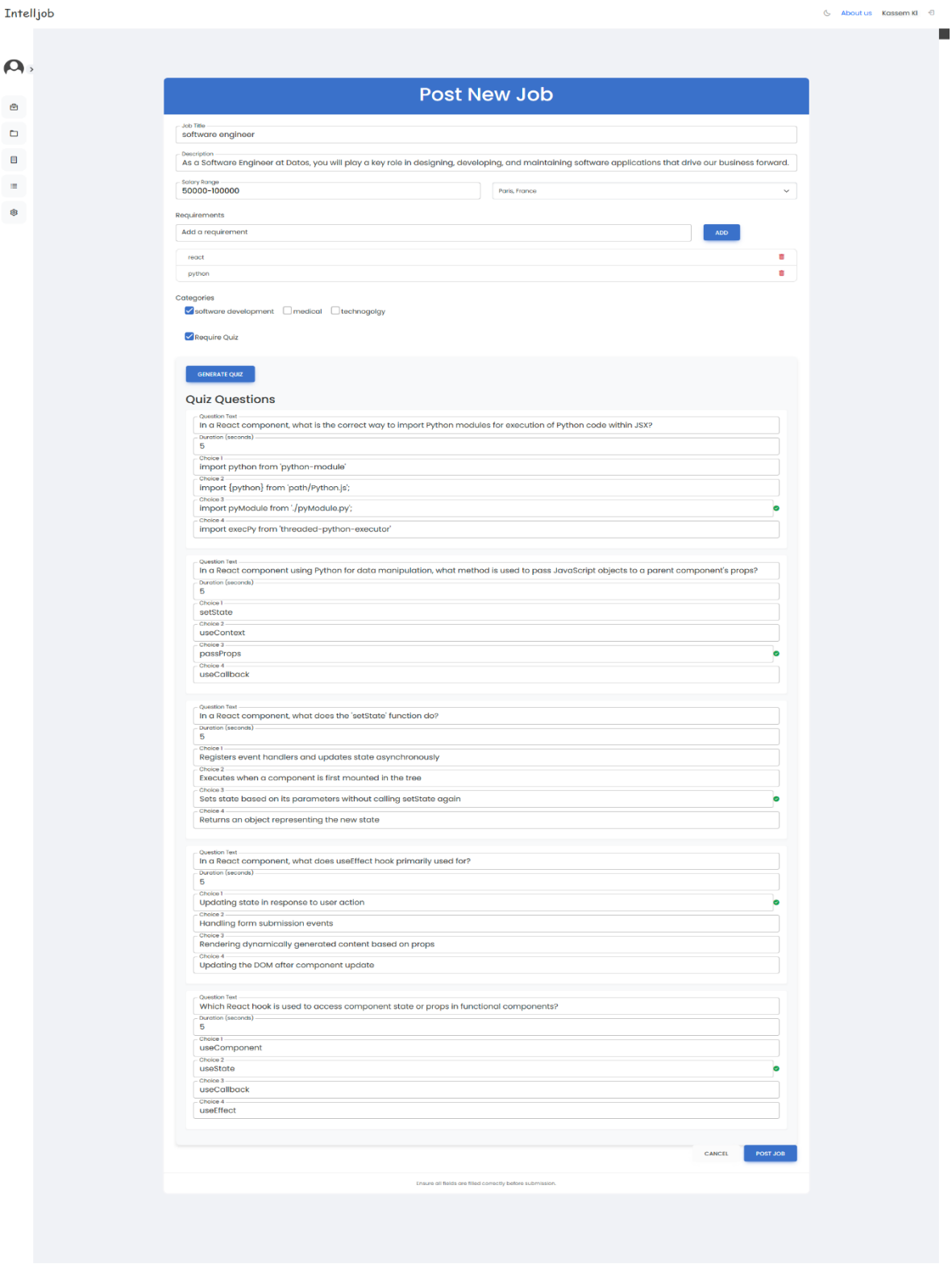
- **Prompt Engineering:**
  The prompt is carefully constructed to instruct the model on the exact output format. It includes an example JSON object and emphasizes that only the JSON output (without markdown or extra commentary) should be returned.
- **Output Parsing:**
  The parse_output_to_json function cleans the raw output by removing markdown formatting and <think> sections, then uses a regex pattern to extract a JSON block. It validates that the JSON contains the required keys and that the list of options contains exactly four items. If any part is missing or misformatted, default values are assigned.
- **API Endpoint:**
  The FastAPI endpoint /generate-questions/ accepts a JSON request with parameters for the skill, difficulty, question type, number of questions, and model. It returns a JSON response with the generated questions.

6.4. Some of System Interfaces:

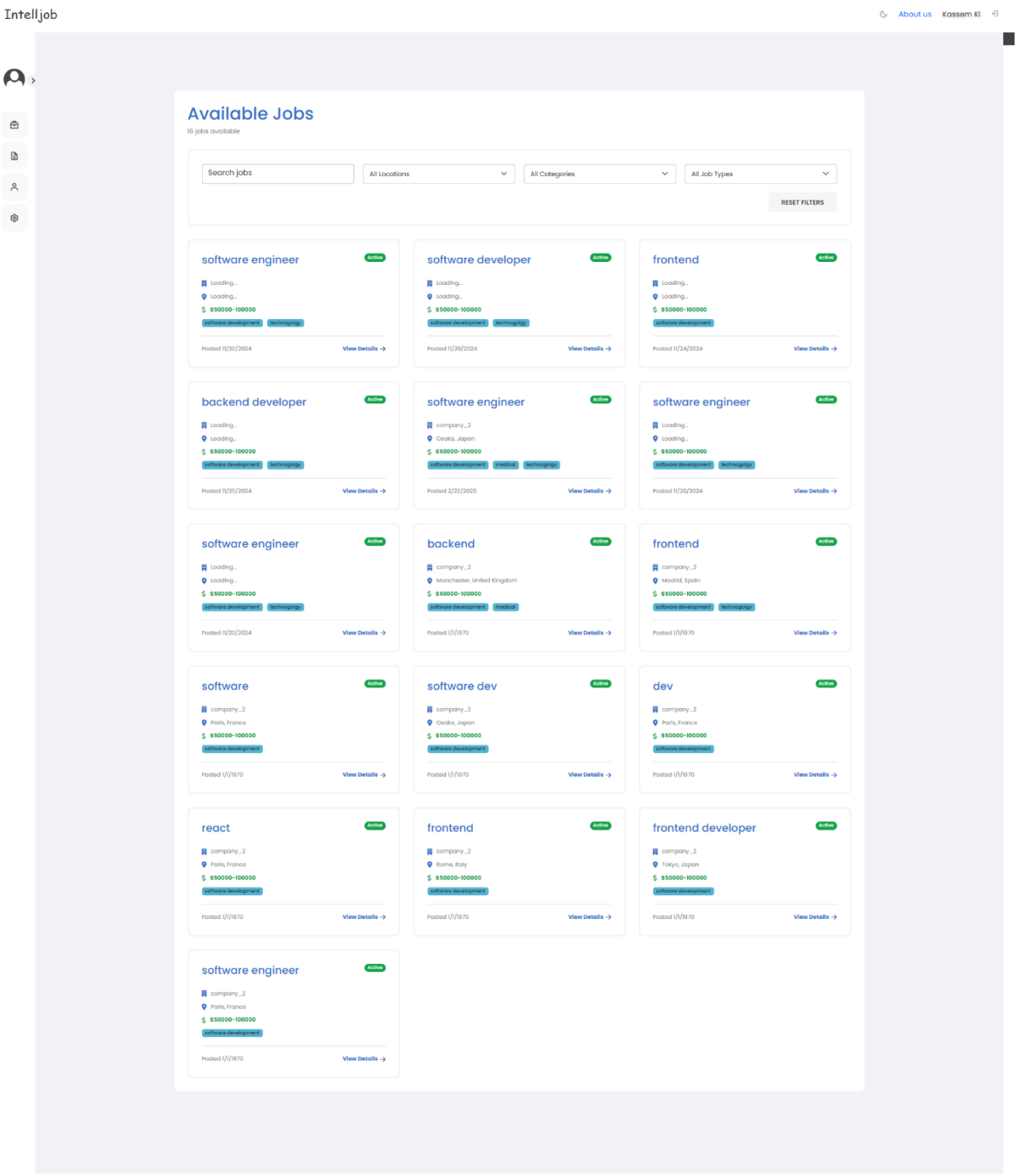
6.4.1.1. Sign in page :



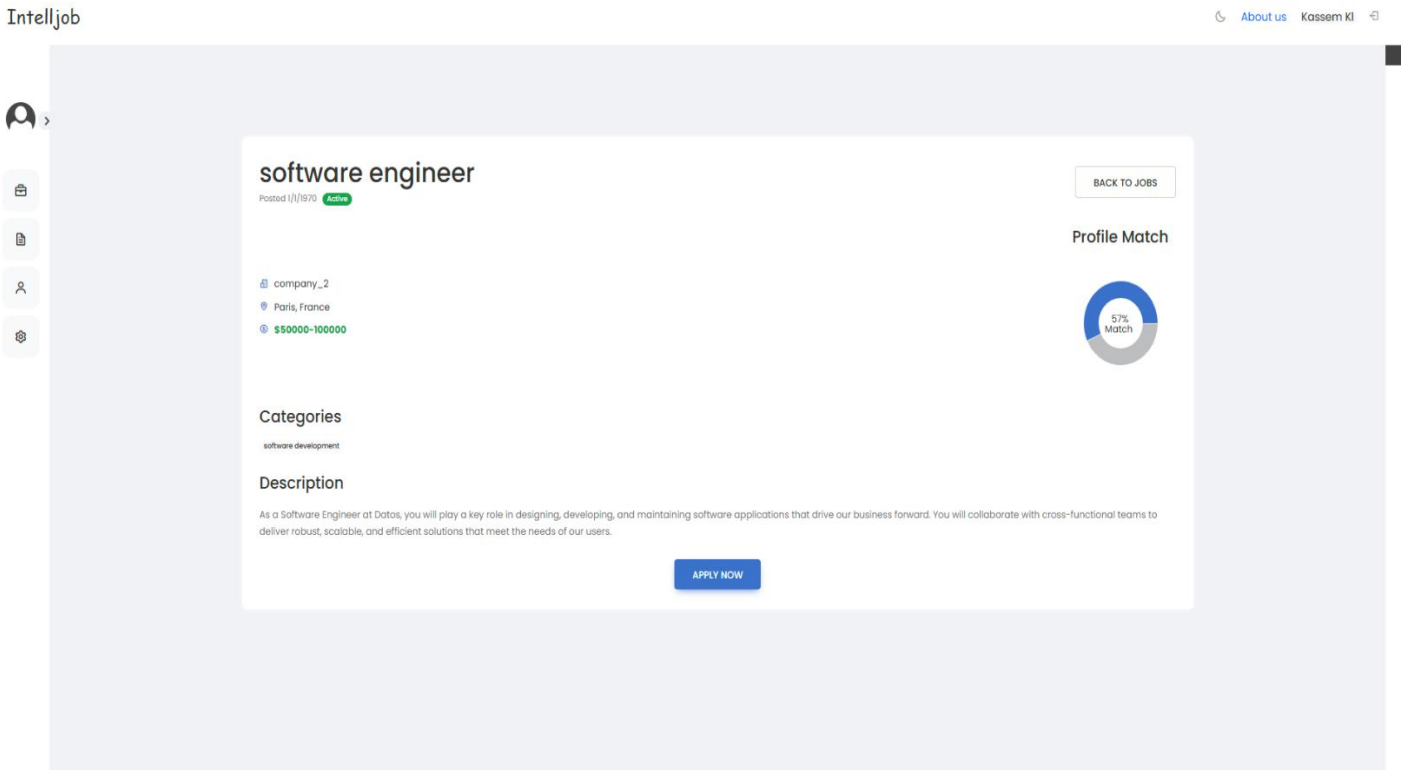
6.4.1.2. Job creation page for company :



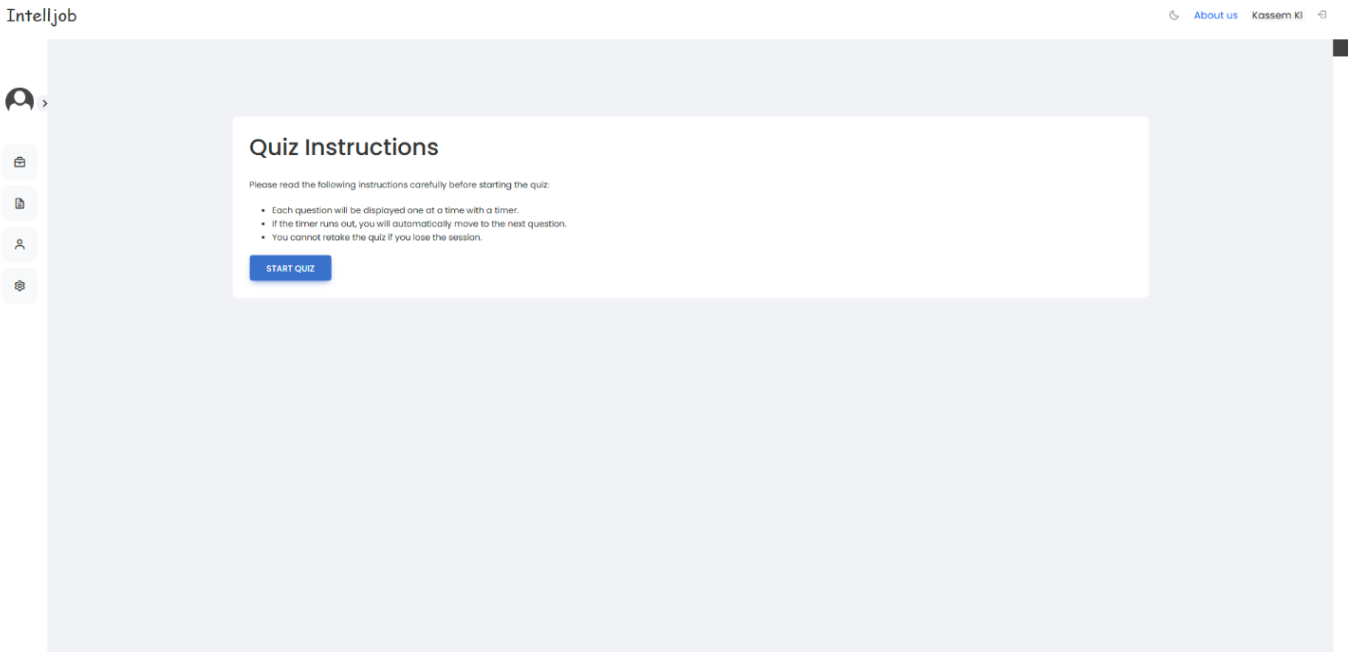
6.4.1.3. Jobs list page :



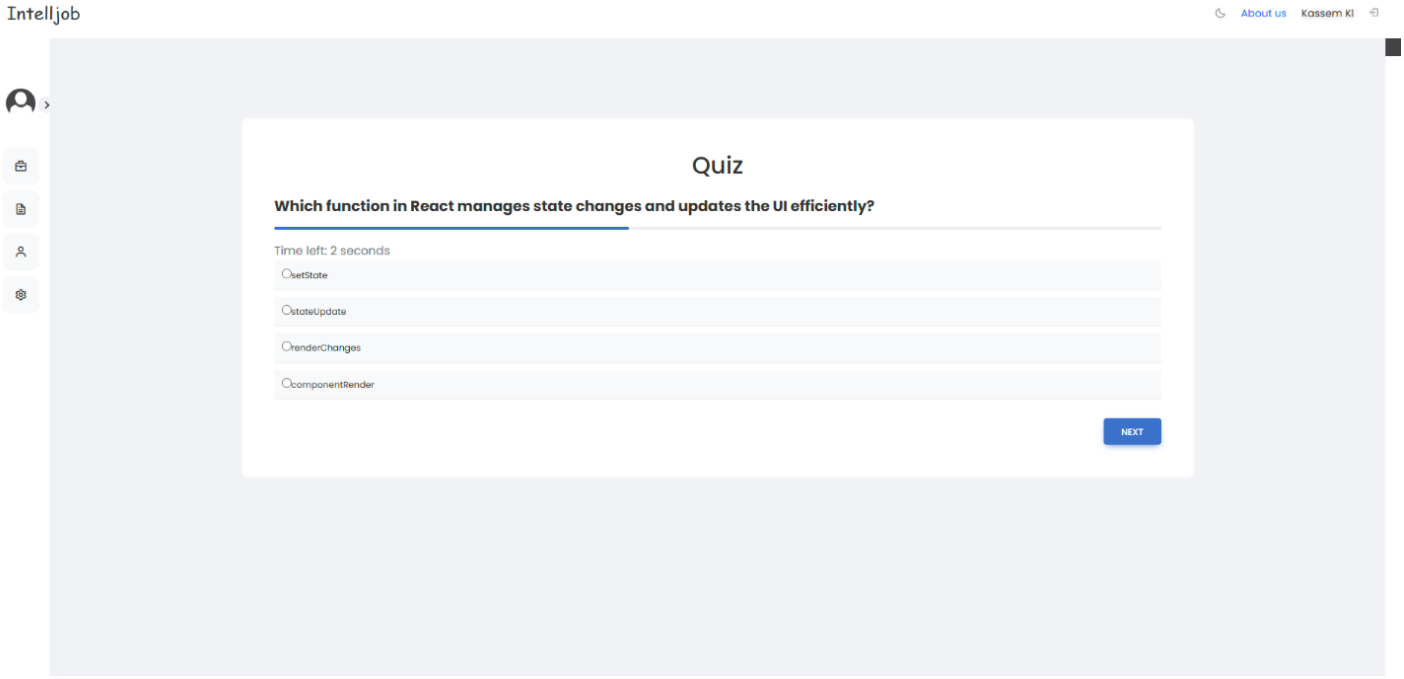
6.4.1.4. Job details page for job seeker :



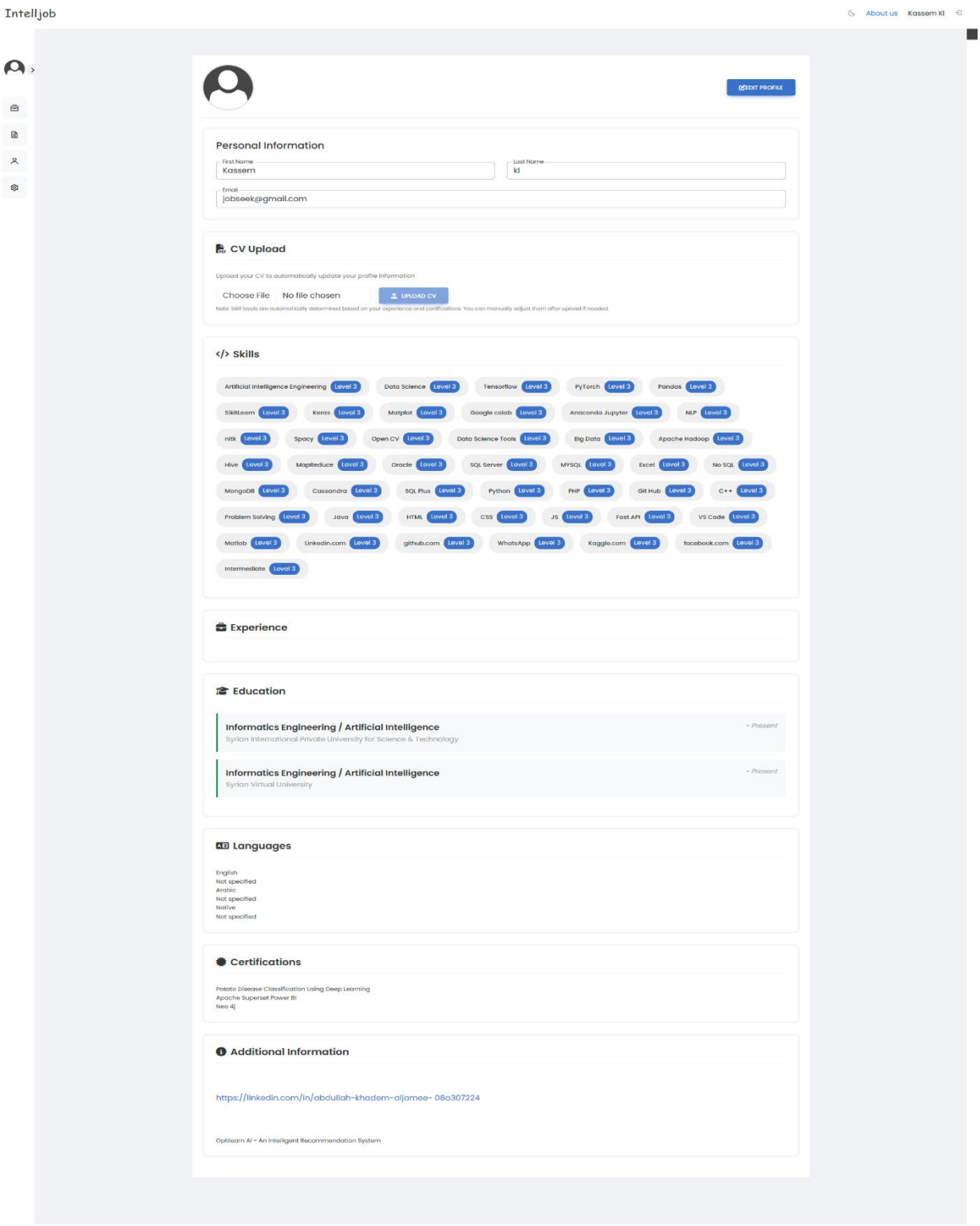
6.4.1.5. Quiz guide page for job seeker:



6.4.1.6. Quiz page for job seeker:



6.4.1.7. Profile page for job seeker:



6.5. Test Cases Execution:

Test case scenario:		Sce-01: Manage Users (Admin)			
Test case id	Test case title	Test steps	Expected result	Actual result	Pass/ fail
Tc-01	Verify successful user management	1. Log in as admin. 2. Navigate to "Manage Users". 3. Add/Edit/Delete a user.	User successfully added/edited/deleted.	User successfully added/edited/deleted.	Pass
Tc-02	Attempt to delete a non-existing user	1. Log in as admin. 2. Navigate to "Manage Users". 3. Try to delete a non-existing user.	Error message "User not found".	Error message "User not found".	Pass

Test case scenario:		Sce-02: Manage Profile (Job Seeker)			
Test case id	Test case title	Test steps	Expected result	Actual result	Pass/ fail
Tc-03	Update profile information	1. Log in as job seeker. 2. Navigate to "Manage Profile". 3. Edit personal details. 4. Save changes.	Profile successfully updated.	Profile successfully updated.	Pass

Tc-04	Attempt to save an empty profile	1. Log in as job seeker. 2. Navigate to "Manage Profile". 3. Clear all fields. 4. Save changes.	Error message "Profile fields cannot be empty".	Error message "Profile fields cannot be empty".	Pass

Test case scenario:		**Sce-03: Upload CV (Job Seeker)**			
Test case id	Test case title	Test steps	Expected result	Actual result	Pass/ fail
Tc-05	Upload a valid CV file	1. Log in as job seeker. 2. Navigate to "Upload CV". 3. Choose a valid CV file (PDF/DOC). 4. Click "Upload".	CV uploaded successfully.	CV uploaded successfully.	Pass
Tc-06	Upload an invalid file type	1. Log in as job seeker. 2. Navigate to "Upload CV". 3. Choose an unsupported file format (e.g., PNG). 4. Click "Upload".	Error message "Invalid file format".	Error message "Invalid file format".	Pass

Test case scenario:	**Sce-04: Search Jobs (Job Seeker)**		

Test case id	Test case title	Test steps	Expected result	Actual result	Pass/ fail
Tc-07	Perform a job search with valid keywords	1. Log in as job seeker. 2. Enter job title or keyword in the search bar. 3. Click "Search".	Relevant job results displayed.	Relevant job results displayed.	Pass
Tc-08	Perform a job search with invalid keywords	1. Log in as job seeker. 2. Enter random or incorrect keywords. 3. Click "Search".	Error message "No jobs found".	Error message "No jobs found".	Pass

Test case scenario:	**Sce-05: Apply for Jobs (Job Seeker)**				
Test case id	Test case title	Test steps	Expected result	Actual result	Pass/ fail
Tc-09	Apply for a job successfully	1. Log in as job seeker. 2. Search for a job. 3. Click "Apply". 4. Confirm application.	Application submitted successfully.	Application submitted successfully.	Pass
Tc-10	Apply for a job without a CV	1. Log in as job seeker. 2. Search for a job. 3. Click "Apply" without uploading a CV.	Error message "CV required".	Error message "CV required".	Pass

Test case scenario:		Sce-06: View Application Status (Job Seeker)			
Test case id	Test case title	Test steps	Expected result	Actual result	Pass/ fail
Tc-11	Check status of submitted job applications	1. Log in as job seeker. 2. Navigate to "Application Status".	Status of all job applications displayed.	Status of all job applications displayed.	Pass

Test case scenario:		Sce-07: Manage Job Post (Company)			
Test case id	Test case title	Test steps	Expected result	Actual result	Pass/ fail
Tc-12	Post a new job	1. Log in as company. 2. Navigate to "Manage Job Post". 3. Fill in job details. 4. Click "Post".	Job post successfully added.	Job post successfully added.	Pass
Tc-13	Delete an existing job post	1. Log in as company. 2. Navigate to "Manage Job Post". 3. Delete an existing job.	Job post successfully deleted.	Job post successfully deleted.	Pass

Test case scenario:		Sce-08: View Job Seeker Profiles (Company)			
Test case id	Test case title	Test steps	Expected result	Actual result	Pass/ fail
Tc-14	View job seeker profile	1. Log in as company. 2. Search for a candidate. 3. Click on a candidate profile.	Candidate profile details displayed.	Candidate profile details displayed.	Pass

Test case scenario:		Sce-09: Communicate with Candidate (Company)			
Test case id	Test case title	Test steps	Expected result	Actual result	Pass/ fail
Tc-15	Send a message to a candidate	1. Log in as company. 2. Search for a candidate. 3. Open messaging section. 4. Send a message.	Message successfully sent.	Message successfully sent.	Pass

Test case scenario:		Sce-10: Match with Job (AI)			
Test case id	Test case title	Test steps	Expected result	Actual result	Pass/ fail
Tc-16	AI matches candidate with job	1. Log in as job seeker. 2. Upload CV. 3. AI suggests jobs based on profile.	Relevant job suggestions displayed.	Relevant job suggestions displayed.	Pass

Test Case Scenario:		Sce-01: Check Login Functionality.			
Test case id	Test case title	Test steps	Expected result	Actual result	Pass/ fail
Tc-17	Check results on entering a valid email and password.	4. Launch the application on the login page. 5. Enter your email and password. 6. Choose “login”.	The login should be successful	The login should be successful	Pass

Tc-18	Check results on entering an invalid email, or password.	4. Launch the application on the login page. 5. Enter an ID and password. 6. Choose “login”.	Error message “invalid id or password.”	Error message “invalid id or password.”	Pass
Tc-19	Check results when a user email is empty, and the “login” button is pressed.	4. Launch the application on the login page. 5. Enter a password. 6. Choose “login”.	Error message “a field is missing”	Error message “a field is missing”	Pass

Test case scenario:		**Sce-02: Check sign-up functionality.**			
Test case id	Test case title	Test steps	Expected result	Expected result	Pass/ fail

Tc-20	Check results on completing the sign-up form correctly, and select sign-up	4. Launch the application on the sign-up page. 5. Complete the form of your information. 6. Choose “sign-up”	The sign-up should be successful.	The sign-up should be successful.	Pass
Tc-21	Check results on entering an existing email.	4. Launch the application on the sign-up page. 5. Complete the form of your information. 6. Choose “sign-up”	Error message “the email is already existed”.	Error message “the email is already existed”.	Pass
Tc-22	Check results when a user chose a weak password (less than 8 characters).	4. Launch the application on the sign-up page. 5. Complete the form of your information. 6. Choose “sign-up”	Error message “weak password!”	Error message “weak password!”	Pass
Tc-23	Check results when a user email is empty, and the “login” button is pressed.	4. Launch the application on the sign-up page. 5. Complete the form of your information. 6. Choose “sign-up”	Error message “a field is missing”	Error message “a field is missing”	Pass

6.6. RTM Last Version:

Req-id	Use cases	analysis	System design	Detailed design	coding	Test cases
RE-FR-JS-1	UC-01: Create Account	analysis	System design	Detailed design	code	TC-01
RE-FR-JS-2	UC-01: Create Account	analysis	System design	Detailed design	code	TC-02
RE-FR-JS-3	UC-01: Create Account	analysis	System design	Detailed design	code	TC-03
RE-FR-JS-4	UC-01: Create Account	analysis	System design	Detailed design	code	TC-04
RE-FR-JS-5	UC-01: Create Account	analysis	System design	Detailed design	code	TC-05
RE-FR-JS-6	UC-01: Create Account	analysis	System design	Detailed design	code	TC-06
RE-FR-JS-7	UC-02: Upload CV	analysis	System design	Detailed design	code	TC-07
RE-FR-JS-8	UC-02: Upload CV, UC-10: CV Parsing	analysis	System design	Detailed design	code	TC-08
RE-FR-JS-9	UC-03: Apply for Jobs, UC-10: Match with Job	analysis	System design	Detailed design	code	TC-09

RE-FR-JS-10	UC-11: Search Jobs	analysis	System design	Detailed design	code	TC-10
RE-FR-JS-11	UC-11: Search Jobs	analysis	System design	Detailed design	code	TC-11
RE-FR-JS-12	UC-03: Apply for Jobs	analysis	System design	Detailed design	code	TC-12
RE-FR-JS-13	UC-03: Apply for Jobs	analysis	System design	Detailed design	code	TC-13
RE-FR-JS-14	UC-07: Track Job Application	analysis	System design	Detailed design	code	TC-14
RE-FR-JS-15	UC-01: Create Account	analysis	System design	Detailed design	code	TC-15
RE-FR-JS-16	UC-11: Search Jobs	analysis	System design	Detailed design	code	TC-16
RE-FR-CO-17	UC-04: Manage Users	analysis	System design	Detailed design	code	TC-17
RE-FR-CO-18	UC-05: Manage Job Post	analysis	System design	Detailed design	code	TC-18
RE-FR-CO-19	UC-06: Search for Candidates	analysis	System design	Detailed design	code	TC-19
RE-FR-CO-20	UC-08: Manage Candidate Application	analysis	System design	Detailed design	code	TC-20
RE-FR-CO-21	UC-06: Search for Candidates	analysis	System design	Detailed design	code	TC-21
RE-FR-CO-22	UC-14: Review Quiz Results	analysis	System design	Detailed design	code	TC-22

RE-FR-CO-23	UC-06: Search for Candidates	analysis	System design	Detailed design	code	TC-23
RE-FR-CO-24	UC-09: Communication Between Job Seeker and Company	analysis	System design	Detailed design	code	TC-24
RE-FR-CO-25	UC-04: Manage Users	analysis	System design	Detailed design	code	TC-25
RE-FR-CO-26	UC-04: Manage Users	analysis	System design	Detailed design	code	TC-26
RE-FR-AI-27	UC-10: Match with Job	analysis	System design	Detailed design	code	TC-27
RE-FR-AI-28	UC-10: CV Parsing	analysis	System design	Detailed design	code	TC-28
RE-FR-AI-29	UC-05: Manage Job Post	analysis	System design	Detailed design	code	TC-29
RE-FR-SYS-30	UC-07: Track Job Application	analysis	System design	Detailed design	code	TC-30
RE-FR-SYS-31	UC-04: Manage Users	analysis	System design	Detailed design	code	TC-31
RE-FR-SYS-32	UC-04: Manage Users	analysis	System design	Detailed design	code	TC-32
RE-FR-SYS-33	UC-04: Manage Users	analysis	System design	Detailed design	code	TC-33
RE-FR-CH-34	UC-09: Communication Between Job Seeker and Company	analysis	System design	Detailed design	code	TC-34

RE-FR-CH-35	UC-09: Communication Between Job Seeker and Company	analysis	System design	Detailed design	code	TC-35
RE-FR-QZ-36	UC-13: Monitor Quizzes	analysis	System design	Detailed design	code	TC-36
RE-FR-QZ-37	UC-12: Take Quiz	analysis	System design	Detailed design	code	TC-37

Chapter7 Evaluations, Results and Discussions

7.1. Resume Parsing:

7.1.1. Evaluations

After training the NER model, three main sets of results are presented:

7.1.1.1. Training Logs:

Below are excerpts from the spaCy training pipeline logs:

```
============================= Training pipeline =============================
[i] Pipeline: ['transformer', 'ner']
[i] Initial learn rate: 0.0
```

E	#	LOSS TRANS...	LOSS NER	ENTS_F	ENTS_P	ENTS_R	SCORE
0	0	1214.57	2002.25	0.01	0.00	0.05	0.00
1	200	367117.65	89929.10	30.38	32.92	28.20	0.30
2	400	128840.63	34804.36	59.87	64.01	56.23	0.60
3	600	21922.74	16360.59	78.81	78.60	79.03	0.79
4	800	11118.55	10203.22	82.18	84.34	80.13	0.82
5	1000	16659.44	8927.76	84.68	82.58	86.89	0.85
6	1200	7602.92	7835.25	85.73	85.98	85.48	0.86
7	1400	5590.74	6332.84	83.73	86.33	81.29	0.84
8	1600	6147.86	6517.54	85.94	85.97	85.92	0.86
10	1800	5727.95	6024.12	84.81	82.84	86.88	0.85
11	2000	9679.55	5333.93	85.55	85.01	86.10	0.86
12	2200	3808.60	5293.23	86.40	86.13	86.67	0.86
13	2400	3047.89	4610.97	85.68	84.86	86.52	0.86
14	2600	12019.88	4658.71	85.43	83.54	87.40	0.85
15	2800	3547.79	4563.97	85.66	86.63	84.70	0.86
16	3000	3692.27	3842.94	85.63	85.30	85.97	0.86
18	3200	4701.99	4466.81	84.21	82.66	85.81	0.84
19	3400	2689.78	3470.77	86.17	85.61	86.73	0.86
20	3600	2524.08	3615.91	84.93	82.93	87.03	0.85
21	3800	2195.86	3509.11	85.73	84.43	87.07	0.86

- **Loss Decrease:** Both transformer and NER losses decrease significantly, indicating successful training.
- **F1-Score Growth:** ENTS_F climbs from near 0% to ~85–86%, showing good convergence.
- **Stabilization:** By epoch ~6–8, performance stabilizes around the mid-80s F1 range.

7.1.1.2. Precision/Recall/F1-Scores (Per Label)

	precision	recall	f1-score	support
AWARDS	0.80	0.31	0.44	91
CERTIFICATION	0.94	0.15	0.26	511
COLLEGE NAME	0.77	0.28	0.41	182
COMPANIES WORKED AT	0.87	0.11	0.20	1116
DEGREE	0.97	0.76	0.85	419
EMAIL ADDRESS	0.87	0.81	0.84	16
LANGUAGE	0.91	0.17	0.28	246
LINKEDIN LINK	0.99	0.99	0.99	305
LOCATION	0.74	0.05	0.10	312
NAME	0.79	0.04	0.07	314
O	0.00	0.00	0.00	0
SKILLS	0.60	0.15	0.23	1478
UNIVERSITY	0.79	0.59	0.68	244
Unlabelled	0.00	0.00	0.00	0
WORKED AS	0.84	0.19	0.31	1290
YEAR OF GRADUATION	0.99	0.63	0.77	139
YEARS OF EXPERIENCE	1.00	0.81	0.90	1256
accuracy			0.34	7919
macro avg	0.76	0.35	0.43	7919
weighted avg	0.84	0.34	0.43	7919

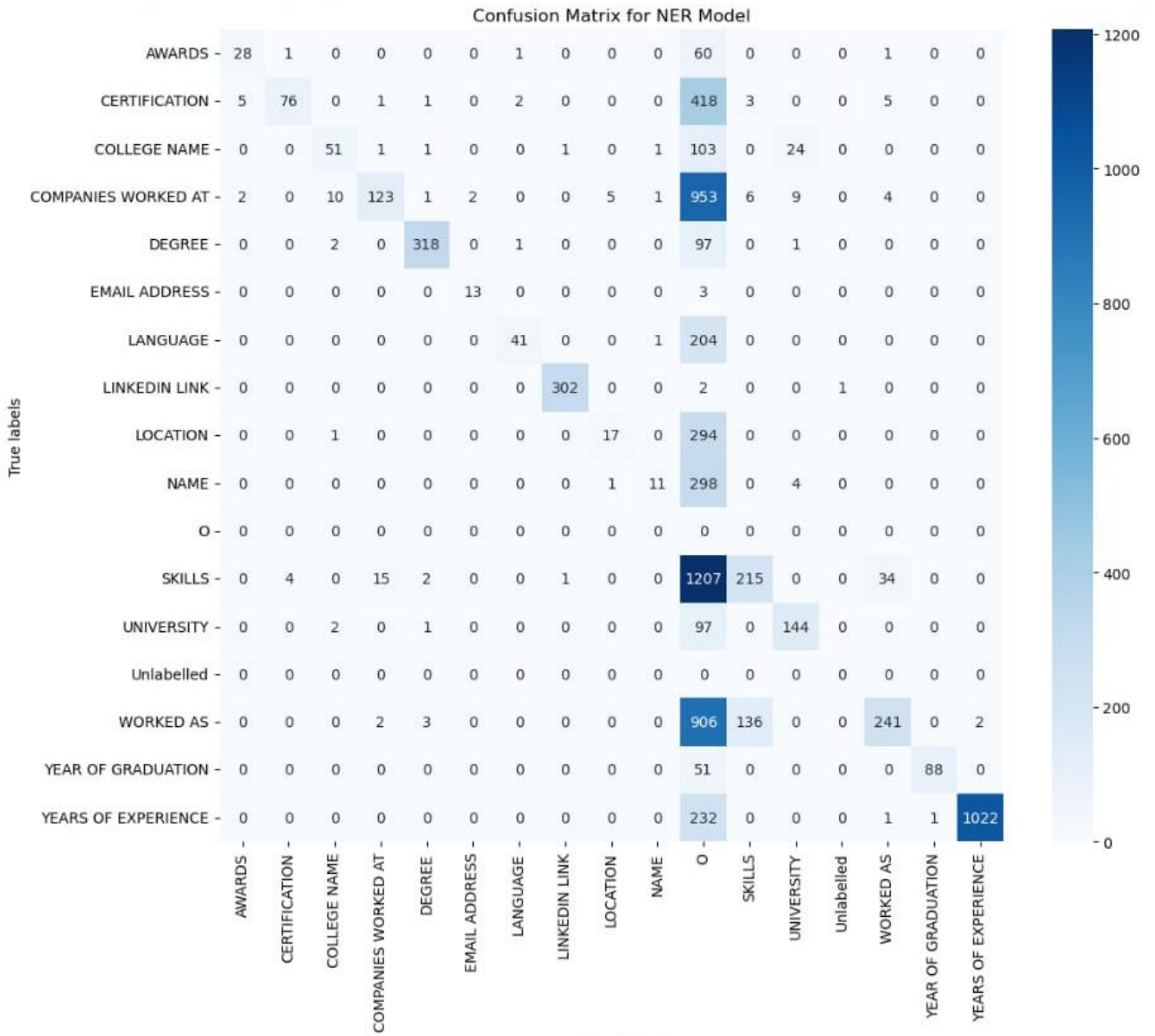
- **High-Performing Labels:** EMAIL ADDRESS, LANGUAGE, LINKEDIN LINK, and DEGREE exhibit F1-scores around 0.80–0.90, suggesting sufficient data and consistent annotations.
- **Moderate Labels:** COMPANIES WORKED AT (F1 ~ 0.79) and WORKED AS (F1 ~ 0.74) perform reasonably well but have room for improvement.
- **Low-Performing Labels:** AWARDS, CERTIFICATION, and COLLEGE NAME show F1-scores around 0.40–0.44, likely due to fewer training examples or more ambiguous contexts.

- **Macro vs. Weighted Averages:** The gap between macro (F1 ~ 0.43) and weighted average (F1 ~ 0.59) indicates class imbalance, where frequent classes dominate overall performance.

7.1.1.3. Confusion Matrix

The confusion matrix (truncated for brevity) reveals how often each label is predicted as itself or confused with another label:

- **Diagonal Values:** Large diagonal values (e.g., 1022 for YEARS OF EXPERIENCE) indicate correct predictions for high-frequency labels.
- **Misclassifications:** Rare labels (e.g., AWARDS, CERTIFICATION) occasionally get classified as more common labels or are mislabeled as UNLABELED.



7.1.2. Discussion

- **Performance**:

  The final model achieves an F1-score in the mid-80s for the most frequent labels, which is typically strong for a custom NER domain. Some labels approach ~90% F1, indicating robust extraction for well-represented classes.

- **Class Imbalance:**The significantly lower F1-scores for AWARDS, CERTIFICATION, and COLLEGE NAME highlight the impact of fewer training samples or more ambiguous contexts. Improving coverage for these labels would likely boost their performance.
- **Error Sources:**
  - **Annotation Inconsistency:** Overlapping or misaligned spans can degrade performance.
  - **Ambiguous Phrases:** Certain job roles might be mistaken for skills, or university names might be confused with location entities.

- **Future Directions:**

  - **Data Augmentation:** Acquire more annotated resumes specifically featuring underrepresented entity types.
  - **Fine-Tuning:** Explore domain-specific language models or further training epochs to handle rarer classes.
  - **Active Learning:** Integrate an active learning loop to refine annotations and correct misclassifications in real-time.

7.2. Job Recommendation

7.2.1. Evaluations

- **Performance Metrics**
  - **Embedding Quality:**
    Evaluate the semantic quality of the embeddings using qualitative assessments or benchmark datasets.
  - **Similarity Accuracy:**
    Validate the cosine similarity scores against a manually curated dataset of candidate-job pairs to ensure the percentage scores accurately reflect match quality.
- **Efficiency**
  - **Inference Time:**
    Measure the time required to generate embeddings and calculate similarity. SentenceTransformer models like all-MiniLM-L6-v2 are known for their efficiency on GPU and CPU.
  - **Resource Usage:**
    Monitor GPU and CPU utilization during inference to ensure that the model can handle real-time requests.
- **Scalability**
- **Batch Processing:**
  For large datasets, embeddings can be computed in batches.

- **Periodic Updates:**
  Consider scheduling periodic re-computation of embeddings and similarity scores to reduce real-time load.

7.2.2. Discussion

- **Strengths**

- **Cost Efficiency:**
  Using an open-source Sentence Transformer model locally avoids per-request API costs.
- **Flexibility:**
  The API design allows embeddings to be generated and compared independently, which supports integration with various systems.
- **Semantic Matching:**
  Cosine similarity provides a robust measure of semantic similarity, which can be fine-tuned further with domain-specific data.

- **Challenges**

- **Storage of High-Dimensional Embeddings:**
  While embeddings can be serialized and stored in databases (e.g., MySQL using BLOBs), handling large numbers of high-dimensional vectors efficiently remains a challenge.
- **Dynamic Updates:**
  Candidate and job data change over time; incorporating real-time or periodic updates is necessary to keep recommendations current.

- **Evaluation:**
  Ground truth for matching quality can be subjective and may require human validation to calibrate similarity thresholds.

7.3. AI-Generated Quiz System.

7.3.1. Evaluations

- **Metrics for Question Generation**

Evaluating a question-generation system can be challenging. Consider the following evaluation methods:

  - **Human Evaluation:**
    Recruiters or domain experts review generated questions for clarity, relevance, and difficulty. This qualitative feedback is crucial for fine-tuning prompt parameters.
  - **Automated Metrics:**
    - **BLEU/ROUGE:** These metrics compare generated questions to a reference set, though they may not fully capture creativity or correctness.
    - **Consistency Checks:** Validate that the output JSON consistently contains the expected keys and that the correct answer is provided as a single option number.
  - **User Acceptance Testing:**
    Deploy the system in a pilot phase to gather feedback on how well the questions assess candidate skills in a real-world recruitment scenario.

- **Experimental Setup**
    - **Data:**
      Use a set of predefined skills and difficulty levels to generate a batch of questions.
    - **Testing:**
      Evaluate the stability of the output format, measure response times, and gather qualitative feedback from users.

7.3.2. Discussion

Strengths

- **Consistent Output:**
  The detailed prompt and robust parsing ensure that each question is output in a fixed JSON format.
- **Flexibility:**
  The system supports different question types and difficulty levels, allowing for dynamic test generation.
- **Local Operation:**
  Using a local model via Ollama (e.g., deepseek-r1:1.5b) avoids external API costs and enables real-time generation.

Challenges

- **Variability in Model Output:**
  Despite robust prompts, transformer models may still occasionally produce unexpected output that requires further post-processing.
- **Quality Control:**
  Ensuring that distractors are plausible and that the correct answer is

consistently provided as an option number is non-trivial and may require manual review during early deployment.

- **Evaluation Complexity:**
  Automated metrics like BLEU/ROUGE may not fully capture the quality and relevance of generated questions, necessitating a blend of automated and human evaluation.

Chapter8 Conclusion

References

Banerjee, V. K. (2020). “Scalable Techniques for Real-Time Resume Parsing in Large Enterprises.”. *IEEE Access, vol. 8*, 154976–154986.

Çelik, D. e. (2019). "Ontology-Based Resume Parsing for Enhanced Candidate Matching.". *IEEE Transactions on Knowledge and Data Engineering,* , 783-792.

Das, P. e. (2019). "Automated Resume Parsing using Text Analytics.". *Proceedings of the IEEE International Conference on Big Data*, 321–326.

Devlin, J. (2019). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” . *NAACL-HLT*.

Devlin, J. C.-W. (2019). “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.” . *NAACL-HLT*.

Du, X. (2017). “Automatic Question Generation for Reading Comprehension: A Survey.”. *Journal of Artificial Intelligence Research*.

Huang, G. (2019). “Deep Learning for Candidate Matching in Job Recommendation Systems.”. *IEEE Transactions on Neural Networks and Learning Systems*.

Hugging Face Transformers Documentation – https://huggingface.co/docs/transformers. (n.d.).

Jadhav, A. M. (2016). "Deep Learning for Information Extraction from Resumes.". *International Journal of Computer Applications, vol.146*, 25-32.

Koren, Y. B. (2009). Matrix Factorization Techniques for Recommender Systems. *Computer*.

Li, Y. (2018). “Semantic Matching for Job Recommendation.”. *Proceedings of the AAAI Conference on Artificial Intelligence*.

Mittal, V. (2018). “Hybrid Approaches to Resume Parsing and Domain-Specific Question Generation.”. *Journal of Information Processing Systems, vol. 14, no. 2*, 210-221.

Mittal, V. (2018). “Hybrid Approaches to Resume Parsing and Domain-Specific Question Generation.” . *Journal of Information Processing Systems, vol. 14, no. 2*, 210–221.

Reimers, N. &. (2019). “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.”. *EMNLP* .

Reimers, N. &. (2019). “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.”. *EMNLP* .

Zhou, M. e. (2020). “Controlled Natural Language Generation for Question Answering.”. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing.*